# 具身智能求职手册

# 具身智能求职手册

## 岗位、面试题和不买真机的作品集

从看懂招聘启事，到拿下第一个 offer

甜菜　著

**具身智能求职手册：岗位、面试题和不买真机的作品集**

甜菜　著

2026 年 10 月第 1 版（样书）

**声明**　本书为独立写作。书中提到的公司和产品名称是其各自所有者的商标；对公司的描述只依据官网、公开报告和公开报道，不构成对任何公司的评价或推荐。书中的招聘启事、简历和面试对话除特别注明外，都是按常见写法改写或虚构的示例。

**数据**　书中数据截至 2026 年 10 月，主要来自公开的招聘报告、公司官网和公开招聘启事，出处见附录 E。招聘和薪资情况变化很快，请以最新信息为准。

**代码**　随书代码在 Python 3.13、4 核 CPU、无显卡的 Linux 机器上运行通过，书中的结果图和数字都来自这些运行。代码的下载地址和许可证在出版时公布。

**写作说明**　本书在资料整理、写作、代码编写和插图制作中使用了人工智能工具。

ISBN：（正式出版时填写）

# 目录

**前言：一条招聘启事，你能看懂几行？** **xi**

这本书怎么回答那条启事 . . . . . xi

这本书是一条漏斗 . . . . . xii

书里的数据和代码 . . . . . xii

怎么读 . . . . . xii

几句实话 . . . . . xiii

**第一部　看清这一行** **1**

**第 1 章　谁在招人：2026 年的具身智能人才市场** **3**

三组数字，各说明了什么 . . . . . 3

谁在招：五类雇主 . . . . . 4

一家头部公司到底在招什么 . . . . . 4

钱：薪资怎么看 . . . . . 5

门槛：学历、经验和城市 . . . . . 6

新面孔：数据岗和新专业 . . . . . 6

这对你意味着什么 . . . . . 7

**第 2 章　一张岗位地图：二十多个岗位各干什么** **9**

本体：把机器人造出来 . . . . . 9

小脑：让它动起来 . . . . . 10

感知：知道自己在哪、周围有什么 . . . . . 11

大脑：理解任务、做决定 . . . . . 11

数据与基础设施：喂饱模型 . . . . . 11

软件：让一切可靠地跑 . . . . . 12
交付：把机器人交到客户手里 . . . . . 12
怎么挑出两个目标岗位 . . . . . 12

**第 3 章 快速见效：把招聘启事拆成能力清单 15**
一条启事的三段 . . . . . 15
“精通、熟悉、了解”各是什么分量 . . . . . 16
藏在字里行间的信息 . . . . . 16
用脚本统计十几条启事 . . . . . 17
把差距排成计划 . . . . . 18

**第二部 按岗位补能力 21**

**第 4 章 所有岗位都会问的基础 23**
知识地图 . . . . . 23
面试题和参考答案 . . . . . 24
30 分钟自测 . . . . . 30

**第 5 章 运动控制岗：从 PID 到 MPC 和 WBC 33**
这个岗位的一天 . . . . . 33
招聘启事里怎么写 . . . . . 33
知识地图 . . . . . 34
面试题和参考答案 . . . . . 34
从哪些背景转过来最顺 . . . . . 40

**第 6 章 强化学习岗：奖励、PPO 和 Sim2Real 43**
这个岗位的一天 . . . . . 43
招聘启事里怎么写 . . . . . 43
知识地图 . . . . . 44
面试题和参考答案 . . . . . 44
从哪些背景转过来最顺 . . . . . 50

**第 7 章　感知与定位岗：标定、SLAM 和状态估计** **51**
这个岗位的一天 . . . . . 51
招聘启事里怎么写 . . . . . 51
知识地图 . . . . . 52
面试题和参考答案 . . . . . 52
从哪些背景转过来最顺 . . . . . 56

**第 8 章　操作与具身大模型岗：模仿学习、VLA 和数据** **59**
这个岗位的一天 . . . . . 59
招聘启事里怎么写 . . . . . 59
知识地图 . . . . . 60
面试题和参考答案 . . . . . 60
从哪些背景转过来最顺 . . . . . 65

**第 9 章　硬件岗：结构、关节、电控和嵌入式** **67**
这个岗位的一天 . . . . . 67
招聘启事里怎么写 . . . . . 67
知识地图 . . . . . 68
面试题和参考答案 . . . . . 68
从哪些背景转过来最顺 . . . . . 73

**第 10 章　软件、数据、测试和交付岗** **75**
这几个岗位每天在干什么 . . . . . 75
面试题和参考答案 . . . . . 76
从哪些背景转过来最顺 . . . . . 80

**第三部　拿到 offer** **83**

**第 11 章　作品集（上）：控制和学习** **85**
准备环境 . . . . . 85
项目一：让 Go2 站起来，比较刚度和阻尼 . . . . . 86
项目二：倒立摆，PID 和 LQR . . . . . 88
项目三：用 PPO 训练 Hopper，对照域随机化 . . . . . 89

**第 12 章 作品集（下）：感知、操作和数据** **93**
项目四：卡尔曼滤波融合 IMU 和腿式里程计 . . . . . 93
项目五：机械臂逆运动学，伪逆和阻尼最小二乘 . . . . . 95
项目六：模仿学习里的“多峰”问题 . . . . . 96
把项目整理成别人愿意看的样子 . . . . . 98

**第 13 章 简历：把过去的经验翻译成机器人公司的话** **101**
一页简历的结构 . . . . . 101
六种背景的改写对照 . . . . . 102
项目经历怎么写 . . . . . 103
最容易犯的六个错 . . . . . 104

**第 14 章 面试：从笔试到 HR 面** **107**
一般有几轮 . . . . . 107
手撕代码 . . . . . 107
八股：背还是理解 . . . . . 108
项目深挖：准备好追问链 . . . . . 108
系统设计题 . . . . . 108
行为面 . . . . . 109
反问：最后十分钟别浪费 . . . . . 109
每面一次，复盘一次 . . . . . 110

**第 15 章 选公司和谈 offer** **113**
先看公司能活多久 . . . . . 113
把 offer 拆开看 . . . . . 114
怎么谈 . . . . . 115
比较两个 offer . . . . . 115

**第 16 章 90 天行动计划** **117**
两套节奏 . . . . . 117
第 1–2 周：定方向，搭环境 . . . . . 117
第 3–6 周：补核心，做第一个项目 . . . . . 118
第 7–9 周：第二个项目，简历，刷题 . . . . . 118

第 10–11 周：投递 . . . . . 119
第 12–13 周：面试，谈 offer . . . . . 119
卡住了怎么办 . . . . . 120

**结语：如果这一波降温了** **121**
哪些能力在别处也值钱 . . . . . 121
怎么给自己留后路 . . . . . 122
回头看 . . . . . 122
最后 . . . . . 122

**附录 A 面试题索引** **123**
第 4 章 所有岗位都会问的基础 . . . . . 123
第 5 章 运动控制岗：从 PID 到 MPC 和 WBC . . . . . 124
第 6 章 强化学习岗：奖励、PPO 和 Sim2Real . . . . . 124
第 7 章 感知与定位岗：标定、SLAM 和状态估计 . . . . . 125
第 8 章 操作与具身大模型岗：模仿学习、VLA 和数据 . . . . . 125
第 9 章 硬件岗：结构、关节、电控和嵌入式 . . . . . 126
第 10 章 软件、数据、测试和交付岗 . . . . . 126

**附录 B 术语表** **127**

**附录 C 资源清单** **131**
书 . . . . . 131
课程 . . . . . 131
开源软件和仓库 . . . . . 132
怎么用 . . . . . 132

**附录 D 模板** **133**
D.1 一页简历 . . . . . 133
D.2 项目 README . . . . . 134
D.3 面试复盘表 . . . . . 134
D.4 投递记录表 . . . . . 134
D.5 offer 比较表 . . . . . 135
D.6 访谈提纲：向已经转进来的人请教 . . . . . 135

**附录 E 资料来源** **137**

招聘市场 . . . . . 137

论文和技术资料 . . . . . 138

软件 . . . . . 139

# 前言：一条招聘启事，你能看懂几行？

先读一条招聘启事。它是按 2026 年公开招聘启事的常见写法拼出来的，不对应任何一家公司，但你在招聘网站上一定见过长得差不多的：

> **具身智能算法工程师（运动控制方向）**
>
> 岗位职责：负责人形机器人的全身运动控制，基于 MPC、WBC 与强化学习实现行走、上下楼梯与抗扰动；完成 Sim2Real 迁移与真机部署。
>
> 任职要求：1. 硕士及以上学历；2. 熟悉刚体动力学、MPC、WBC，熟悉 PPO 等强化学习算法；3. 熟练使用 C++、Python、PyTorch，熟悉 Isaac Lab 或 MuJoCo；4. 熟悉 ROS 2；5. 有真机调试经验者优先；6. 了解 VLA 等具身大模型者优先。

如果你能看懂其中的每一行，并且知道自己离它还差什么、怎么补、怎么在面试里证明，这本书对你用处不大。

如果你像大多数想进这一行的人一样，看到 WBC、Sim2Real、VLA 有点心虚，不知道自己的经验算不算“相关”，家里没有机器人、不知道没有真机怎么做项目，那就接着读下去。这本书就是为你写的。

## 这本书怎么回答那条启事

把启事里的每一行，对应到书里的位置：

- “MPC、WBC”“刚体动力学”：第 5 章，运动控制岗的知识地图和 12 道面试题；
- “强化学习”“PPO”“Sim2Real”：第 6 章，强化学习岗；
- “C++、Python”“ROS 2”：第 4 章，所有岗位都会问的基础；
- “Isaac Lab 或 MuJoCo”“真机调试经验”：第 11、12 章，6 个只用笔记本电脑就能做的作品集项目；

- “VLA 等具身大模型”：第 8 章，操作与具身大模型岗；
- “硕士及以上”：第 1 章讲门槛到底有多硬，第 13 章讲没达到时怎么在简历上补；
- 还有启事里没写、但你一定关心的：这个岗位每天干什么（第 2 章）、面试怎么考（第 14 章）、给多少钱、这家公司能活多久（第 1、15 章）。

## 这本书是一条漏斗

全书按求职的顺序排成一条漏斗：

1. **看清这一行**（第 1–3 章）：谁在招人、有哪些岗位、每个岗位要什么。第 3 章结束时，你会用一个小脚本得到一份属于自己的差距清单。
2. **按岗位补能力**（第 4–10 章）：从所有岗位共通的基础，到运动控制、强化学习、感知、具身大模型、硬件、软件和交付，每章都有知识地图和 10–12 道面试题，每道题都给了参考答案和“考官想听到什么”。
3. **拿到 offer**（第 11–16 章）：作品集、简历、面试、选公司和谈 offer，最后一章把所有事排进 90 天。

## 书里的数据和代码

这本书尽量用证据说话。

**数据**来自公开的招聘报告（脉脉、猎聘、智联）、宇树官网 2026 年 10 月 30 日的在招岗位清单，以及公开招聘启事的常见写法。每个数字都写了统计期；只看到媒体转述、没拿到原文的，写了“转引”。附录 E 列出了全部来源。

**代码**在随书的 `代码/` 目录里：6 个作品集项目，加一个第 3 章用的差距统计脚本。每个项目都在一台 4 核 CPU、没有显卡的 Linux 机器上实际跑过，书里的每一张结果图、每一个数字都来自那次运行。依赖的版本写在 `requirements.txt` 里，你照着装，应该能得到同样的结果。

**面试题**的答案对照教材和原始论文核对过，但它们是“常见考法”的参考答案，不是某家公司的原题，也不是唯一正确的答法。面试官更看重你能不能用自己的话讲清楚，并且联系到自己做过的事。

## 怎么读

- **想转行、还没选方向**：按顺序读第 1–3 章，选出两个目标岗位，再跳到对应的那一章。

- **方向已经定了**：直接读第 4 章和你的岗位那一章，然后去做第 11、12 章里对应的项目。
- **下周就要面试**：翻附录 A 的面试题索引，找到你的岗位，把那十几道题过一遍；再看第 14 章的追问链和复盘表。

每章都有几个固定的小框：“考官视角”讲面试官怎么想，“转行者笔记”写给从别的行业过来的人，“动手试试”是马上就能做的一件事，“本章一句话”是这一章最重要的结论。

## 几句实话

这本书不卖捷径。看完它你不会自动拿到 offer，你需要真的去跑项目、改简历、投出去、面试、复盘。

这本书也不吓唬人。具身智能的招聘确实热，但它会波动。书里的大部分内容（运动学、控制、状态估计、强化学习、嵌入式、测试）是机器人工程的地基，就算这一波热度过去了，它们在工业机器人、汽车、无人机行业照样用得上。结语专门讲这件事。

如果你读过这本书的姊妹篇《拆解宇树：从一颗电机到会翻跟头的机器人》，会发现两本书常常互相引用：那一本讲“机器人是怎么做出来的”，这一本讲“你怎么进去”。没读过也没关系，这本书自成一体。

翻到第 1 章，先看看 2026 年到底是谁在招人。

第一部

# 看清这一行

第 1 章

# 谁在招人：2026 年的具身智能人才市场

2026 年 5 月，脉脉发布了一份春招报告。里面有一个数字被转了很多遍：具身智能领域的招聘指数，从 2025 年的 36 涨到了 579。

看到这种新闻，你大概会同时冒出两个念头。一个是“机会来了”，另一个是“可我能干什么?”

这一章先回答几个更基本的问题：到底是谁在招人，招多少，招什么样的人，给多少钱，门槛多高。答案和新闻标题给人的印象不太一样。

## 三组数字，各说明了什么

先把几份公开报告里的数字摆出来，注意看统计期和口径。

**脉脉《2026 春招职场洞察报告》**，统计 2026 年 1–4 月：具身智能的招聘指数 579，上一年 36；具身智能岗位的平均月薪从 58,696 元涨到 61,625 元，涨了 5%。同一份报告里，大模型算法岗的平均月薪是 71,534 元。

**猎聘大数据研究院《2026 机器人领域人才供需趋势洞察报告》**（转引），统计 2025 年 4 月到 2026 年 3 月：机器人领域新发职位同比增长 75.26%；研发岗占比普遍在四到六成；算法岗年薪的主流区间是 30 万到 70 万元。

**智联招聘**（转引），2026 年一季度：机器人行业职位数增长 38%，其中机器人调试工程师增长 60%，机器人算法工程师增长 37%。

三组数字放在一起，能读出三件事。

第一，**热是真的**。三家平台、三种口径，方向一致：招人的公司在变多，岗位在变多。

第二，**“15 倍”不等于岗位多了 15 倍**。脉脉的 579 和 36 是“招聘指数”，不是岗位个数。猎聘统计的是新发职位，增长七成多；智联统计的是职位数，增长不到四成。读报告时先看清楚它数的是什么。

第三，**涨得最快的不一定是算法**。智联的数据里，调试工程师比算法工程师涨得快。这是第一个和新闻标题不一样的地方，后面还会反复看到。

> **这些数字的局限**
>
> 招聘平台的数据只反映在这家平台上发布的岗位，很多机器人公司还通过官网、内推、高校宣讲招人。“转引”的意思是我们没拿到报告原文，只看到了媒体转述。所以这些数字适合看方向，不适合精确比较。

## 谁在招：五类雇主

“具身智能公司”不是一种公司。按做什么来分，招人的雇主大致有五类。

1. **整机公司**：做四足、人形、轮式或轮足机器人的整机，比如宇树、智元、优必选、傅利叶。岗位最全，从结构到大模型都有。
2. **核心零部件公司**：做关节模组、减速器、灵巧手、触觉传感器、电机驱动。偏硬件和嵌入式，传统制造业的工程师在这里最容易接上。
3. **大厂和车企的机器人团队**：小米、小鹏这些公司都公开发布过四足或人形机器人，美团的校招里也出现过电机电驱方向的算法岗。待遇和流程更像大厂，团队的方向可能随公司战略调整。
4. **研究机构和创新中心**：高校实验室、地方政府牵头的创新中心（比如北京人形机器人创新中心），偏研究和平台建设，对学历要求高。
5. **应用和集成公司**：把机器人用到巡检、教育、科研、展厅、工厂里，做二次开发、部署和售后。门槛相对低，最看重“能把东西在现场跑起来”。

这五类公司对同一个岗位名称的期待可能完全不同。同样叫“运动控制工程师”，整机公司可能要你从零写控制器，集成公司可能要你会调用厂商的 SDK、把动作编排好。投之前先看清楚是哪一类。

## 一家头部公司到底在招什么

新闻里的总数太抽象。我们拿一家公司的在招清单来看。

2026 年 10 月 30 日，宇树官网的“招贤纳士”页面一共挂着 26 个岗位，全部在杭州。其中 4 个是销售类（解决方案、技术支持、海外销售、大客户销售），22 个是技术类。把这 22 个技术岗按方向归一下类：

表 1-1 宇树官网在招的 22 个技术岗位（2026 年 10 月 30 日）

| 方向 | 个数 | 岗位 |
| --- | --- | --- |
| 算法 | 4 | 运动控制算法、深度强化学习算法、AI 算法（大模型）、激光 SLAM 算法 |
| 数据与 AI 基础设施 | 4 | 机器人数据运营、数据管线、AI Infra、具身数据评估 |
| 软件 | 4 | 具身智能软件、C++ 开发、Java 后端、前端 |
| 嵌入式、硬件与机电 | 5 | 嵌入式软件（Linux）、嵌入式软件（电机控制）、嵌入式硬件、高级机电控制、机电全栈 |
| 机械与工业设计 | 2 | 机械工程师、工业设计师 |
| 测试、安全、项目管理 | 3 | 机器人测试、产品安全架构、研发项目经理 |

**算法岗 4 个，只占技术岗的不到五分之一**。要注意，这是按岗位种类数的，一个岗位可能招好几个人，官网不写人数。即便如此，结构已经很清楚了：一家做机器人的公司，大部分工作是把机器人造出来、让它稳定地跑、把数据管好、把产品交到客户手里。

还有一个细节：数据运营、数据管线、AI Infra、数据评估这 4 个岗位，在三年前的机器人公司里几乎见不到。它们是“具身大模型”火起来以后才长出来的新岗位，第 2 章和第 10 章会细讲。

**本章一句话**

**具身公司最缺的不只是算法博士，还有能把机器人造出来、跑起来、管好数据、交到客户手里的工程师。**

## 钱：薪资怎么看

先说结论：**算法岗确实给得高，但“平均月薪六万”不代表你能拿六万**。

脉脉的 61,625 元是新发岗位的平均月薪，平均数会被高薪岗位往上拉。首席科学家、算法专家这类岗位开出的是百万年薪甚至更高，几个这样的岗位就能把平均数抬上去。

更有参考价值的是具体岗位的区间。2026 年猎聘上几条公开的具身岗位，标题里写着这样的薪资（月薪，k 是千元）：

- 运动控制算法工程师：60–90k，13 薪；
- 具身智能算法工程师（小脑运动控制方向）：100–110k，14 薪；
- 具身智能机器人应用开发工程师：30–50k·16 薪、40–70k·14 薪、60–85k·15 薪三条；
- 运动控制算法工程师（系统工程师）：40–70k，15 薪。

再往前一点，2025 年 2 月杭州媒体报道过宇树的招聘：深度强化学习算法工程师 40–70K·13 薪。

这些只是例子，不代表平均水平，但能看出两个规律：一是同一个岗位名称，区间可以差一倍，取决于资历和公司；二是“几薪”差别很大，13 薪和 16 薪，年收入差两成以上。第 15 章会讲怎么拆一个 offer。

## 门槛：学历、经验和城市

**学历**。从公开招聘启事看，算法方向（运控、强化学习、大模型）大多写“硕士及以上”；嵌入式、硬件、机械、测试、软件这些工程岗位多写“本科及以上”，再加几年相关经验。这不是死规定，有真机项目、竞赛成绩、开源作品的本科生也能拿到算法岗面试，但你要用别的东西把学历的差距补上。

**经验**。社招岗位常写“3 年以上相关经验”。“相关”两个字有弹性：做自动驾驶定位的，去投机器人的 SLAM 岗，经验大多算数；做手机结构的，去投机器人结构岗，量产经验很值钱。第 2 章会把每种背景能接上的岗位列出来。

**城市**。宇树的 26 个岗位全在杭州。再看 2027 届校招的公开信息：优必选在深圳，北京人形机器人创新中心在北京，还有一些公司同时在北京、杭州、深圳招人。整体上，岗位集中在北京、上海、深圳、杭州这几个城市及周边。如果你不能换城市，可选的公司会少很多，这一点最好在一开始就想清楚。

## 新面孔：数据岗和新专业

人才市场的供给一侧也在变。

据钛媒体转引的消息，教育部 2026 年的本科专业目录里新增了“具身智能”专业，首批 9 所高校当年招生，北京理工大学计划招 120 人，东北大学 60 人，北邮、北航、上交、西交、浙大各 30 人。这些学生要到 2030 年才毕业，短期内改变不了供需，但说明这个方向被当成了长期的事。

另一边，想进这一行的人也在变多。脉脉 2026 年 9 月的一份报告里，求职意向调研中有 17.40% 的受访者选择具身智能或智能硬件企业。

供给在涨，需求也在涨。你要比的不是“会不会具身智能”，而是在你投的那个岗位上，能不能比别的候选人更快上手。

## 这对你意味着什么

把这一章的信息合在一起，对想进这一行的人有三条实际的含义。

1. **别只盯算法岗**。算法岗竞争最激烈，学历门槛最高。工程、数据、测试、交付岗位多，需求增长不比算法慢，很多时候更适合转行的人。
2. **你过去的工程经验可能很值钱**。猎聘的报告把“硬件与系统工程人才”“产品与商业化落地人才”列为和算法并列的紧缺方向。会量产、会调试、会和客户打交道，在机器人公司里都是稀缺能力。
3. **热度会波动，能力要通用**。今天的招聘热，跟人形机器人量产、融资和上市潮绑在一起。万一降温，运动学、控制、嵌入式、测试这些能力在工业机器人、汽车、无人机行业照样用得上。结语会专门讲这一点。

**转行者笔记｜先别急着学，先看清自己站在哪**

很多人一看到具身智能热，就去报一个强化学习课程。可如果你做了五年电机控制，最快的路可能是关节和电控岗位，而不是从零开始学强化学习。先找到自己现有的能力能接上哪一段，再决定补什么。

**动手试试｜填一张“我的起点”表**

在纸上或文档里写下面八行，后面每一章都会用到：

1. 现在的岗位和行业：
2. 相关工作年限：
3. 最高学历和专业：
4. 我最拿得出手的三项技能：
5. 我完全不会、但招聘启事里常见的三项：
6. 能不能换城市，可以去哪几个：
7. 能接受的最低月薪：

8. 每周能拿出几个小时准备：

下一章，我们把机器人公司里的岗位一个一个摊开，画成一张地图，帮你找到这张表最可能接上的那一格。

第 2 章

# 一张岗位地图：二十多个岗位各干什么

招聘网站上搜“具身智能”，能翻出几百个岗位名称：运控算法、具身算法、机器人学习、Sim2Real、具身数据、AI Infra、机电全栈……同一件事，不同公司起的名字不一样；同一个名字，不同公司要的人也不一样。

这一章把一家机器人公司里的岗位按“机器人的哪一层”排开，画成一张地图。每个岗位写四件事：**干什么，考什么，谁转过来最顺，门槛多高**。读完这一章，你要从里面挑出两个目标岗位。

《拆解宇树》里把一台机器人拆成了本体、小脑、大脑几层。找工作时可以分得更细一点：本体、小脑、感知、大脑四层是机器人自己；再加上数据与基础设施、软件、交付三层，才是一家公司让机器人卖出去、用起来需要的全部人手。

## 本体：把机器人造出来

**机械/结构工程师**。干：整机和关节的结构设计、选材、减重、抗摔、可制造性设计，跟进试制和量产。考：材料力学、有限元分析、公差配合、注塑钣金机加工艺，以及“这个零件为什么断了”。最顺的来路：手机、家电、汽车、无人机的结构工程师。门槛：本科，三年以上经验最受欢迎。

**关节/执行器工程师**。干：把电机、减速器、编码器、驱动板、散热装进一个关节模组，搭测试台测扭矩、温升、寿命。考：电机原理、减速器选型、减速比和反射惯量、发热和散热。来路：伺服电机、汽车电驱、工业减速器行业。门槛：本科或硕士。

**电机控制（电控）工程师**。干：写关节驱动板上的 FOC 程序，调电流环、速度环、位置环，处理过流、过温、堵转保护。考：永磁同步电机、FOC、PI 调参、电流采样、PWM。来路：新能源汽车电控、伺服驱动器、变频器。门槛：本科加经验。

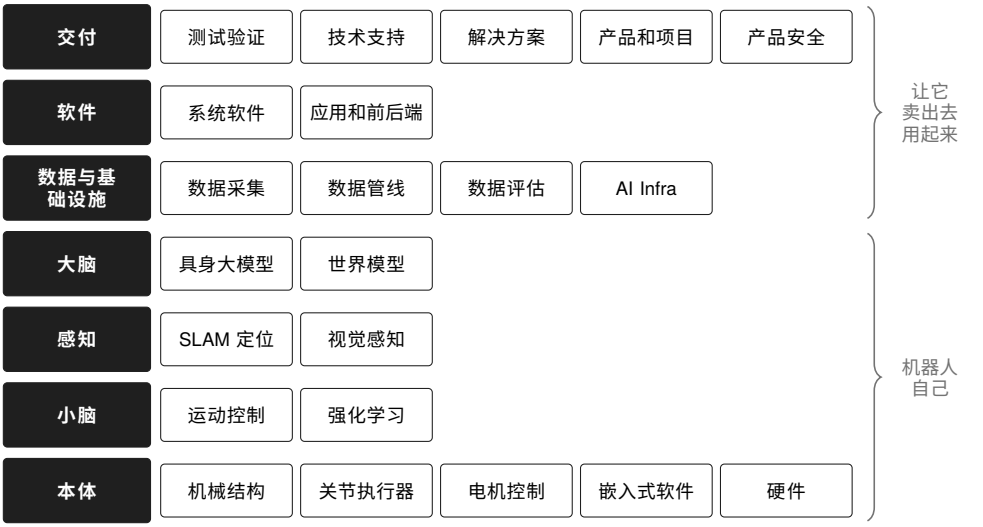



图 2-1 岗位地图：机器人从下到上的七层，每一层对应一组岗位

**嵌入式软件工程师**。干：单片机和嵌入式 Linux 上的驱动、总线通信（CAN、EtherCAT、RS-485）、实时任务、固件升级。考：C 语言、RTOS、中断和 DMA、通信协议、调试手段。来路：汽车电子、消费电子、工控。门槛：本科。

**硬件工程师**。干：主控板、电源板、传感器板、电机驱动电路，元器件选型，EMC。考：电路设计、PCB 布局、电源设计、信号完整性。来路：任何做电子硬件的团队。门槛：本科。

## 小脑：让它动起来

**运动控制算法工程师**。干：步态、平衡、MPC 和 WBC 控制框架，写运动控制 SDK，上真机调。考：刚体动力学、优化、控制理论，以及“为什么这么设计”。来路：控制、机械、机器人方向的研究生，自动驾驶规划控制工程师。门槛：多数要硕士。

**强化学习算法工程师**。干：在仿真里训练走、跑、跳、爬楼梯的策略，设计奖励和域随机化，把策略搬上真机。考：PPO 等算法、仿真器、机器人动力学常识、Sim2Real 的坑。来路：强化学习研究生、游戏 AI、自动驾驶决策。门槛：多数要硕士，有论文或真机经验加分。

## 感知：知道自己在哪、周围有什么

**SLAM/定位导航工程师**。干：多传感器标定和融合、建图、定位、地形感知、路径规划。考：滤波和图优化、点云处理、相机和激光雷达模型、标定。来路：自动驾驶定位、扫地机器人、AGV。门槛：本科或硕士加经验。

**视觉感知工程师**。干：检测、分割、物体位姿估计、深度估计，给抓取和导航提供输入。考：计算机视觉、深度学习、相机模型。来路：CV 算法工程师。门槛：多数要硕士。

## 大脑：理解任务、做决定

**具身大模型/VLA 算法工程师**。干：训练和微调“视觉-语言-动作”模型，设计数据配方，做评测，推动在机械臂和灵巧手上的部署。考：Transformer、模仿学习、扩散策略、多卡训练，以及数据怎么采。来路：大模型、CV、NLP 算法工程师，机器人学习方向的研究生。门槛：硕士、博士居多，竞争最激烈。

**世界模型/仿真研究员**。干：让模型预测“做了这个动作以后世界会变成什么样”，或者用生成模型造仿真数据。考：生成模型、视频模型、物理仿真。来路：生成模型、图形学方向的研究生。门槛：博士居多，岗位少。

## 数据与基础设施：喂饱模型

这一层是 2025 年以后才大量冒出来的。宇树官网的在招岗位里就有 4 个属于这一层。

**数据采集/数据运营工程师**。干：组织遥操作采集，清洗、标注、归档数据，维护采集设备，排查“这批数据为什么不能用”。考：动手能力、细致、Python 脚本。来路：数据标注和运营、实验员、技术员，也有刚毕业的本科生。门槛：相对低，这一层最容易进。

**数据管线/数据平台工程师**。干：把机器人上传的多模态数据（图像、关节状态、力、语言指令）做成可检索、可回放、可训练的流水线。考：数据工程、存储和索引、Python 或 C++。来路：互联网的数据开发、大数据工程师。门槛：本科加经验。

**数据评估/模型评测工程师**。干：评估一批数据的价值，挖难例，设计“模型到底好不好”的评测体系。考：统计、实验设计、机器学习基础。来路：算法评测、数据科学。门槛：本科或硕士。

**AI Infra 工程师**。干：分布式训练集群、训练加速、低延迟推理、把模型部署到机器人上的芯片里。考：CUDA、分布式训练、模型压缩和量化。来路：互联网和大模型公司的 AI Infra。门槛：本科加经验。

## 软件：让一切可靠地跑

**机器人系统软件工程师（C++）**。干：运动控制系统的软件架构、实时调度、通信中间件、日志和回放工具、对外 SDK。考：C++、多线程、Linux 实时性、ROS 2 和 DDS。来路：自动驾驶中间件、C++ 后端、游戏引擎。门槛：本科。

**应用和前后端工程师**。干：手机 App、云平台、远程运维、机队管理。考：常规的开发技能，加上机器人通信的基本概念。来路：互联网开发。门槛：本科。

## 交付：把机器人交到客户手里

**测试/验证工程师**。干：整机和关节的测试计划，跌落、续航、环境、寿命测试，自动化测试脚本，问题跟踪。考：测试设计、Python、硬件常识。来路：汽车、消费电子、工控的测试工程师。门槛：本科。

**技术支持/解决方案/售前工程师**。干：给客户设计方案、做演示、现场部署、培训、处理故障。考：产品理解、ROS 和 Python 基础、沟通。来路：工业自动化和 AGV 项目、售前、FAE。门槛：本科，经常出差。

**产品经理、研发项目经理**。干：定义产品和需求、管研发进度、推动认证和量产。来路：硬件、汽车、机器人行业的产品和项目经理。门槛：本科加经验。

**产品安全工程师**。干：找机器人产品的安全漏洞，设计安全方案，做合规。《拆解宇树》第 17 章讲过两次公开的安全事件，这个岗位就是为了不再出这种事。来路：网络安全。门槛：本科加经验。

---

**本章一句话**

**先找到你在地图上的那一格：岗位名称会变，但“机器人的哪一层”不会变。**

---

## 怎么挑出两个目标岗位

地图摊开了，下一步是挑。不建议同时准备五个方向，也不建议只押一个。挑两个：一个主攻，一个备选。

给你感兴趣的每个岗位打四项分，每项 1–5 分：

表 2-1　目标岗位打分表

| 项 | 问自己 |
| --- | --- |
| 接得上 | 我现有的经验能直接用上多少？ |
| 够得着 | 学历、年限、城市这些硬门槛我满足吗？ |
| 有人要 | 我能去的城市里，这个岗位多不多？ |
| 想干 | 一年以后天天做这件事，我愿意吗？ |

四项相加，挑分最高的两个。“接得上”和“够得着”有一项是 1 分的，先不选，哪怕它最热门。

举个例子。第 3 章会用到一位做了 4 年自动驾驶感知的工程师（虚构的例子），会 C++、ROS、点云、标定、卡尔曼滤波。这位工程师给几个岗位打的分是这样的：

表 2-2　一位自动驾驶感知工程师的打分（虚构示例）

| 岗位 | 接得上 | 够得着 | 有人要 | 想干 | 合计 |
| --- | --- | --- | --- | --- | --- |
| SLAM/定位导航 | 5 | 4 | 3 | 4 | 16 |
| 机器人系统软件（C++） | 4 | 5 | 3 | 3 | 15 |
| 数据管线 | 3 | 5 | 3 | 3 | 14 |
| 强化学习算法 | 2 | 2 | 4 | 5 | 13 |
| 运动控制算法 | 2 | 2 | 4 | 4 | 12 |

最想做的是强化学习，但“接得上”和“够得着”都只有 2 分。更稳的路是先以 SLAM 岗位进入机器人公司，进去以后再往强化学习靠。很多人就是这么转的：先进门，再换方向。

**转行者笔记｜别只看岗位名称**

岗位名称经常和实际工作对不上。“具身算法工程师”可能是训练 VLA 模型，也可能是在机械臂上调抓取。投之前把“岗位职责”那一段读三遍，看它每天到底在做什么。第 3 章教你一条一条拆。

**动手试试｜选出你的两个岗位**

1. 从这一章里圈出 5 个你感兴趣的岗位；
2. 用表 2-1 给每个打分；
3. 写下分最高的两个，再写一句为什么。

岗位选好了，下一步是看清它到底要什么。下一章，我们拿一条一条真实写法的招聘启事来拆。

第 3 章

# 快速见效：把招聘启事拆成能力清单

一条招聘启事三四百字，大多数人只认真看两处：薪资和学历。看完心里只剩一个模糊的判断：“好像够不着”或者“好像能试试”。

这一章教你把启事拆开读，读出三样东西：这个岗位每天干什么，门槛到底卡在哪，团队真正缺什么。然后用一个小脚本，把你收集的十几条启事一起统计，得到一份属于你自己的差距清单。

今天就能做完，做完你会知道接下来三个月该先补哪几样。

## 一条启事的三段

几乎所有启事都分三段：**岗位职责、任职要求、加分项**。三段各说明一件事。拿一条按常见写法改写的运动控制岗位来看（不对应任何一家公司）：

> **运动控制算法工程师**
>
> 岗位职责：1. 负责足式机器人（四足/人形）运动控制算法的研发与真机调试；2. 开发 MPC、WBC 等控制框架，提升机器人在复杂地形下的稳定性；3. 编写控制 SDK 与文档。
>
> 任职要求：1. 硕士及以上学历，控制、机械电子、自动化等相关专业；2. 熟悉机器人运动学与动力学建模，熟悉 PID、LQR、MPC；3. 熟练使用 C++，熟悉 Linux 和 ROS 2；4. 有足式机器人真机调试经验者优先；5. 有 RoboMaster 等机器人竞赛经历者优先。

**岗位职责说的是“你每天干什么”，也透露了团队眼下在解决什么问题**。这一条里，“复杂地形下的稳定性”说明他们的机器人在平地上已经能走了，现在要解

决的是台阶、斜坡、碎石路；“编写控制 SDK 与文档”说明控制算法要交给别人（客户或者别的团队）调用，你的代码要写得让别人看得懂。

**任职要求是筛选门槛**。学历和专业是第一道筛子，往往由 HR 或系统按关键词过；技能清单是第二道，由面试官验证。

**加分项往往是团队最痛的地方**。“真机调试经验”和“机器人竞赛经历”都指向一件事：他们需要不怕碰硬件、能在真机上把问题找出来的人。这一条在面试里很可能被追问：“你调过真机吗？遇到过什么问题?”

> **考官视角｜加分项不是可有可无**
>
> 候选人多的时候，面试官会先挑加分项沾边的人。如果你不满足加分项，就在简历和面试里拿一个“最接近的经历”顶上去。比如没调过机器人，但调过汽车的线控底盘，或者在仿真里复现过一个真机问题。第 13 章讲具体写法。

## “精通、熟悉、了解”各是什么分量

启事里描述技能的动词是有分量的，面试官会按这个分量来考。

- **精通**：能独立做项目，面试被连续追问三层不倒。比如“精通 C++”，要能讲清楚移动语义、智能指针的所有权、多线程里的数据竞争怎么查。
- **熟悉**：用过、做过、能讲清楚原理和取舍。比如“熟悉 MPC”，要能写出优化问题的形式，说清楚预测时域怎么选、约束怎么加。
- **了解**：知道是什么、为什么用、读过文档或论文。比如“了解 VLA”，能说出它的输入输出和几种代表模型即可。

还有一种常见写法：“掌握 WBC、MPC、强化学习、状态估计**中至少一个方向**”。意思是要你在一个方向上够深，而不是每样都懂一点。准备的时候，选一个方向做到“熟悉”以上，其他的做到“了解”。

## 藏在字里行间的信息

除了明写的要求，启事里还藏着一些信息，能帮你判断这个岗位和这家公司。

- **工具名透露技术栈**。写 Isaac Lab、MuJoCo，说明在用新一代仿真器训练；写 Gazebo、ROS 1，技术栈可能偏老。
- **“量产”“供应链”“DFM”**：公司已经在批量交付，工程岗位多，节奏偏稳。
- **“数据采集”“遥操作”“数据闭环”**：公司在做具身大模型，数据相关的岗位会持续招。

- **“能接受出差”“现场部署”**：岗位要跑客户现场。对有些人是缺点，对喜欢和客户打交道的人反而是机会。
- **写了很长的技术要求、却没写具体职责**：可能是团队还在搭建，你进去要做的事会比描述的多。

## 用脚本统计十几条启事

一条启事有偶然性，十几条放在一起看才有规律。随书代码里有一个小工具 `jd_check.py`，它做三件事：统计每项技能在多少条启事里出现；和你自己的技能对照；按出现次数列出你最该补的几项。

**第一步，收集启事**。在招聘网站、公司官网上找你目标岗位的启事，10–30 条，尽量来自不同的公司。把全文复制进一个文本文件，启事之间用单独一行 `---` 隔开。只给自己用、不外传，这样做没有问题。

**第二步，写下你的技能**。新建一个文本文件，一行写一项你会的技能，写法随意，“ROS2”“ROS 2”都认。

**第三步，运行：**

```
python jd_check.py --jd 我收集的启事.txt --me 我的技能.txt
```

随书代码里带了一份示例数据：10 条按常见写法改写的启事，覆盖运控、强化学习、大模型、SLAM、嵌入式、测试、数据采集、机械、系统软件、解决方案 10 个岗位；还有第 2 章那位自动驾驶感知工程师的技能清单。用示例数据跑一遍，输出的开头和结尾是这样的：

```
共 10 条启事

技能                出现   占比   你有吗
真机调试/部署        5     50%   ✗
ROS/ROS 2           5     50%   ✓
Linux               4     40%   ✓
足式/人形运控        3     30%   ✗
C++                 3     30%   ✓
……
你最该补的 10 项（按出现次数排）：
  1. 真机调试/部署（5/10 条启事提到）
  2. 足式/人形运控（3/10 条启事提到）
  3. 测试/验证（3/10 条启事提到）
  4. 运动学（2/10 条启事提到）
  5. 动力学（2/10 条启事提到）
  ……
```

```
你已经覆盖六成以上要求的启事：4/10 条

学历：
  提到本科：7/10 条
  提到硕士：3/10 条
  提到博士：0/10 条
  提到工作年限的 4 处，中位数 3 年
```

示例数据故意横跨 10 个岗位，所以每项技能的占比都不高。你自己用的时候，只收集目标岗位的启事，规律会清楚得多。

这个工具靠一张关键词表匹配，粗糙但够用。它会漏，也会误判，比如“测试”这个词在很多启事里只是顺带一提。看到可疑的结果，回到原文核对一下。你的方向有它不认识的词，就照着格式往词表里加一行：

```
SKILLS = {
    "MPC": r"\bmpc\b| 模型预测控制",
    "WBC": r"\bwbc\b| 全身控制 |whole[- ]body",
    # 在这里加你的词：名字和正则，正则不区分大小写
}
```

## 把差距排成计划

拿到差距清单以后，别从第一项开始一口气学下去。先把每项差距分进三个筐：

表 3-1　差距的三个筐

| 筐 | 例子 | 怎么办 |
| --- | --- | --- |
| 两周能补 | Docker、MuJoCo 入门、ROS 2 基本用法 | 马上做，做完写进简历 |
| 一两个月能补 | 运动学和动力学、强化学习基础、实时系统 | 排进 90 天计划，配一个作品集项目 |
| 半年以上或补不了 | 学历、足式机器人真机经验、顶会论文 | 不硬补；用别的东西顶上，或者换个目标岗位 |

**优先级 = 出现次数 × 补起来有多快**。出现得多、补得快的放最前面。

用示例结果试一下。对那位自动驾驶感知工程师来说：

- “真机调试/部署”出现得最多，但没有机器人很难补。可以换个说法：车辆上的传感器标定、实车路测排障，都是“真机调试”，简历里要这样写；再用第 11 章的项目在仿真里补“机器人怎么调”。
- “运动学”“动力学”放进一两个月的筐，配第 12 章的机械臂项目。
- “足式/人形运控”需要半年以上，暂时不补，所以第 2 章给这位工程师选的主攻方向是 SLAM，不是运控。

---

**本章一句话**

**差距是能列出来、能排优先级的。先补出现次数多、补得快的，补不了的就换个说法或者换个岗位。**

---

**转行者笔记｜别被“优先”吓退**

很多人看到“有足式机器人经验者优先”就不敢投了。“优先”的意思是有更好，没有也能投。真正的硬门槛一般写在任职要求的前两条。加分项一条都不满足，也值得投十家试试，面试本身就是最好的调研。

**动手试试｜拿到你的差距清单**

1. 收集你两个目标岗位的启事，各 10 条以上；
2. 写下你的技能清单；
3. 运行 `jd_check.py`，把“最该补的 10 项”分进表 3-1 的三个筐；
4. 从“两周能补”的筐里挑一项，今天就开始。

差距清单在手，第二部按岗位一项一项补。下一章先从所有岗位都会问的基础开始。

第二部

# 按岗位补能力

第 4 章

# 所有岗位都会问的基础

不管投哪个岗位，技术面试的前二十分钟往往长得差不多：一道旋转或坐标变换的题，一道 C++ 或多线程的题，一道和控制频率、通信有关的题，再手写一小段代码。

这些题不难，但答不好会直接出局：面试官会觉得你连机器人工程的地基都没有，后面的项目讲得再漂亮也不敢要。

这一章先给一张基础知识的地图，再给 12 道最常见的题和参考答案。最后有一个 30 分钟的自测，测完你就知道自己的地基牢不牢。

## 知识地图

表 4-1　所有岗位通用的基础知识

| 方向 | 必会 | 加分 | 可以先放一放 |
| --- | --- | --- | --- |
| 数学 | 矩阵运算、最小二乘、旋转表示、齐次变换、高斯分布 | SVD、李群李代数、凸优化 | 微分几何 |
| 编程 | C++ 的 RAII、智能指针、移动语义、STL；Python 和 NumPy | 模板、无锁数据结构、CUDA | 编译器原理 |
| 系统 | Linux 进程和线程、Git、调试工具 | 实时内核、性能分析 | 内核开发 |
| 机器人常识 | 坐标系约定、正逆运动学、控制频率和延迟 | 刚体动力学 | 多体动力学的高级算法 |

| 方向 | 必会 | 加分 | 可以先放一放 |
|---|---|---|---|
| 中间件 | ROS 2 的话题、服务、动作、QoS、tf2 | DDS 的发现机制和调优 | 自己写中间件 |

“必会”那一列，哪个岗位都绕不开；“加分”那一列，跟你的目标岗位有关就补。

## 面试题和参考答案

### 4.1 旋转矩阵、欧拉角、四元数、轴角，各有什么优缺点？什么是万向锁？

**一句话**：四个都是表示三维旋转的方法，各有擅长的场合：算变换用旋转矩阵，存储和插值用四元数，优化里用轴角（旋转向量），给人看用欧拉角。

- **旋转矩阵**：9 个数，满足正交、行列式为 1 的约束。旋转一个向量、把几个旋转连起来都只要矩阵乘法，很直接；缺点是冗余，数值误差积累以后会不再正交，需要重新正交化。
- **欧拉角**：3 个数（比如偏航、俯仰、横滚），直观；缺点是有 12 种转轴顺序，团队之间经常搞混，插值不平滑，还有万向锁。
- **四元数**：4 个数，单位长度。没有万向锁，连乘计算量小，插值平滑。要注意 $q$ 和 $-q$ 表示同一个旋转（双重覆盖），比较两个姿态时要考虑这一点。
- **轴角/旋转向量**：3 个数，方向是转轴，长度是转角。适合表示小的旋转增量，所以优化和滤波里常用它做“误差状态”。

**万向锁**：以“偏航-俯仰-横滚”（ZYX）顺序为例，俯仰转到 $\pm 90^\circ$ 时，第一次和第三次旋转的轴重合了，偏航和横滚变成同一个效果，丢掉一个自由度。这时姿态还在，但欧拉角解不唯一，在这附近数值会剧烈跳变。

**考官想听到**：万向锁能举出具体的角度；知道 $q$ 和 $-q$ 的问题；知道什么场合用哪种表示。

### 4.2 两个坐标系之间的变换怎么表示？$T_{AB}$ 的逆是什么？怎么把相机看到的点变到机器人基座坐标系？

**一句话**：用 4×4 的齐次变换矩阵把旋转 $R$ 和平移 $p$ 写在一起；求逆不用对整个矩阵求逆，直接写出来：

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}, \qquad T^{-1} = \begin{bmatrix} R^{\top} & -R^{\top}p \\ 0 & 1 \end{bmatrix}$$

约定 $T_{AB}$ 表示“坐标系 B 在坐标系 A 里的位姿”，它把 B 里的点变到 A 里：$p_A = T_{AB}\,p_B$（点写成齐次坐标）。这样写的好处是下标能“消掉”：

$$p_{\text{基座}} = T_{\text{基座,相机}}\,p_{\text{相机}}, \qquad T_{\text{基座,相机}} = T_{\text{基座,法兰}}\,T_{\text{法兰,相机}}$$

其中 $T_{\text{基座,法兰}}$ 来自正运动学，$T_{\text{法兰,相机}}$ 来自手眼标定。

团队里最常见的错误是两个人对 $T_{AB}$ 的含义理解相反。所以第一件事是把约定写进文档。ROS 2 的 tf2 里，`lookupTransform(target, source)` 返回的就是把 source 坐标系里的数据变到 target 坐标系的变换。

**考官想听到**：不用矩阵求逆就能写出逆变换；下标约定清楚；能说出手眼标定在这条链里的位置。

### 4.3 两个姿态之间怎么插值？为什么不能直接对四元数做线性插值？

**一句话**：用球面线性插值（slerp），沿着单位球面上的大圆走，角速度均匀。

对四元数的四个分量直接线性插值，中间结果不是单位四元数，要再归一化（这叫 nlerp）。nlerp 的路径是对的，但角速度不均匀：两头慢中间快。slerp 的公式是：

$$q(t) = \frac{\sin((1-t)\Omega)}{\sin\Omega}\,q_0 + \frac{\sin(t\Omega)}{\sin\Omega}\,q_1, \qquad \cos\Omega = q_0 \cdot q_1$$

两个实现细节：一是如果 $q_0 \cdot q_1 < 0$，先把 $q_1$ 取反，否则会绕远路走 360° 减去那个角度；二是两个姿态非常接近时 $\sin\Omega \to 0$，改用 nlerp 防止除零。

**考官想听到**：取反走短路径；接近时的数值处理；知道 nlerp 在很多实时场合也够用。

### 4.4 什么是雅可比矩阵？在机器人里有什么用？什么叫奇异？

**一句话**：雅可比 $J(q)$ 把关节速度映射成末端速度：$v = J(q)\,\dot{q}$。

它的用处至少有四个：

1. **速度控制**：想让末端以某个速度走，就解 $\dot{q} = J^{-1}v$（或者伪逆）。
2. **数值逆运动学**：每次迭代用 $J$ 把末端误差变成关节修正量。
3. **力的映射**：末端要施加力 $F$，关节需要的力矩是 $\tau = J^{\top}F$。这条可以用虚功原理推出来：$\tau^{\top}\dot{q} = F^{\top}v = F^{\top}J\dot{q}$ 对任意 $\dot{q}$ 成立。
4. **可操作度**：$J$ 的奇异值告诉你末端往哪个方向容易动、哪个方向难动。

**奇异**是 $J$ 降秩：末端在某个方向上动不了，想硬让它动，关节速度会趋向无穷大。机械臂完全伸直、两个手腕关节的轴重合，都是典型的奇异位形。工程上常用阻尼最小二乘处理：$\dot{q} = J^{\top}(JJ^{\top} + \lambda^2 I)^{-1}v$，代价是在奇异附近跟踪误差变大。第 12 章的项目五会实际跑一遍：伪逆在接近伸直时把关节速度推到几千 rad/s，阻尼最小二乘一直不超过 3 rad/s。

**考官想听到**：$\tau = J^{\top}F$ 能推出来；能举出奇异位形的例子；知道阻尼最小二乘和它的代价。

### 4.5 最小二乘怎么解？什么时候不该用正规方程？

**一句话**：求 $\min\|Ax - b\|^2$，正规方程给出 $x = (A^{\top}A)^{-1}A^{\top}b$，但数值上更稳的是用 QR 分解或 SVD。

问题出在 $A^{\top}A$ 的条件数是 $A$ 的条件数的平方。$A$ 本来就有点病态（比如标定时数据都挤在一个小范围里），平方以后精度就丢了。QR 分解直接在 $A$ 上做，不会平方条件数；SVD 最稳，还能处理 $A$ 不满秩的情况，这时给出的是伪逆解。

两个常见的变体也要会：**加权最小二乘**，给可信的测量更大的权重；**带正则的最小二乘**，在目标里加 $\lambda^2\|x\|^2$，它和上一题的阻尼最小二乘是同一件事。

**考官想听到**：条件数平方；知道什么时候用 SVD；能把正则化和阻尼最小二乘联系起来。

### 4.6 C++ 的 unique_ptr、shared_ptr、weak_ptr 有什么区别？什么情况下还会内存泄漏？

**一句话**：unique_ptr 独占所有权，shared_ptr 引用计数共享所有权，weak_ptr 不拥有、只观察，用来打破循环引用。

- `unique_ptr` 不能拷贝，只能移动，开销和裸指针一样，默认首选。
- `shared_ptr` 每次拷贝都要原子地加减引用计数，有开销；计数归零时释放。
- `weak_ptr` 不增加计数，用之前要 `lock()` 成 shared_ptr，对象已经没了就得到空指针。

用了智能指针还会泄漏的典型情况是**循环引用**：父节点用 shared_ptr 指向子节点，子节点也用 shared_ptr 指回父节点，两个计数永远不归零。子节点指回父节点的那一个改成 weak_ptr 就好了。另一个常见坑是回调里捕获了 `shared_from_this()`，回调没注销，对象就一直活着。

机器人代码里还要加一条：**实时控制循环里别拷贝 shared_ptr、别分配内存**。原子操作和内存分配的耗时都不确定，会让 1 ms 的控制周期偶尔超时。

**考官想听到**：循环引用和解法；shared_ptr 的开销；实时循环里的禁忌。

### 4.7 std::move 做了什么？移动以后原来的对象还能用吗？

**一句话**：`std::move` 什么也没移动，它只是把一个对象转换成右值引用，让编译器去选择移动构造或移动赋值。

真正“移动”的是移动构造函数：它把资源（比如堆上的缓冲区指针）从源对象拿过来，再把源对象里的指针置空，省掉一次深拷贝。移动之后，源对象处于“有效但未指定”的状态：可以析构、可以重新赋值，但不应该再读它的内容。

两个相关的点：函数按值返回局部对象时，编译器通常直接做返回值优化，连移动都省了，这时再写 `return std::move(x);` 反而可能阻止优化；一个类自己管理资源时，要按“五法则”把析构、拷贝构造、拷贝赋值、移动构造、移动赋值一起考虑。

**考官想听到**：move 只是类型转换；被移动对象的状态；返回值优化。

### 4.8 互斥锁、条件变量、原子变量怎么选？实时控制线程里为什么尽量不加锁？

**一句话**：互斥锁保护一段共享数据，条件变量用来“等某个条件成立”，原子变量适合单个变量的无锁读写。

条件变量要配合互斥锁使用，等待必须放在 `while` 循环里检查条件，因为存在虚假唤醒。

实时线程里少用锁，原因是**优先级反转**：低优先级线程拿着锁，被中优先级线程抢占，高优先级的控制线程等锁，结果被一个毫不相干的中优先级线程间接卡住。1997 年火星探路者号反复重启，原因就是一次优先级反转。

常见的替代做法：

1. 单生产者单消费者的**无锁环形缓冲区**，控制线程和通信线程之间传数据；
2. **双缓冲**：写线程写一份，写完原子地切换指针；
3. 实在要用锁，用支持**优先级继承**的互斥锁，或者在控制线程里只用 `try_lock`，拿不到就用上一周期的数据。

**考官想听到**：优先级反转能讲清楚；知道无锁队列和双缓冲；知道条件变量的虚假唤醒。

### 4.9 一个 1 kHz 的控制循环，怎么保证它真的每 1 ms 跑一次？怎么测？

**一句话**：用绝对时间睡眠、实时调度和实时内核，循环里不做任何耗时不确定的事；用时间戳统计周期的抖动和最坏延迟。

具体做法：

- 用 `clock_nanosleep` 加 `TIMER_ABSTIME`，按“下一个周期的绝对时间点”睡，而不是每次睡 1 ms。后者会把每次的处理时间累积成漂移。
- 线程设成 `SCHED_FIFO` 实时调度，给一个高优先级；系统用 PREEMPT_RT 实时内核；把控制线程绑到一个隔离出来的 CPU 核上。
- 启动时 `mlockall` 锁住内存，避免运行中缺页；预先分配好所有内存。
- 循环里不打日志、不读写文件、不分配内存，日志通过无锁队列交给另一个线程写。

测的方法：每个周期记录时间戳，统计实际周期的分布，重点看最坏值而不是平均值。系统层面可以先用 `cyclictest` 看这台机器的调度延迟。

**考官想听到**：绝对时间睡眠；最坏延迟比平均值重要；能说出两三条“循环里不能做的事”。

### 4.10 ROS 2 的话题、服务、动作有什么区别？QoS 的 reliable 和 best effort 怎么选？

**一句话**：话题是持续的数据流，服务是一问一答，动作是有反馈、能取消的长任务。

- **话题**（topic）：发布订阅，适合传感器数据、状态、控制指令。
- **服务**（service）：请求响应，适合查询参数、触发一次性的操作。
- **动作**（action）：发一个目标，过程中持续反馈，可以中途取消，适合“走到某个位置”“抓起某个物体”这种要花几秒到几分钟的任务。

QoS 里最常被问的是可靠性：

- **reliable**：丢了会重传，保证送到。适合控制指令、配置、状态切换。
- **best effort**：丢了就丢了。适合高频的传感器数据，比如图像和 IMU，你要的是最新的一帧，重传一帧旧的没有意义。

一个常见的坑：订阅方设成 reliable、发布方是 best effort，两边 QoS 不兼容，**根本连不上**。程序不会崩，只在日志里打一条不太显眼的“QoS 不兼容”警告，表现就是收不到数据。排查的时候用 `ros2 topic info -v` 看两边的 QoS。

**考官想听到**：三种通信的适用场景；QoS 不兼容导致收不到数据；为什么传感器数据用 best effort。

## 4.11 DDS 是什么？为什么同一网段里两套 ROS 2 程序会“串台”？怎么隔离？

**一句话**：DDS 是 ROS 2 底下的通信中间件，它会在局域网里自动发现其他参与者，所以同一网段、同一个域里的程序会互相看见。

DDS 的自动发现靠组播：每个参与者启动时广播“我在这里，我发布什么、订阅什么”，匹配上就直接通信，不需要中心节点。这很方便，但两台机器人、或者仿真和真机放在同一个网段里，就可能互相收到对方的话题。

隔离的办法有三种：

1. **不同的域号**（`ROS_DOMAIN_ID`），不同域之间互相看不见。《拆解宇树》第 19 章讲过，宇树的仿真器用域号 1、真机用域号 0，切换时只改这个数；
2. 只在本机通信：较新的 ROS 2 版本把环境变量 `ROS_AUTOMATIC_DISCOVERY_RANGE` 设成 `LOCALHOST`；
3. 指定 DDS 使用的网卡，或者配置只和白名单里的地址通信。

**考官想听到**：自动发现的机制；域号；仿真和真机怎么隔离。

## 4.12 手写一个带抗积分饱和的 PID 控制器

**一句话**：输出限幅以后要停止积分（或者往回退），否则积分项会越积越大，退出饱和时严重超调。

```
class PID:
    def __init__(self, kp, ki, kd, out_min, out_max, d_alpha=0.1):
        self.kp, self.ki, self.kd = kp, ki, kd
        self.out_min, self.out_max = out_min, out_max
        self.d_alpha = d_alpha          # 微分项一阶低通的系数
        self.integral = 0.0
        self.prev_meas = None
        self.d_filt = 0.0

    def update(self, target, meas, dt):
        err = target - meas
        # 对测量值求微分，而不是对误差：目标值突变时不会有“微分冲击”
        d = 0.0 if self.prev_meas is None else -(meas - self.prev_meas) / dt
        self.prev_meas = meas
```

```
        self.d_filt += self.d_alpha * (d - self.d_filt)
        out = self.kp * err + self.integral + self.kd * self.d_filt
        if self.out_min < out < self.out_max:
            self.integral += self.ki * err * dt     # 只在不饱和时积分
        return min(max(out, self.out_min), self.out_max)
```

讲代码的时候主动说出三个设计：**条件积分**防止积分饱和；**对测量值求微分**，目标值突变时不会出现微分冲击；**微分项低通滤波**，因为微分会放大测量噪声。积分项乘 $k_i$ 以后再累加，这样在线改 $k_i$ 时输出不会突跳。

**考官想听到**：三个设计都能说出来；知道 $dt$ 要用实际测得的周期；能说出另一种抗饱和方法（反算法，back-calculation）。

> **考官视角｜基础题怎么答更像“自己人”**
>
> 基础题的标准答案网上都有，拉开差距的是“工程上的那一句”：四元数要说 $q$ 和 $-q$，最小二乘要说条件数，多线程要说优先级反转，ROS 2 要说 QoS 不兼容时根本收不到数据。这些细节只有真正踩过坑的人才会主动提，面试官一听就知道你写过代码。

## 30 分钟自测

不看书，30 分钟内完成下面 8 件事。卡住的，就是你要回去补的：

1. 写出 ZYX 欧拉角在俯仰 90° 时为什么会丢一个自由度；
2. 写出齐次变换的逆；
3. 用 NumPy 写四元数乘法；
4. 写出 $\tau = J^{\top} F$ 的推导；
5. 说出正规方程的数值问题；
6. 画一张图解释优先级反转；
7. 说出 1 kHz 控制循环里不能做的三件事；
8. 写出带抗积分饱和的 PID。

> **转行者笔记｜不同背景的短板不一样**
>
> 互联网和自动驾驶软件背景的人，C++、多线程、Linux 一般没问题，短板在旋转表示、运动学和控制；机械、电控背景的人正好相反，物理直觉好，短板在 C++ 和系统软件。照着自己的短板补，别从头学一遍。

**本章一句话**

**基础题答不好，后面的项目讲得再好也会被刷。答基础题时，多说一句“工程上要注意什么”。**

地基打好了，接下来按岗位往上盖。下一章从最典型的机器人岗位开始：运动控制。

第 5 章

# 运动控制岗：从 PID 到 MPC 和 WBC

运动控制是最典型的机器人岗位，也是面试最“硬”的岗位之一：面试官会让你推公式、画框图，再追问你在真机上调过什么。

这一章先讲这个岗位每天在干什么、招聘启事怎么写，再给一张知识地图和 12 道面试题。对应的作品集项目是第 11 章的项目一（让机器狗站起来）和项目二（倒立摆）。

## 这个岗位的一天

下面是一个典型的一天，不是某个真人。

上午，看昨天测试的日志：一台四足在湿地砖上打滑了两次，你把那两段的关节力矩、足端力估计和 IMU 数据调出来，发现打滑前摆动腿落地比计划早了 30 毫秒。中午在仿真里复现，改了接触检测的阈值。下午把机器人吊在架子上测新参数，再放到地上走斜坡。傍晚给 SDK 补一段文档，因为客户问了第三遍“步态切换的时候能不能不停”。

这个岗位一半时间在推公式和写代码，另一半时间在看数据、上真机。

## 招聘启事里怎么写

把 2026 年公开招聘启事里的高频要求归纳一下：

- 学历：多数写硕士及以上，控制、机械电子、自动化、机器人相关专业；
- 理论：机器人运动学和动力学建模，PID、LQR、MPC，WBC、强化学习、状态估计至少精通一个方向；
- 工程：C++ 和 Python，Linux，ROS 2，Isaac Gym/Isaac Lab 或 MuJoCo；
- 加分：足式或人形机器人真机调试经验，机器人竞赛经历。

## 知识地图

表 5-1 运动控制岗的知识地图

| 必会 | 加分 | 可以先放一放 |
| --- | --- | --- |
| PID 和关节 PD 控制；带宽、相位裕度的直觉 | 非线性 MPC、轨迹优化（iLQR、DDP） | 李群上的几何控制 |
| 状态空间和 LQR | 阻抗控制、力位混合控制 | 高级鲁棒控制理论 |
| 浮动基刚体动力学、单刚体模型 | 混合动力学、接触隐式优化 | 自己写多体动力学库 |
| 倒立摆、ZMP、捕获点 | 人形机器人的全身规划 | |
| 接触力、摩擦锥、凸 MPC（QP） | QP 求解器的原理和调参 | |
| WBC 的基本形式和任务优先级 | 分层 QP | |
| 步态调度、摆动腿轨迹、落脚点 | 地形感知步态 | |

## 面试题和参考答案

### 5.1 关节 PD 控制里的 kp 和 kd 分别像什么？调大调小各会怎样？

**一句话**：kp 像一根弹簧，kd 像一个阻尼器。力矩 $\tau = k_p(q_{目标} - q) - k_d\,\dot{q}$，就是在关节上装了一根虚拟的弹簧加阻尼。

- **kp 太小**：关节“软”，站不住或者站得低，负载一压就塌。项目一里 kp=10 时，Go2 根本站不起来，机身高度只有 0.15 m（目标约 0.27 m）。
- **kp 大**：站得准、负载下下沉小，但力矩需求大，对延迟更敏感。项目一里 kp=300 时膝关节误差只有约 1°，压上 60 牛的力只下沉 2 mm 左右。
- **kd 小**：振荡、超调。
- **kd 大**：在理想情况下更稳，但它放大速度测量的噪声，**有延迟时还会引起抖动**。

最后一点最容易答错。项目一里，同样 kp=300、控制延迟 5 ms：kd=3 稳稳站住；kd=8 时膝关节以大约 45 Hz 来回抖，力矩在正负极限之间来回打满，5 秒里有 4.8 秒处于饱和。

**考官想听到**：弹簧阻尼的类比；kd 在有延迟、有噪声时的副作用；能说出“真机上抖了，第一反应不该是加 kd”。

### 5.2 四足机器人为什么常用“PD 加前馈力矩”，而不是纯位置控制或者纯力矩控制？

**一句话**：前馈力矩负责“已知的那部分”（重力、规划好的接触力），PD 负责“意料之外的误差”，两者合起来既准又柔。

很多四足的底层关节指令有五个数：目标角度、目标角速度、kp、kd、前馈力矩，关节驱动板按 $\tau = k_p(q_d - q) + k_d(\dot{q}_d - \dot{q}) + \tau_{\text{ff}}$ 算出电机力矩。《拆解宇树》第 9 章讲过，宇树的底层指令就是这五个字段。

- **纯位置控制**（kp 很大）：腿很硬，落地冲击大，对地面高度误差毫无容忍度。
- **纯力矩控制**（kp=kd=0）：完全依赖上层模型准确，模型一有误差就飘。
- **PD 加前馈**：支撑腿上，MPC 算出的足端力通过 $\tau_{\text{ff}} = J^{\top} f$ 变成前馈力矩，PD 增益设得低一点，让腿保持柔顺；摆动腿上，增益设高一点，保证轨迹跟得准。

**考官想听到**：前馈和反馈的分工；支撑腿和摆动腿的增益为什么不同；$J^{\top} f$ 的来历。

### 5.3 线性倒立摆模型怎么推？时间常数是多少？捕获点是什么？

**一句话**：假设质心高度 $z$ 不变、腿没有质量，质心水平位置满足 $\ddot{x} = \frac{g}{z} x$，偏差按 $e^{\omega t}$ 发散，$\omega = \sqrt{g/z}$。

推导：支撑点在原点，腿对质心的力沿着腿的方向。竖直方向受力平衡（高度不变），水平分量和竖直分量之比等于 $x/z$，于是水平加速度 $\ddot{x} = g\, x/z$。

质心高 0.6 m 时，$\omega \approx 4.0\ \text{s}^{-1}$，时间常数约 0.25 s：偏差每过四分之一秒放大约 $e$ 倍。个子越高，时间常数越长，留给控制器的反应时间越多。

**捕获点**：$x_{\text{捕获}} = x + \dot{x}/\omega$。把下一步踩到这里，机器人就能停下来。直觉是：速度越快，要往前多迈一些，多迈的量是“速度除以 $\omega$”。

**考官想听到**：两个假设（质心高度不变、腿无质量）；$\omega$ 的物理意义；捕获点的公式和直觉。

### 5.4 ZMP 是什么？它在什么情况下不适用？

**一句话**：ZMP（零力矩点）是地面上的一个点，地面反力对它的水平力矩为零。只要 ZMP 落在支撑多边形里面，脚就不会绕着边缘翻转。

传统人形机器人（比如本田的 ASIMO）的走法就是：先规划 ZMP 轨迹，让它始终在脚掌范围内，再反推质心轨迹。这种方法稳，但动作保守。

ZMP 判据的前提是：所有接触都在同一个平面上、脚不打滑、一直有脚着地。以下情况不适用或者不够用：

- 腾空阶段（跑、跳、翻跟头），没有支撑面；
- 接触不在同一平面，比如手扶着墙、脚踩在台阶边缘；
- 摩擦不够、会打滑，ZMP 在支撑面里也可能滑倒；
- 动态步态，比如四足的对角小跑，支撑多边形退化成一条线。

**考官想听到**：定义说得准；能说出三种以上不适用的情况；知道为什么现在的足式机器人更多用 MPC 和强化学习。

## 5.5 四足机器人的 MPC 一般怎么建模？为什么用单刚体模型？

**一句话**：把整个机器人看成一个刚体，四只脚的接触力是控制输入，在未来零点几秒里优化这些力，让机身的位置和姿态跟上目标；每次只执行第一步，下一周期重新算。

典型做法（以 MIT Cheetah 3 的凸 MPC 为代表）：

1. **模型**：单刚体，状态是机身位置、速度、姿态、角速度；输入是各支撑脚的地面反力。腿的质量相对机身小，忽略掉。
2. **线性化**：假设横滚和俯仰角不大，把姿态动力学在当前偏航角附近线性化，整个问题变成一个线性系统。
3. **约束**：每只支撑脚的力要满足摩擦锥（见下一题）和力的上下限；摆动脚的力为零。
4. **求解**：目标是跟踪误差加力的大小，整体是一个二次规划（QP），可以在几毫秒内解完。

为什么用单刚体：完整模型是非线性的，求解慢，不能保证实时；单刚体模型加线性化以后是凸问题，有可靠的快速解法，又保留了“力怎么影响机身运动”这个核心。

**考官想听到**：输入是接触力而不是关节力矩；线性化的假设；凸 QP 的好处；滚动时域“只执行第一步”。

## 5.6 摩擦锥约束怎么写？为什么常常线性化成“摩擦金字塔”？

**一句话**：脚不打滑，要求切向力不超过摩擦系数乘以法向力：$\sqrt{f_x^2 + f_y^2} \le \mu f_z$，同时 $f_z \ge 0$（地面只能推不能拉）。

这个约束在三维力空间里是一个圆锥，属于二阶锥约束，普通的 QP 求解器处理不了。把它换成内接的四棱锥就变成线性不等式：

$$|f_x| \le \mu f_z, \qquad |f_y| \le \mu f_z, \qquad f_z \ge 0$$

为了保守，$\mu$ 通常取得比实际摩擦系数小一些，比如实测 0.8 时取 0.5–0.6，留出余量。湿滑地面上 $\mu$ 设错，是机器人打滑的常见原因之一。

**考官想听到**：圆锥变金字塔的原因；$f_z \ge 0$；$\mu$ 取保守值。

### 5.7 WBC 是什么？它和 MPC 是什么关系？任务优先级怎么实现？

**一句话**：WBC（全身控制）在完整的动力学模型上，把上层给的目标（机身加速度、足端力、摆动腿轨迹）变成每个关节的力矩，同时满足动力学、摩擦和力矩限制。

动力学方程是 $M(q)\ddot{q} + h(q,\dot{q}) = S^\top \tau + J_c^\top f$，其中 $J_c$ 是接触雅可比。WBC 通常也写成一个 QP，优化变量是 $\ddot{q}$、$\tau$、$f$。

**和 MPC 的关系**：MPC 在简化模型（单刚体）上、以几十到一百多赫兹的频率规划未来零点几秒；WBC 在完整模型上、以几百赫兹到一千赫兹的频率，把 MPC 这一刻的规划落实成关节力矩。一个管“往哪走”，一个管“这一刻每个关节出多少力”。

**任务优先级**两种实现：

- **加权**：所有任务放进一个目标函数，重要的权重大。简单，但重要任务也可能被别的任务“拉偏”；
- **严格分层**：先满足高优先级任务（比如不摔倒、接触脚不动），再在它的零空间里满足低优先级任务（比如手臂姿态）。可以用零空间投影，也可以按优先级依次解一串 QP（分层 QP）。

**考官想听到**：动力学方程写得出来；MPC 和 WBC 的频率、模型分工；加权和严格分层的区别。

### 5.8 LQR 和 PID 有什么区别？LQR 的 Q 和 R 怎么调？

**一句话**：PID 是单输入单输出的误差反馈，靠调参；LQR 是基于线性模型的全状态反馈，通过最小化“状态误差加控制代价”自动算出最优增益。

LQR 求的是 $u = -Kx$，使 $\int (x^\top Q x + u^\top R u)\, dt$ 最小，$K$ 由 Riccati 方程解出。Q 越大，越在意状态误差，控制越激进；R 越大，越在意控制量，越省力、越温和。

实践中先按“每个状态能容忍多大误差”的倒数平方设 Q 的对角元，再调整整体比例。

项目二用小车倒立摆比较过两者：只管摆角的 PID 能让杆立住，但小车回不到原点，8 秒里漂出去 0.46 m；LQR 同时管四个状态，杆和小车都回到原位。LQR 对模型误差很宽容：实际的杆比模型重 5 倍也照样稳；但对延迟敏感：控制延迟 80 ms 还能撑住，100 ms 就倒了。

**考官想听到**：全状态反馈和单变量反馈的区别；Q、R 的含义；知道 LQR 依赖线性化，离工作点远了会失效；延迟比模型误差更危险。

### 5.9 控制频率怎么选？延迟对稳定性有什么影响？

**一句话**：采样频率一般要比闭环带宽高一个数量级；延迟相当于给系统加了一个随频率增长的相位滞后，会吃掉相位裕度，增益越高越危险。

一台腿足机器人里通常是几层频率套在一起：

- 关节电流环：十几到几十千赫兹，在驱动板上；
- 关节 PD 和全身控制：几百赫兹到一千赫兹；
- MPC：几十到一百多赫兹；
- 步态和路径规划：几赫兹到几十赫兹。

延迟 $\tau_d$ 在频率 $\omega$ 处带来 $\omega\tau_d$ 弧度的相位滞后。增益调高，闭环带宽变大，同样的延迟吃掉的相位裕度就更多，系统开始振荡。这就是项目一里“kp=300、5 ms 延迟、kd=8”出现抖动的原因。

**考官想听到**：几层频率的数量级；延迟和相位裕度的关系；高增益和延迟不能兼得。

### 5.10 真机上一条腿在高频抖动，你怎么排查？

**一句话**：先采数据看抖动的频率和发生条件，再按“控制、传感、通信、机械”的顺序逐个排除，不要一上来就改参数。

1. **记录数据**：关节角、角速度、指令力矩、实际电流、控制周期的时间戳。算出抖动频率。
2. **看是不是控制问题**：抖动时力矩有没有饱和、来回打满？临时把 kd、kp 降一半，抖动是否消失？抖动频率是否和控制延迟对得上？
3. **看传感器**：角速度是不是由编码器差分得来、噪声很大？加一阶低通滤波，或者降低 kd。

4. **看通信**：控制周期的抖动和丢包，偶发的几毫秒延迟就足以让高增益系统振荡。
5. **看机械**：减速器背隙、螺丝松动、线缆拉扯、结构共振。共振的频率一般固定，和增益关系不大。
6. **看温度**：电机发热后电阻变大，同样电流下扭矩下降，表现可能是“越调越软”或者保护动作。

**考官想听到**：先测量再改参数；能分清控制振荡和机械共振；提到饱和、延迟、速度噪声这三个常见原因。

### 5.11 摆动腿和支撑腿的控制有什么不同？落脚点怎么选？

**一句话**：支撑腿管“推地面”，做力控；摆动腿管“把脚送到下一个落脚点”，做轨迹跟踪。

- **摆动腿**：从离地点到落脚点规划一条笛卡尔空间的曲线（常用贝塞尔曲线或摆线，中间抬高几厘米避免绊脚），用较高的 PD 增益或笛卡尔阻抗控制跟踪。
- **支撑腿**：MPC 算出这条腿该出的地面反力，通过 $\tau = J^{\top} f$ 变成关节力矩，PD 增益很低。
- **落脚点**：最经典的是雷伯特启发式，$p_{\text{落脚}} = p_{\text{髋}} + \frac{v\,T_{\text{支撑}}}{2} + k\,(v - v_{\text{目标}})$：前一项让脚落在支撑期中点正下方，后一项按速度误差修正，跑快了就多往前踩一点。

还有两个工程细节：**落地检测**（靠足端力传感器或者用关节力矩估计接触），以及**早落地和晚落地**的处理：没到计划时间就碰到地面，要立即切到支撑模式；到了时间还没碰到，要继续往下探。

**考官想听到**：两种腿的控制目标和增益差别；雷伯特公式；落地检测和异常处理。

### 5.12 让你从零给一台四足写一个能站立、能慢走的控制器，你会怎么分步做？

**一句话**：先站，再原地踏步，再走；每一步都先在仿真里跑通，再上吊架，最后落地。

1. **状态估计**：IMU 给姿态，腿的运动学加“支撑脚不动”的假设给机身速度，用卡尔曼滤波融合（第 7 章、项目四）。
2. **站立**：关节 PD 加重力补偿，从趴下的姿势平滑过渡到站立姿势（项目一）。
3. **步态调度**：对角小跑，固定周期，每条腿在支撑和摆动之间切换。
4. **摆动腿**：落脚点用雷伯特公式，轨迹用贝塞尔曲线。

5. **支撑腿**：先用一个简单的力分配（按重量平均分到支撑脚），能走以后再换成 MPC。
6. **安全**：力矩、速度、关节角限位；摔倒检测（姿态角超过阈值就进入阻尼模式）；急停。
7. **测试顺序**：仿真 → 吊架上空跑 → 落地站立 → 原地踏步 → 平地慢走 → 斜坡和台阶。

**考官想听到**：模块划分清楚；先简单后复杂；安全措施和测试顺序。这道题考的是工程思路，不是某个公式。

> **考官视角｜运控面试最常见的三个错答**
>
> 一是“kd 越大越稳”，忽略了噪声和延迟；二是把 MPC 说成“预测未来”，说不出优化变量、约束和滚动执行；三是只会讲论文，被问到“真机上遇到过什么问题”时答不上来。第三个最致命。没有真机，就用项目一的“延迟加高 kd 会抖”这类在仿真里复现过的问题来答，并且说清楚你是怎么发现、怎么验证的。

## 从哪些背景转过来最顺

- **控制、机械、机器人方向的研究生**：理论现成，补真机和工程能力。
- **自动驾驶规划控制**：MPC、优化、C++ 都能直接用上；要补的是接触动力学和腿足特有的问题（摩擦锥、步态、落地冲击）。
- **工业机器人和机械臂控制**：动力学、伺服调参经验很值钱；要补浮动基和接触切换。
- **无人机飞控**：状态估计、串级控制、实时系统都是现成的；腿足的接触是新东西。
- **电机控制**：最懂“真机怎么调”，能从关节和底层控制切入，再往上学 MPC 和 WBC。

> **转行者笔记｜先在仿真里把一个问题吃透**
>
> 运控岗位最看重真机经验，但没有真机的人也不是没机会。把项目一和项目二做透：每个现象都能复现、能解释、能给出改法，面试时比“调过真机但说不清原理”的候选人更有说服力。

> **动手试试｜换一组参数**
>
> 跑项目一，把 kp 和 kd 换成你自己选的几组值，再把延迟改成 0、2、5、10 毫秒，记下哪组开始抖。面试时被问到“kd 怎么调”，你就有自己的数据可以讲。

**本章一句话**

**运控面试考的是“为什么这么设计”：每个公式背后的假设，每个参数调大调小的后果，以及真机上出了问题你怎么查。**

下一章是另一条让机器人动起来的路：不推公式，让它在仿真里自己练出来。

第 6 章

# 强化学习岗：奖励、PPO 和 Sim2Real

上一章的运动控制是“先把模型推清楚，再设计控制器”。强化学习走的是另一条路：在仿真里让机器人摔几亿次，自己摔出一套走路的办法。现在机器狗的跑跳、人形机器人的翻跟头，大多是这么练出来的。

这个岗位的面试有个特点：公式题不多，但追问很深。面试官最想知道的是，你有没有真的训练过一个策略，碰到不收敛、奖励作弊、上真机就倒的时候，知不知道该怎么查。

对应的作品集项目是第 11 章的项目三：用 PPO 训练 MuJoCo 里的 Hopper，对照“加不加域随机化”。

## 这个岗位的一天

一个典型的一天（不是某个真人）：

早上看昨晚在 GPU 服务器上跑的训练：四千个仿真环境并行，曲线涨得不错。回放一看，机器狗学会了用膝盖蹭着地走，因为这样不容易摔、也不违反任何一条奖励。加了一项“膝盖碰地就结束回合”，重新开训。下午把上周的策略放进另一个仿真器 MuJoCo 里验证，发现关节顺序配错了一处。傍晚上真机，机器人在地毯上一走就打滑，你决定把地面摩擦系数的随机化范围往下扩。

## 招聘启事里怎么写

- 学历：多数要硕士，有论文或真机部署经验的优先；
- 算法：PPO、SAC 等常见算法，奖励设计，域随机化；
- 工具：PyTorch 或 JAX；Isaac Gym、Isaac Lab、MuJoCo；
- 基础：运动学、动力学、控制常识；

- 加分：人形、四足或灵巧手的真机策略部署经验。

## 知识地图

表 6-1 强化学习岗的知识地图

| 必会 | 加分 | 可以先放一放 |
| --- | --- | --- |
| 马尔可夫决策过程、回报、价值函数 | 非对称 actor-critic、教师-学生蒸馏 | 收敛性证明 |
| 策略梯度、PPO、GAE | 动作模仿（DeepMimic、AMP） | 多智能体强化学习 |
| 足式运动的奖励设计 | 课程学习、地形课程 | 离线强化学习理论 |
| 观测和动作的设计、归一化 | SAC 等离线策略算法 | |
| 域随机化、Sim2Sim、Sim2Real | 执行器建模、系统辨识 | |
| GPU 并行仿真的基本用法 | 自己改仿真器 | |

## 面试题和参考答案

### 6.1 PPO 的核心思想是什么？“截断”在防什么？

**一句话**：PPO 是一种策略梯度方法，每次更新时限制新策略不能离旧策略太远，避免一步走太大把策略毁掉。

记新旧策略在同一个动作上的概率比为 $r_t = \pi_\theta(a_t|s_t)/\pi_{\text{旧}}(a_t|s_t)$，$A_t$ 是优势（这个动作比平均水平好多少）。PPO 的策略目标是：

$$L = \mathbb{E}\left[\min\left(r_t A_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\, A_t\right)\right]$$

$\epsilon$ 常取 0.2。优势为正时，概率比超过 $1+\epsilon$ 就不再给奖励；优势为负时，低于 $1-\epsilon$ 就不再惩罚。这样一批数据可以反复用好几遍（通常 5–10 遍），又不会把策略推得太远。完整的损失里还有价值函数的误差项和一个可选的熵奖励。

**考官想听到**：概率比的含义；min 和 clip 的作用方向；为什么能多次复用同一批数据。

### 6.2 GAE 是什么？$\lambda$ 怎么影响偏差和方差？

**一句话**：GAE（广义优势估计）把多步的时序差分误差按 $\gamma\lambda$ 指数加权求和，在“偏差小”和“方差小”之间折中。

先定义一步的时序差分误差 $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$，优势估计为：

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

- $\lambda = 0$：只看一步，$A_t = \delta_t$。方差小，但完全依赖价值函数，价值函数不准就有偏差。
- $\lambda = 1$：相当于用完整的蒙特卡洛回报减去基线。没有这部分偏差，但方差大。

常用 $\gamma = 0.99$、$\lambda = 0.95$。一个实现细节：回合因为超时被截断（机器人没摔倒，只是时间到了）时，不能把下一个状态的价值当成 0，要用最后一帧的价值去补，否则会把“活到最后”误当成“摔倒”。项目三的代码专门处理了这一点。

**考官想听到**：$\delta$ 和 $A$ 的公式；$\lambda$ 两个极端的含义；超时截断和真正终止的区别。

### 6.3 on-policy 和 off-policy 有什么区别？为什么足式机器人训练多用 PPO？

**一句话**：on-policy 只能用当前策略采的数据更新，off-policy 可以反复用旧数据。PPO 是 on-policy，样本效率低，但稳定、好调，而且能吃满大规模并行仿真。

SAC 这类 off-policy 算法把数据存进回放缓冲区反复使用，样本效率高，适合真机上采样很贵的场景。但在 GPU 并行仿真里，几千个环境同时跑，样本几乎不要钱，这时稳定性和吞吐量更重要。所以从 legged_gym 开始，足式机器人的仿真训练大多用 PPO。

**考官想听到**：两类方法的数据使用方式；为什么在仿真里样本效率不是主要矛盾。

### 6.4 训练四足走路，奖励函数一般有哪些项？

**一句话**：一项主要的“跟踪速度指令”奖励，加一串让动作像样、对机器有好处的惩罚。

常见的项（以 legged_gym 一类框架为例）：

- **跟踪奖励**：线速度、角速度跟上指令，用 $\exp(-\|e\|^2/\sigma)$ 这种形式，误差为 0 时最大；
- **姿态和高度**：机身不要歪、不要太低；
- **能耗和平滑**：关节力矩、关节加速度、相邻两步动作的变化量；

- **步态**：脚的腾空时间（奖励抬脚，防止拖着走），脚打滑的惩罚；
- **安全**：大腿、机身碰地就惩罚或者直接结束回合。

难点不在列出这些项，而在**权重**：跟踪奖励太大，动作会很难看、很耗电；惩罚太大，策略学会原地不动。通常先只用跟踪奖励加少量惩罚训出能走的策略，再逐项加约束。

**考官想听到**：跟踪奖励的形式；三四类正则项；权重怎么调的实际经验。

### 6.5 什么是奖励作弊？举个例子，怎么发现？

**一句话**：策略找到了奖励函数的漏洞，分数很高，但行为不是你想要的。

足式机器人里常见的例子：用膝盖或者机身蹭着地前进（不容易摔，又能跟上速度）；脚在地面上高频振动来“刷”腾空时间；利用仿真接触模型的瑕疵，在某个奇怪的姿态下获得额外推力。

发现的办法：

1. **一定要看回放视频**，不能只看总奖励曲线；
2. 把每一项奖励**分开记录**，看是哪一项在涨；
3. 检查物理上不合理的指标：关节速度、力矩、接触力有没有异常峰值；
4. 换一个仿真器验证（Sim2Sim），利用仿真瑕疵学出来的动作往往在另一个仿真器里立刻失效。

**考官想听到**：一个具体的例子；“分项记录加看视频”的习惯；作弊行为和仿真瑕疵的关系。

### 6.6 动作空间为什么常用“关节目标角度加底层 PD”，而不是直接输出力矩？

**一句话**：输出目标角度，让高频的 PD 控制器去处理刚度和阻尼，策略本身可以在较低频率（比如 50 Hz）运行，学起来更容易，动作也更平滑。

直接输出力矩，策略必须自己学会“弹簧和阻尼”，并且要以很高的频率运行才稳定，探索空间也大得多。用目标角度加 PD 的方式，相当于给策略加了一个合理的先验。代价是 PD 的 kp、kd 成了训练环境的一部分，**仿真里用什么增益，真机上就必须用一样的**，否则策略的行为会变。

动作通常还要乘一个缩放系数、加在默认站立姿态上：$q_{\text{目标}} = q_{\text{默认}} + s \cdot a$。这个缩放系数、默认姿态、关节顺序，是策略部署时最容易配错的三样东西。

**考官想听到**：低频策略加高频 PD 的分工；增益要和真机一致；动作缩放和默认姿态。

## 6.7 什么是域随机化？随机化哪些参数？范围怎么定？

**一句话**：训练时让仿真环境的物理参数在一个范围内随机变化，逼策略学会对这些变化不敏感，这样真机和仿真不一样时，它也能应付。

常见的随机化对象：质量和质心位置、地面摩擦、电机力气、关节阻尼和摩擦、控制延迟、传感器噪声、地形高低，以及随机推一把。

范围怎么定：从测量值出发加余量。比如实测摩擦系数在 0.6–0.9 之间，就随机到 0.4–1.1。范围太窄，真机一变就失效；范围太宽，策略会变得过分保守，平地上也走得畏畏缩缩。

项目三做了一组对照（3 个随机种子取平均，每个策略在每种环境里测 20 个回合）。不加随机化和加随机化的策略，在原始环境里得分差不多（2,792 分和 2,696 分），但换了环境就拉开了：躯干重 1.5 倍时是 1,031 分对 1,680 分，地面摩擦减半时是 364 分对 691 分，电机力气只剩八成时是 427 分对 1,327 分。

**考官想听到**：随机化哪些东西；范围的取舍；“原始环境里不吃亏、变了环境才见差距”这个规律。

## 6.8 Sim2Real 的差距通常来自哪里？怎么缩小？

**一句话**：最大的差距往往在执行器（电机和减速器）的动态，其次是接触、延迟和传感器。

主要来源：

1. **执行器**：真实电机有电流限制、发热降额、摩擦、背隙、带宽有限，仿真里的“理想力矩源”没有这些。
2. **延迟**：从传感器读数到电机出力，真机上有几毫秒到十几毫秒的延迟。
3. **接触**：仿真的接触模型是简化的，地面的软硬、摩擦和真实世界不一样。
4. **传感器**：IMU 噪声和零偏、编码器量化、状态估计的误差。
5. **模型参数**：质量、惯量、质心位置不准。

缩小的办法：系统辨识，把参数测准；对执行器单独建模，经典做法是苏黎世联邦理工学院 2019 年用真机数据训练一个“执行器网络”放进仿真；域随机化覆盖剩下的不确定性；先在另一个仿真器里做 Sim2Sim 验证，再上真机。

**考官想听到**：把执行器排在第一位；能说出执行器网络这类做法；Sim2Sim 是上真机前的必经一步。

### 6.9 什么是教师-学生训练（特权学习）？为什么有效？

**一句话**：先在仿真里训练一个“开卷考试”的教师策略，它能看到真机上拿不到的信息（地形高度、摩擦系数、接触状态）；再训练一个只看真实传感器的学生策略，去模仿教师的动作。

直接用真实传感器的观测训练很难：机器人要从关节和 IMU 的历史数据里自己推断“我踩的是什么地面”，强化学习的信号太弱。教师有特权信息，很容易学好；学生的任务变成监督学习（模仿教师），容易得多。学生通常要看一段历史观测，从中隐式地估计地形和摩擦。

这条路线的代表是苏黎世联邦理工学院 2020 年发表在《科学·机器人学》上的复杂地形四足行走工作；2021 年的 RMA 用类似的思路，让学生在线估计环境参数，实现快速适应。

**考官想听到**：教师和学生各看什么；为什么学生要看历史；和 DAgger 的关系（学生的训练数据最好来自学生自己的轨迹）。

### 6.10 为什么要几千个环境同时训练？并行环境数怎么影响训练？

**一句话**：PPO 每次更新都要新鲜的数据，GPU 并行仿真一次跑几千个环境，几分钟就能采到 CPU 上要跑几小时的数据量。

2021 年的 legged_gym 工作把几千只机器狗放在一块 GPU 上同时仿真：平地上的策略大约 4 分钟训出来，崎岖地形大约 20 分钟。并行环境数和每个环境每次采样的步数，决定了一批数据的大小：

- 环境多、每个环境步数少：数据多样，但每段轨迹短，长时间才见效的奖励不容易学到；
- 环境少、步数多：轨迹长，但数据相关性强，梯度方差大。

作为对照，项目三在 4 核笔记本的 CPU 上，单个环境每秒只能采约 1,200 步，训练 100 万步要 14 分钟左右。这就是为什么做足式机器人的团队都离不开 GPU 仿真。

**考官想听到**：并行仿真的意义；批量大小和轨迹长度的权衡；对训练时间有数量级的概念。

### 6.11 训练曲线不涨，或者涨了又崩了，你怎么排查？

**一句话**：先排查“工程 bug”，再怀疑“算法”。

按顺序查：

1. **奖励的符号和尺度**：惩罚项写成了奖励？某一项比其他项大几个数量级？

2. **观测归一化**：没有归一化，或者训练和测试用的统计量不一样；
3. **动作**：缩放是否合理，有没有被截断；
4. **终止条件**：太严，机器人一开始就全部结束；太松，学会了“趴着拿分”；
5. **策略的标准差**：探索噪声塌缩到 0，策略就不再探索；
6. **价值函数的损失**：一直很大，说明优势估计不可信；
7. **数值问题**：NaN、梯度爆炸，学习率太大。

有效的习惯：先在最简单的设置下跑通（平地、固定指令），再逐项加复杂度；固定随机种子，保证结果能复现；每次只改一个东西。

**考官想听到**：有顺序的排查清单；“先工程后算法”；能复现、一次改一处的习惯。

### 6.12 策略在 Isaac 里走得很好，放到 MuJoCo 里就站不住，可能是什么原因？

**一句话**：十有八九是两边的配置不一致，而不是策略不行。

常见的不一致：

- **关节顺序**：两个仿真器、两份模型文件里，关节的排列顺序可能不同；
- **正负号和零位**：关节转动方向、零位定义不同；
- **默认姿态、动作缩放、PD 增益**：任何一个对不上，策略输出的含义就变了；
- **控制频率和降采样**：策略 50 Hz、PD 1 kHz，降采样倍数写错；
- **观测**：重力向量的坐标系、角速度用的是机身坐标系还是世界坐标系、观测的缩放系数；
- **接触和求解器**：摩擦系数、仿真步长、接触参数。

排查办法：在两个仿真器里放同一个初始状态，喂同一串动作，逐个对比观测向量的每一维，第一个不一致的地方就是线索。《拆解宇树》第 19 章的 Sim2Sim 步骤，就是为了在上真机之前把这些错找出来。

**考官想听到**：关节顺序、默认姿态、增益、频率这几个高频坑；逐维对比观测的方法。

> **考官视角｜讲一个失败**
>
> 强化学习岗的面试里，“你遇到过最难查的问题是什么”几乎必问。最好的答案是一个具体的失败：现象是什么，你怎么定位，最后是什么原因，怎么验证修好了。比如项目三里，超时截断如果当成摔倒处理，价值估计会出偏差，这就是一个可以讲清楚的坑。只讲成功、不讲失败的候选人，面试官很难判断你是不是真的训练过。

## 从哪些背景转过来最顺

- **强化学习方向的研究生**：算法现成，补机器人动力学和真机部署。
- **游戏 AI、推荐系统里做过强化学习的工程师**：训练和调参的经验能直接用上，要补的是物理仿真和机器人常识。
- **自动驾驶决策规划**：熟悉仿真、评测和长尾问题；补策略梯度和足式机器人的特点。
- **运动控制工程师**：懂机器人、懂真机，补强化学习后在 Sim2Real 上很有优势，这类复合型的人最抢手。

**转行者笔记｜GPU 不是门槛**

很多人以为没有 GPU 就没法练强化学习。项目三只用笔记本的 CPU，半小时就能跑完一组有意义的对照实验。面试官看重的是你会不会设计实验、会不会读曲线、会不会查问题，而不是你用了多少张卡。

**动手试试｜先跑一个短的**

把项目三的训练步数改成 20 万步，十分钟以内就能看到 Hopper 学会往前跳。然后回放一遍，找找它有没有“作弊”的动作。

---

**本章一句话**

**会调奖励不稀奇，能把策略搬上真机才值钱。面试里讲清楚一次失败，比讲十次成功更有说服力。**

---

运控和强化学习解决“怎么动”。下一章是“在哪动”：机器人怎么知道自己在哪、脚下是什么。

第 7 章

# 感知与定位岗：标定、SLAM 和状态估计

做自动驾驶定位的工程师去面机器狗公司，常被问到的第一个问题是：“你觉得腿足机器人的定位，和车有什么不一样?”

这个问题答好了，后面就顺了。不一样的地方至少有四个：

1. **机身在晃**：每一步落地都是一次冲击，IMU 噪声大，相机图像容易糊；
2. **没有轮速计，但有腿**：支撑脚和地面基本不动，用关节角能算出机身速度（腿式里程计），只是脚一打滑就算错；
3. **要看脚下**：车只关心前方的路和障碍物，腿足机器人还要知道每只脚下一步踩在哪，台阶边缘、坑和碎石都要看清；
4. **算力有限**：所有东西都跑在机器人自己的计算板上，还要和运动控制抢 CPU。

这一章按这个思路讲感知与定位岗，最后的作品集项目是第 12 章的项目四：用卡尔曼滤波融合 IMU 和腿式里程计，再加上打滑检测。

## 这个岗位的一天

一个典型的一天（不是某个真人）：

上午重新标定一台样机的激光雷达和 IMU 外参，因为上周换了雷达支架。中午处理一份现场日志：机器人在玻璃幕墙旁边定位跳了一下，原因是激光打在玻璃上产生了虚假的点。下午优化高度图的更新速度，让机器人上台阶时能更早看清台阶边缘。晚上和运控同事对接：他们希望状态估计输出的机身速度再平滑一点，但延迟不能增加。

## 招聘启事里怎么写

- 学历：本科或硕士，多要求几年相关经验；

- 理论：多传感器标定和融合，定位与建图，卡尔曼滤波、因子图优化；
- 工程：C++、ROS，PCL、OpenCV；
- 加分：腿足机器人或自动驾驶的定位经验，地形感知，全局和局部路径规划。

## 知识地图

表 7-1 感知与定位岗的知识地图

| 必会 | 加分 | 可以先放一放 |
| --- | --- | --- |
| 相机模型、内参外参和畸变、标定 | 视觉惯性里程计（MSCKF、VINS 一类） | SLAM 的理论证明 |
| 手眼标定 | 激光惯性里程计（FAST-LIO、LIO-SAM 一类） | 自己实现稀疏求解器 |
| IMU 误差模型 | 因子图和 GTSAM | |
| 卡尔曼滤波、EKF、误差状态卡尔曼 | 学习式深度估计、语义地图 | |
| 点云配准（ICP、NDT） | 多层地图、三维占据栅格 | |
| SLAM 的前端、后端、回环 | | |
| 时间同步、高度图 | | |

## 面试题和参考答案

### 7.1 相机的内参、外参和畸变各是什么？怎么标定？

**一句话**：内参描述相机自己（焦距、主点），畸变描述镜头把直线拍弯的程度，外参描述相机相对别的东西（机身、另一个传感器）的位姿。

针孔模型下，三维点投影到像素是 $u = f_x \frac{X}{Z} + c_x$，$v = f_y \frac{Y}{Z} + c_y$，$f_x, f_y, c_x, c_y$ 就是内参。畸变一般用径向畸变（几个系数 $k_1, k_2, \ldots$）加切向畸变两类参数描述。

标定内参和畸变的标准做法是张正友方法：从不同角度拍十几到几十张棋盘格，用已知的格子尺寸解出参数，再用非线性优化精修。好不好看重投影误差，一般要在半个像素以内。拍的时候要让棋盘覆盖画面的各个角落、倾斜不同的角度，只拍正面中间是标不准边缘畸变的。

**考官想听到**：投影公式；张正友方法；重投影误差；数据采集的注意事项。

## 7.2 手眼标定是什么？$AX = XB$ 是怎么来的？

**一句话**：求相机和机械臂法兰（或者机身）之间的固定变换 $X$。

以相机装在机械臂末端为例。让机械臂摆几个不同的姿态，每次都拍同一个固定的标定板：

- 两个姿态之间，法兰的相对运动 $A$ 由正运动学算出；
- 同样两个姿态之间，相机的相对运动 $B$ 由拍到的标定板算出；
- 相机和法兰之间的变换 $X$ 不变，所以“法兰动了 $A$ 再到相机”等于“先到相机、相机动了 $B$”：$AX = XB$。

经典解法先解旋转再解平移（比如 Tsai-Lenz 方法）。注意事项：姿态之间的旋转要绕几个不平行的轴，只绕一个轴转会让问题退化，解不出唯一的 $X$；姿态数量要够，通常十组以上。

**考官想听到**：$A$、$B$、$X$ 各是什么、从哪来；旋转轴不能平行；先旋转后平移。

## 7.3 IMU 的误差模型是什么？零偏怎么估计？

**一句话**：测量值等于真实值加零偏加白噪声，零偏本身还在慢慢漂（随机游走）。

$$\tilde{a} = a + b_a + n_a, \qquad \tilde{\omega} = \omega + b_g + n_g, \qquad \dot{b} \sim \text{白噪声}$$

零偏的危害在于积分：加速度计零偏积分两次，位置误差随时间平方增长。项目四里，0.08 $\mathrm{m/s^2}$ 的零偏在 30 秒后让纯 IMU 积分的位置差了 35.7 米。

估计零偏的办法：把它放进滤波器的状态里，让其他传感器的测量去校正它。项目四的卡尔曼滤波就是这样做的，30 秒后零偏估计为 0.079，真值是 0.08。IMU 的噪声参数可以用 Allan 方差分析测出来，静止放几个小时记录数据即可。

**考官想听到**：误差模型的三项；积分后误差的增长方式；零偏放进状态里估计；Allan 方差。

## 7.4 卡尔曼滤波的预测和更新各做什么？Q 和 R 怎么调？

**一句话**：预测用运动模型把状态和不确定度往前推，更新用测量把状态拉回来，拉多少由两边的不确定度决定。

- **预测**：$x \leftarrow Fx + Bu$，$P \leftarrow FPF^{\top} + Q$。不确定度只增不减。

- **更新**：新息 $y = z - Hx$，新息协方差 $S = HPH^\top + R$，增益 $K = PH^\top S^{-1}$，然后 $x \leftarrow x + Ky$，$P \leftarrow (I - KH)P$。

Q 是过程噪声（模型有多不准），R 是测量噪声（传感器有多不准）。Q 相对大，滤波器更相信测量，响应快但更抖；R 相对大，更相信模型，平滑但滞后。初值从传感器手册和标定结果来，再看滤波器是否“自洽”：新息的马氏距离平方 $y^2/S$ 平均下来应该接近测量的维数。

**考官想听到**：五个公式能写出来；Q、R 的物理含义和调大调小的效果；用新息检验滤波器是否自洽。

## 7.5 EKF 和误差状态卡尔曼有什么区别？姿态为什么常用误差状态？

**一句话**：EKF 直接对整个状态线性化；误差状态卡尔曼（ESKF）把状态分成“名义值”和“小误差”，只对小误差做线性化和滤波，再把误差注入名义值。

姿态用误差状态的好处：

1. 四元数有单位长度约束，直接放进 EKF 会让协方差矩阵奇异；误差状态用一个三维的小旋转向量表示，没有约束；
2. 误差总是在 0 附近，线性化更准；
3. 名义状态可以用高频的 IMU 精确积分，滤波器只管误差，结构清晰。

这也是大多数视觉惯性、激光惯性里程计里的做法。

**考官想听到**：名义状态加误差状态的结构；四元数约束的问题；误差状态线性化更准。

## 7.6 腿式里程计怎么算？脚打滑了怎么办？

**一句话**：假设支撑脚在地面上不动，用关节角和角速度通过运动学算出机身速度；打滑时这个假设不成立，要靠检验把错误的测量挡在外面。

对一只支撑脚，足端在机身坐标系里的位置是 $r(q)$，速度是 $J(q)\dot{q}$。脚不动，就意味着机身速度满足 $v_{\text{机身}} = -J(q)\dot{q} - \omega \times r(q)$（机身坐标系下）。几只支撑脚各算一个，取平均或者都当作测量送进滤波器。

打滑时这个测量会突然离谱。处理办法：

1. **卡方检验**：新息的马氏距离平方超过阈值（一维时常用 9，约等于 $3\sigma$）就丢掉这次测量；
2. **接触概率**：结合足端力或关节力矩判断脚是否真的着地；
3. **加大 R**：在不可靠的阶段（刚落地、快抬脚时）给腿速更大的测量噪声。

项目四对比了不同做法，30 秒里的位置均方根误差：直接积分腿速 0.63 m；卡尔曼融合但不设门限 0.34 m；加上卡方门限以后只有 0.012 m。门限挡掉了 256 个腿速样本，其中 240 个确实是打滑时的。

**考官想听到**：“支撑脚不动”的假设；速度公式；卡方门限的原理；门限的风险（滤波器本身发散时，门限会把好的测量也全挡掉）。

## 7.7 ICP 的原理是什么？什么时候会失败？

**一句话**：迭代最近点：给两帧点云找对应点（最近邻），求让对应点距离最小的刚体变换，变换以后再找对应点，反复迭代直到收敛。

已知对应点时，最优旋转和平移可以用 SVD 一步解出来。“点到平面”的变体用目标点云的法向量，收敛更快；NDT 把点云变成一格一格的正态分布再配准，对初值没那么敏感。

失败的典型场景：

- **初值太差**：最近邻找错了对应点，收敛到错误的位置；
- **几何退化**：长走廊、空旷平地，某个方向上没有约束，配准沿这个方向滑动；
- **动态物体**：行人、车辆把配准往错误的方向拉；
- **重叠太少**：两帧看到的东西大部分不一样。

**考官想听到**：两步交替的过程；SVD 求解；至少两种失败场景，尤其是退化。

## 7.8 SLAM 里为什么需要回环检测？回环错了会怎样？

**一句话**：里程计的误差会一直累积，回到走过的地方时认出来（回环），就能把累积的误差一次性分摊回整条轨迹上。

回环检测先用外观（图像特征、点云描述子）找“这里好像来过”的候选，再用几何验证确认，最后作为约束加进图优化。

**错误的回环比没有回环危险得多**：一个错误的约束会把地图的两部分硬拉到一起，整张图被扭曲，而且很难自动恢复。所以回环要严格验证，后端用鲁棒核函数，或者可以自动关掉的“可切换约束”，降低单个错误约束的影响。

**考官想听到**：回环解决漂移；验证的步骤；错误回环的后果和防范。

## 7.9 多传感器的时间同步怎么做？不同步会怎样？

**一句话**：最好用硬件触发或者网络授时（PTP）让传感器共用一个时钟，在采集的那一刻打时间戳；做不到就要估计并补偿每个传感器的延迟。

不同步的后果在运动快的时候最明显：机身转动时，几十毫秒的时间差就会让激光点云和相机图像对不上，融合出来的地图出现重影，速度估计也会偏。腿足机器人机身一直在晃，对同步的要求比低速的轮式机器人更高。

做法上：每个数据都用采集时刻而不是到达时刻的时间戳；异步的传感器用插值对齐到同一时刻；有些滤波器把时间偏差也作为状态一起估计。

**考官想听到**：硬件同步和软件补偿两类方法；用采集时刻打时间戳；快速运动下不同步的后果。

### 7.10 腿足机器人的地形感知为什么常用 2.5D 高度图？它有什么局限？

**一句话**：高度图把地面分成一个个小格，每格存一个高度和它的不确定度，正好回答“这只脚踩下去是什么高度、平不平”，计算量也小。

它由深度相机或激光雷达的点云，结合机身的位姿逐帧更新。落脚点规划和强化学习策略都可以直接用它：取机身周围一圈格子的高度作为观测。《拆解宇树》第 14 章讲过它的原理。

局限：

- **只能存一个高度**：桌子底下、楼梯背面这种上下两层的结构表示不了；
- **软的东西看着是硬的**：草地、雪地、沙子在高度图里和水泥地一样；
- **反光和透明表面**：玻璃、水面让深度测量出错；
- **依赖位姿**：定位漂了，高度图就会“错层”。

**考官想听到**：高度图回答的问题；更新方式；至少三条局限。

> **考官视角｜别只讲算法名字**
>
> 这个岗位的候选人很多来自自动驾驶，简历上写满了算法名字。面试官更想听的是：你在哪种场景下用过、指标是多少、失败过什么。比如“在长走廊里 ICP 沿走廊方向滑动，我们加了 IMU 的约束”，比“熟悉 ICP、NDT、LOAM、LIO-SAM”有用得多。

## 从哪些背景转过来最顺

- **自动驾驶定位和感知**：最顺。标定、融合、点云、时间同步都能直接用上；要补的是腿式里程计、地形感知和机载算力的限制。
- **扫地机器人、AGV 的 SLAM**：室内场景和低成本传感器的经验很对口。
- **AR/VR 的视觉惯性里程计**：视觉惯性融合的经验直接可用。
- **无人机**：IMU、滤波、实时性都是强项。

**转行者笔记｜用“不一样的四点”开场**

面试时被问到“为什么想转机器人”，可以用这一章开头的四点来回答：你知道腿足机器人的定位难在哪，也知道自己的哪些经验正好能解决这些难点。这比泛泛地说“看好行业”有说服力得多。

**动手试试｜调一下门限和 R**

跑项目四，把卡方门限从 9 改成 4 和 25，再把腿速的测量噪声 R 放大 10 倍，看位置误差和被拒绝的样本数怎么变。你会亲眼看到门限太紧和太松各有什么问题。

**本章一句话**

**先把“自己在哪、走了多远”算准。腿足机器人的感知，难点在晃动、打滑和脚下的地形。**

运动控制、强化学习、感知，都是让机器人“自己动”的岗位。下一章是这两年最热的方向：让机器人用手干活、听懂指令的具身大模型。

第 8 章

# 操作与具身大模型岗：模仿学习、VLA 和数据

“具身大模型”是这两年最热的词。很多人以为这个方向招的全是训练大模型的算法博士。

回到第 1 章的表 1-1：宇树官网在招的 22 个技术岗里，和大模型直接相关的算法岗只有 1 个，数据相关的岗位却有 4 个。这不是宇树一家的特点：要让机器人用手干活，最缺的往往不是模型结构，而是高质量的示范数据，以及能把数据采好、管好、用好的人。

这一章讲这个方向的算法岗和数据岗：要会什么，面试问什么。对应的作品集项目是第 12 章的项目五（机械臂逆运动学）和项目六（模仿学习里的“多峰”问题）。

## 这个岗位的一天

一个典型的一天（不是某个真人）：

上午检查昨天采的 200 条“叠毛巾”示范：有 60 条不能用，有的相机被手臂挡住了，有的操作员中途犹豫了好几秒。下午用新数据微调一个 VLA 模型，在真机上测 50 次，成功率从 38% 涨到 42%。你犹豫要不要在周会上说“提升了”，因为 50 次的波动可能比 4 个百分点还大（见 8.8 题）。晚上排查数据管线：有一批数据的关节角和图像时间戳差了 80 毫秒。

## 招聘启事里怎么写

- 学历：硕士、博士居多，计算机、人工智能相关专业；
- 算法：Transformer，模仿学习，扩散策略、ACT 一类方法，多模态大模型（VLM）；
- 工程：PyTorch，多机多卡训练；

- 职责：VLA 模型的训练、微调、评测，遥操作数据采集方案和数据闭环，机械臂和灵巧手上的真机部署；
- 加分：顶会论文。

数据岗的要求门槛低得多：会用 Python 处理数据、熟悉 Linux、做事细致，了解机器人或遥操作设备的优先。

## 知识地图

表 8-1 操作与具身大模型岗的知识地图

| 必会 | 加分 | 可以先放一放 |
| --- | --- | --- |
| 行为克隆、分布偏移、DAgger | 强化学习微调 VLA | 从零预训练大语言模型 |
| 多峰问题和几种解法 | 世界模型 | |
| Transformer、视觉编码器 | 多机多卡训练、混合精度 | |
| 扩散策略、动作分块 | 模型量化和端侧部署 | |
| VLA 的结构和动作表示 | 跨形态数据 | |
| 遥操作方式和数据质量 | 触觉和力的数据 | |
| 评测方法和置信区间 | | |
| 机械臂运动学、抓取基础 | | |

## 面试题和参考答案

### 8.1 行为克隆为什么会“误差累积”？DAgger 怎么解决？

**一句话**：行为克隆只在专家走过的状态上学习，学生一旦犯一点错，就进入专家没去过的状态，在那里它更容易出错，错误一步步放大。

形式上，训练数据的状态分布来自专家，测试时的状态分布来自学生自己，两者不一样，这叫分布偏移（协变量偏移）。理论分析表明，最坏情况下累积误差随任务长度平方增长。

DAgger 的做法：让学生自己去执行，记录它走过的状态，请专家给这些状态补标注“这里该怎么做”，合进数据重新训练，反复几轮。这样训练数据就覆盖了

学生自己会走到的地方。实际中请专家在线标注很贵，常见的替代办法有：采示范时故意加扰动，让专家演示怎么纠偏（DART 的思路）；人在机器人出错时接管，把接管的片段加进数据。

要补一句我们自己做实验时看到的现象：在一个低维、平滑的小车循迹任务上，示范里只要有几厘米的纠偏动作，普通的行为克隆就已经 100% 成功，看不出 DAgger 的优势。分布偏移真正致命的，是高维的图像输入和长时序的任务：网络在没见过的画面上很难像在低维状态上那样平滑地外推。

**考官想听到**：分布偏移的定义；平方增长的结论；DAgger 的流程和代价；知道什么场景下这个问题严重。

## 8.2 什么是示范的“多峰”问题？为什么均方误差回归会出事？

**一句话**：同一个状态下有好几种都对的做法，均方误差会学到它们的平均，而平均值本身可能是错的。

经典例子：前面有个障碍物，有的示范从左边绕，有的从右边绕。均方误差最小的预测是“左右的平均”，也就是直着撞上去。

项目六专门做了这个实验：一个点状机器人绕过障碍物，专家一半从上面绕、一半从下面绕。40 条示范时：

- 均方误差回归，61 个起点里撞了 8 次；
- 把方向离散成 16 个格子当分类问题学，撞了 0 次；
- 把“接下来 8 步”整段动作聚成 32 类再分类，也是 0 次。

解法的共同点是**让模型表示一个分布，而不是一个平均值**：离散成 token 再分类（RT-2 这类模型）；混合高斯；用 k 均值聚类或者向量量化把动作变成 token（Behavior Transformer、VQ-BeT）；扩散模型或流匹配直接建模连续的动作分布（扩散策略、$\pi 0$）。

**考官想听到**：平均值为什么错；至少三种解法；能讲清楚自己做过的对照实验。

## 8.3 扩散策略（Diffusion Policy）是什么？为什么在机器人上效果好？

**一句话**：把“生成一段动作”看成去噪过程：从随机噪声出发，在观测（图像、机器人状态）的条件下一步步去噪，得到一段未来的动作序列。

它效果好有三个原因：

1. **能表示多峰分布**，不会把几种做法平均掉；
2. **一次生成一段动作**（比如未来十几步），执行一部分再重新生成，动作连贯；
3. **训练稳定**，不像生成对抗网络那样难调。

代价是推理慢：标准的扩散模型要迭代几十步去噪。常见的加速办法是用更少的去噪步数（DDIM 一类采样器）、蒸馏，或者换成流匹配（π0 用的就是流匹配）。这个方法由哥伦比亚大学等团队在 2023 年的 RSS 会议上提出。

**考官想听到**：去噪生成动作的基本过程；多峰、动作序列、稳定三个优点；推理延迟和加速办法。

### 8.4 ACT 的“动作分块”解决什么问题?

**一句话**：一次预测未来一段（比如几十步）动作，而不是一步，从而减少误差累积，让动作更连贯。

ACT 来自 2023 年的 ALOHA 工作（低成本双臂遥操作平台）。分块的好处：

- **有效决策次数变少**：每 $k$ 步才做一次决策，误差累积的机会少了；
- **能表达停顿、犹豫这类不满足马尔可夫性的动作**：人在示范里经常停一下再动，单步策略很难学；
- **时间集成**：相邻几次预测的动作块有重叠，加权平均以后更平滑。

ACT 还用了一个条件变分自编码器（CVAE），用一个隐变量表示“这一次示范用的是哪种风格”，生成时固定这个隐变量，同一段动作前后一致。

项目六里也能看到“前后一致”的重要性。示范增加到 200 条、上下两种绕法几乎五五开以后，每一步都重新挑方向的分类策略开始在分叉点附近来回切换，61 次里撞了 5 次；把整段动作当成一个 token 来选，一次也没撞。

**考官想听到**：分块减少误差累积的原因；时间集成；CVAE 的作用；前后一致为什么重要。

### 8.5 VLA 模型一般由哪几部分组成？动作是怎么输出的?

**一句话**：视觉编码器 + 预训练的语言模型骨干 + 动作输出；动作要么变成离散的 token 让语言模型像说话一样“说”出来，要么接一个专门输出连续动作的头。

两种主流的动作表示：

- **离散 token**：把动作的每一维离散成固定数量的格子，当成文字 token。RT-2 把动作写成文字 token，直接混进视觉语言模型的训练数据里。开源的 OpenVLA 也是这条路：70 亿参数，以 Llama 2 语言模型为骨干，视觉编码器融合了 DINOv2 和 SigLIP 的特征，用 97 万条真实机器人示范训练，在论文的测试中比 550 亿参数的 RT-2-X 绝对成功率高 16.5 个百分点。缺点是一个 token 一个 token 地生成，慢。

- **连续动作头**：在视觉语言模型后面接一个专门的“动作专家”，用扩散或流匹配直接生成连续的动作块。π0 就是这样：在预训练的视觉语言模型之上加流匹配，用单臂、双臂和移动操作机器人的数据训练，能叠衣服、收拾桌子、组装纸箱。

**考官想听到**：三个组成部分；两种动作表示的取舍（离散好接语言模型、连续更精确更快）；能说出一两个代表模型和它们的关键设计。

## 8.6 遥操作采数据有哪些方式？各有什么优缺点？

**一句话**：主从臂、VR 加手部追踪、动作捕捉手套、手把手拖动示教，各有取舍：精度、成本、速度、操作员疲劳，以及人手和机械手的差别。

- **主从臂**（ALOHA 这一类）：操作员拖动一套和机器人同构的主臂，机器人跟着动。动作映射直接、精度高、便宜；但只适合和主臂同构的机器人。
- **VR 加手部追踪**：戴头显，用手势控制。灵活，适合人形和灵巧手；但没有力反馈，精细操作难。
- **动作捕捉手套**：采灵巧手的手指动作。手指信息丰富；但人手和机械手的结构不同，需要“重定向”。
- **拖动示教**：直接用手拖着机械臂走。最直观；但操作员的手会出现在画面里，而且很累。

不管用哪种，都要考虑**操作员疲劳**和**一致性**：同一个任务，不同操作员的做法可能不一样，这正是多峰问题的来源之一。

**考官想听到**：几种方式的取舍；人手和机器手的差异；操作员带来的数据问题。

## 8.7 怎么判断一批示范数据的质量？哪些该丢掉？

**一句话**：先自动筛掉“坏数据”，再看“好数据”有没有覆盖足够多的变化。

该丢掉或者修正的：

- 任务没成功的（除非专门要失败数据）；
- 传感器问题：丢帧、相机被遮挡、时间戳对不上；
- 长时间停顿、犹豫、反复试错的片段（可以截掉）；
- 动作明显不平滑、有异常跳变的。

还要检查覆盖面：物体的位置、朝向、光照、背景有没有变化；一种做法和另一种做法的比例。数据的**多样性**往往比数量更重要：同一个位置拍一千遍，不如一百个位置各拍十遍。

自动检查之后，最好再抽一部分人工看视频。

**考官想听到**：坏数据的几类；多样性比数量重要；自动检查加人工抽查。

### 8.8 真机上测 50 次，新模型成功率 42%、旧模型 38%，能说明新模型更好吗？

**一句话**：不能。50 次试验时，成功率本身的波动就有十几个百分点。

成功率是二项分布的比例，标准误差约为 $\sqrt{p(1-p)/N}$。$p = 0.4$、$N = 50$ 时约为 0.069，95% 置信区间大约是正负 14 个百分点。4 个百分点的差距完全在噪声里。

更可靠的做法：

1. **增加次数**，或者至少报告置信区间；
2. **配对评测**：两个模型在完全相同的一组初始条件下各测一遍，比较每一对的结果；
3. **按失败原因分类**：新模型减少的是哪一类失败？
4. 先在仿真或离线指标上筛选，真机只测最有希望的几个。

**考官想听到**：能算出标准误差的量级；配对评测；不只看总成功率，还要看失败模式。

### 8.9 什么是数据配比？为什么要混入互联网数据或者别的机器人的数据？

**一句话**：训练数据里各个来源占多少比例，叫数据配比。混入别的数据，是为了让模型有更好的通用知识，也让它见过更多的场景。

- **互联网的图文数据**：VLA 的骨干是在互联网图文上预训练的视觉语言模型，它带来“这是杯子”“把它放到盘子左边”这些语义知识；微调时继续混一部分图文数据，可以减少遗忘。
- **别的机器人的数据**：Open X-Embodiment 汇集了二十多种机器人的上百万条轨迹。不同机器人的动作空间不一样，混合训练要统一表示，但能显著提升泛化。
- **自己的机器人数据**：最贴近目标任务，比例太低学不会，太高又会过拟合到少数场景。

配比没有通用答案，通常要在一组验证任务上扫几种比例来定。

**考官想听到**：为什么要混；混的风险（遗忘、负迁移）；配比靠实验定。

### 8.10 VLA 模型部署到机器人上，推理太慢怎么办？

**一句话**：从“让模型更小更快”和“少推理几次”两个方向下手。

- **让模型更快**：量化（比如 8 位或 4 位整数）、蒸馏成小模型、用推理加速框架、换用专门的车载或机载芯片；
- **少推理几次**：动作分块，一次推理执行多步；视觉编码器降频运行；语言指令只编码一次缓存起来；
- **并行**：推理和执行异步，机器人在执行上一段动作时，下一段已经在算；
- **扩散类模型**：减少去噪步数，或者换成流匹配、一致性模型。

部署前先量清楚每一部分的耗时（视觉编码、语言模型、动作头、通信），找到瓶颈再优化。

**考官想听到**：多个方向的优化手段；动作分块和异步执行；先测量后优化。

> **考官视角｜别把论文名当答案**
>
> 这个方向的候选人最容易犯的错，是把一串论文名当成能力：“熟悉 RT-2、OpenVLA、π0、扩散策略……”。面试官会接着问：“你自己训过吗？数据怎么来的？成功率怎么测的？失败的样本长什么样？”能用项目六这种小实验讲清楚“均方误差为什么会撞上去”，比背十篇论文更有说服力。

## 从哪些背景转过来最顺

- **大模型、多模态、CV 算法工程师**：模型和训练是强项，补机器人的数据、评测和真机部署。
- **自动驾驶端到端**：从驾驶日志模仿人类驾驶，和机器人的模仿学习是同一个问题，分布偏移、多峰、闭环评测的经验都能直接用。
- **工业机器人、机械臂集成**：运动学、抓取、标定是现成的，补模仿学习。
- **数据标注、数据运营、数据开发**：数据岗最顺的来路，门槛低，进去以后可以往数据评估、数据配方的方向走。

> **转行者笔记｜从数据岗进门是一条真实的路**
>
> 如果你够不着 VLA 算法岗，可以考虑数据采集、数据管线、数据评估这些岗位。它们离模型最近，天天看模型在哪些数据上失败，往算法方向走的机会比在别的公司从头准备要大。

> **动手试试｜自己算一个置信区间**
>
> 找一篇机器人论文里的真机实验，记下它的试验次数和成功率，按 8.8 题的公式算一下 95% 置信区间。你会发现很多“提升”其实在误差范围内。

**本章一句话**

**具身大模型方向，招的很多是“会做数据的人”。会用数据和实验说话，比会背论文名更值钱。**

下一章回到机器人的本体：结构、关节、电控和嵌入式，这是传统行业工程师最容易接上的一层。

第 9 章

# 硬件岗：结构、关节、电控和嵌入式

第 1 章提到，猎聘的报告把“硬件与系统工程人才”列为和算法并列的紧缺方向。再看宇树官网：在招的技术岗里，嵌入式、硬件和机电相关的有 5 个，机械和工业设计 2 个，加起来比算法岗还多。

这一层是传统行业的工程师最容易接上的地方。做过新能源汽车电控的人懂电机，做过手机结构的人懂量产，做过汽车电子的人懂总线和功能安全。机器人公司最头疼的恰恰是这些：样机能跑，量产以后关节一批批地坏；实验室里能走，客户现场爬了二十分钟楼梯就过热保护。

这一章的动手环节不需要写代码：读一份真实的关节电机参数表，算几个数。

## 这个岗位的一天

一个典型的一天（不是某个真人）：

上午，一批膝关节在热测试里不合格：连续爬楼梯 20 分钟后触发过热保护。你调出电流和温度曲线，发现问题集中在机身较重的那一侧。中午量了供应商新送来的减速器样品的背隙，比规格书大了一倍。下午给 12 个关节批量升级固件，其中一个总是升级失败，最后查到是它的引导程序版本太老。

## 招聘启事里怎么写

- **机械**：本科，机械相关专业，三年以上结构设计经验；SolidWorks 或 Creo，有限元分析；注塑、钣金、机加工艺；减速器设计和供应商对接；解决量产问题。
- **嵌入式（电机控制）**：电气、自动化、电子专业；STM32 等单片机，FreeRTOS；永磁同步电机原理和 FOC；CAN、EtherCAT 通信；会看原理图、会用示波器。

- **嵌入式硬件**：主控板、传感器、电机驱动电路设计，元器件选型，硬件调试，EMC 设计。

## 知识地图

表 9-1 硬件岗的知识地图

| 必会 | 加分 | 可以先放一放 |
| --- | --- | --- |
| 电机原理：扭矩常数、反电动势、铜损 | 无位置传感器 FOC | 电磁场仿真的细节 |
| FOC 的流程、电流环调参 | 关节力矩传感器 | |
| 编码器的类型和安装 | 功能安全（ISO 13849、IEC 61508 一类） | |
| 减速器的类型和取舍、反射惯量 | 电池和 BMS | |
| 热设计和降额 | 高功率密度驱动器设计 | |
| CAN、CAN FD、EtherCAT | 时间敏感网络 | |
| RTOS、中断、DMA | | |
| 结构：材料、轻量化、跌落、疲劳、可制造性 | 拓扑优化 | |

## 面试题和参考答案

### 9.1 永磁同步电机的扭矩和电流是什么关系？发热主要来自哪里？

**一句话**：扭矩和交轴电流成正比，$\tau = K_t\, i_q$；低速大扭矩时，发热主要来自绕组的铜损 $P = I^2R$。

腿足机器人有个特点：站着不动也要出力。机身的重量一直压在膝关节上，电机转速几乎为零，却要持续输出大电流。这时机械功率接近零，电功率几乎全部变成热。所以腿足机器人关节的瓶颈经常不是峰值扭矩，而是**能持续多久**。

高速时还有铁损（涡流和磁滞）。电机的连续扭矩由散热能力决定，峰值扭矩由电流上限和磁路饱和决定。

**考官想听到**：$\tau = K_t i_q$；铜损和电流平方成正比；站立时功率全变成热；连续扭矩和峰值扭矩的区别。

## 9.2 FOC 的基本流程是什么？为什么要做 Clarke 和 Park 变换？

**一句话**：把三相交流电流变到跟着转子转的坐标系里，变成两个直流量，再用两个 PI 控制器分别控制，最后变回三相电压。

一个控制周期里：

1. 采样相电流 $i_a, i_b, i_c$；
2. **Clarke 变换**：三相变两相静止坐标 $i_\alpha, i_\beta$；
3. **Park 变换**：用转子电角度 $\theta_e$ 旋转到转子坐标 $i_d, i_q$；
4. 两个 PI 控制器：$i_d$ 跟踪 0（表贴式电机），$i_q$ 跟踪扭矩需要的电流；
5. 逆 Park 变换得到 $u_\alpha, u_\beta$，再用空间矢量调制（SVPWM）生成三相 PWM。

为什么要变换：在静止坐标下电流是正弦交流，PI 控制器跟踪正弦量会有相位滞后；在转子坐标下，稳态时 $i_d$、$i_q$ 是常数，PI 能无静差地跟踪。代价是必须准确知道转子角度，编码器的零位要标定，零位偏了，扭矩会下降，还会产生多余的 $i_d$。

**考官想听到**：五个步骤；dq 坐标下变成直流量的意义；转子角度和编码器零位的重要性。

## 9.3 电流环、速度环、位置环的带宽怎么安排？

**一句话**：内环快、外环慢，每一层的带宽一般比内层低几倍到十倍；调参从内往外调。

电流环在驱动板上以十几到几十千赫兹采样，带宽通常在千赫兹量级；速度环、位置环依次更慢。外环的带宽如果接近内环，内环的动态就不能忽略，两层会互相干扰。

腿足机器人的关节常常不用传统的“位置环套速度环套电流环”，而是由上层直接给“目标角度、目标速度、kp、kd、前馈力矩”，驱动板算出扭矩，再折算成 $i_q$ 的指令。这相当于把位置和速度环合成了一个 PD，直接坐在电流环上面，好处是关节的刚度可以由上层随时调整。

**考官想听到**：内快外慢、从内往外调；带宽的数量级；腿足机器人里 PD 直接驱动电流环的做法。

### 9.4 腿足机器人的关节为什么偏爱低减速比？减速比怎么选？

**一句话**：低减速比让关节“透明”：外力能反推电机（可反驱），撞击时不容易坏，还能直接用电流估计关节力矩。代价是同样的电机，输出扭矩小一些。

关键是**反射惯量**：电机转子惯量折算到关节输出端要乘以减速比的平方，$J_{反射} = N^2 J_m$。减速比 6 时放大 36 倍，减速比 50 时放大 2,500 倍。高减速比的关节在脚落地的瞬间，相当于一块很重的飞轮硬生生被停住，冲击全落在齿轮上。

所以四足机器人常用 6 到 10 左右的行星减速器（宇树 GO-M8010-6 的减速比是 6.33），用大扭矩密度的电机来补扭矩；机械臂、人形机器人的手臂更看重精度和扭矩，多用谐波减速器这类高减速比方案。选减速比时把这几样放在一起看：峰值和持续扭矩、最高转速、反射惯量和抗冲击、背隙和效率、成本和寿命。

**考官想听到**：反射惯量 $N^2$；可反驱和本体感知；腿和手臂的不同选择。

### 9.5 编码器有哪些类型？为什么关节常用两个编码器？

**一句话**：按原理分光电和磁编码，按输出分增量式和绝对式；关节常在电机端和输出端各放一个，一个管电机控制，一个管关节真实位置。

- **电机端编码器**：分辨率高，用来给 FOC 提供转子电角度、算转速；
- **输出端编码器**：直接测减速器后面的关节角，绝对式的，上电就知道关节在哪，不用回零；还能看出减速器的背隙和弹性变形。

两个编码器的读数对不上，往往是减速器出了问题，这也是一种在线诊断手段。磁编码比光电编码便宜、抗污染，所以消费级的机器人关节大多用磁编码。

**考官想听到**：两个编码器的分工；绝对式省掉回零；两个读数的差值可以用来诊断。

### 9.6 CAN、CAN FD、EtherCAT 怎么选？一条 CAN 总线能挂几个关节、跑多高的频率？

**一句话**：按带宽和同步需求选；可以先算一笔账。

经典 CAN 的速率是 1 Mbps。一帧带 8 个字节数据的标准帧，加上帧头、校验和位填充，大约 110–130 位，所以一条总线每秒最多传七千到九千帧。

一台四足有 12 个关节，如果每个关节每毫秒要一帧指令、一帧反馈，就是每秒 24,000 帧，是一条 CAN 总线容量的三倍左右。常见的办法：

1. **每条腿一条总线**：每条 3 个关节，每秒 6,000 帧，负载率七成左右，勉强可用；
2. **CAN FD**：数据段速率提高到几 Mbps，一帧能带 64 字节；

3. **EtherCAT**：100 Mbps，一帧数据在总线上走一圈就把所有从站读写完，还自带分布式时钟，适合关节多、同步要求高的人形机器人；
4. 也有厂商用高速 RS-485：宇树 GO-M8010-6 用的就是 4 Mbps 的 RS-485。

**考官想听到**：能当场算出帧率和负载；几种总线的带宽量级；分布式时钟和同步的意义。

## 9.7 RTOS 和 Linux 在机器人里分别干什么？中断服务程序里不能做什么？

**一句话**：单片机加 RTOS 负责硬实时的事（电流环、保护、通信），Linux 负责计算量大但实时性要求稍低的事（感知、规划、强化学习策略推理）。

关节驱动板的电流环必须在几十微秒内完成，一次都不能晚，交给单片机；过流、过温保护也在这一层，不能依赖上层系统。Linux 打上实时补丁可以做到亚毫秒级的调度延迟，适合跑 500 Hz 到 1 kHz 的全身控制，但最坏情况的保证不如 RTOS。

中断服务程序要短：只做最必要的事（读寄存器、清标志、把数据放进队列），然后通知任务去处理。**不能**在中断里等待信号量或者调用会阻塞的函数，**不能**分配内存，**不要**打印日志。

**考官想听到**：两类系统的分工；中断的禁忌；保护逻辑放在最底层。

## 9.8 机器人结构怎么做轻量化？跌落和冲击怎么考虑？

**一句话**：把重的东西往身体中心放，让结构件同时承担多种功能，再用仿真和实测验证抗冲击。

- **质量分布**：电机尽量靠近机身。很多四足机器人把膝关节的电机也放在髋部附近，用连杆或同步带把动力传到膝盖，这样腿的转动惯量小，摆得快，落地冲击也小（《拆解宇树》第 4 章）；
- **一件多用**：电机外壳同时是结构件、散热片和腿的连接件；
- **材料**：铝合金为主，关键部位用镁合金或碳纤维，外壳用工程塑料；
- **冲击**：做跌落仿真和跌落测试，在容易磕碰的地方加缓冲；反复冲击下还要考虑疲劳寿命和螺丝松动（螺纹胶、防松结构）；
- **线缆**：关节反复转动，线缆的弯折寿命经常比结构件先到。

**考官想听到**：质量分布对腿的意义；一件多用；冲击、疲劳、线缆这几个容易忽略的点。

### 9.9 一个关节连续爬楼梯 20 分钟后过热保护，你怎么分析和解决？

**一句话**：先分清是“需要的扭矩太大”还是“散热太差”，再从需求、散热、软件三方面下手。

分析：

1. 记录这个关节的电流、温度、关节角随时间的曲线，算出平均的 $I^2$，和电机的持续电流能力比较；
2. 看是不是某一侧、某一个关节特别严重，可能是机身重心偏、步态不对称；
3. 估算热时间常数：温度上升的快慢，判断是绕组到外壳的热阻大，还是外壳到空气的散热差。

解决：

- **降低需求**：调整步态和机身姿态，减少这个关节的平均扭矩；把重心调回中间；
- **改善散热**：加散热片、导热垫，改风道；
- **软件降额**：用一个热模型实时估算绕组温度，接近上限时平滑地降低电流上限，而不是突然保护停机；
- **换硬件**：扭矩常数更大的电机，或者调整减速比。

**考官想听到**：先测量再动手；$I^2$ 的平均值；热模型加平滑降额。

### 9.10 样机好好的，量产以后不良率很高，你怎么查？

**一句话**：先确认不良出在哪个环节，再用数据找出和不良相关的变量，最后从设计和工艺两头改。

1. **分类**：不良是来料问题、装配问题，还是设计余量不够？把不良品按现象分类统计；
2. **追溯**：每个关节有序列号，查不良品用的是哪批零件、哪条产线、哪个时间段；
3. **找变量**：公差叠加是不是在极端情况下超限？某个供应商的零件尺寸是否偏到了公差边缘？
4. **改进**：设计上加大余量、改成更容易装配的结构（可制造性设计）；工艺上加检测工装、来料检验、老化测试；
5. **验证**：小批量试产，用统计过程控制看问题是否真的解决了。

**考官想听到**：分类和追溯；公差叠加；设计和工艺两头改；有没有用过 8D、FMEA 这类方法。

**考官视角｜硬件岗最看重“见过”**

硬件面试的追问常常是：“你做过的产品年产量多少？”“出过最严重的质量问题是什么?”“你怎么定位的?”这些问题没有标准答案，比的是经历。从汽车、手机、家电行业转过来的人，在这里有天然的优势：把你经历过的量产问题按“现象、定位、原因、改进、验证”讲清楚。

## 从哪些背景转过来最顺

- **新能源汽车三电和电控**：最顺。电机、FOC、功率电子、热设计、功能安全都是同一套东西，只是功率小一些、关节多一些。
- **伺服驱动器、变频器、工控**：电机控制和总线经验直接可用。
- **汽车电子和消费电子的嵌入式**：RTOS、总线、引导程序和固件升级都是现成的。
- **手机、家电、汽车的结构工程师**：量产和可靠性的经验最值钱，要补的是关节和腿的力学特点。

**转行者笔记｜你的“老本行”就是卖点**

硬件岗转行的人最容易犯的错，是简历上拼命写自己刚学的强化学习，反而把十年的电机经验写成一行。机器人公司招硬件，要的就是你的老本行。新学的东西放在“补充技能”里就够了。

**动手试试｜读一份关节电机参数表**

宇树 GO-M8010-6 的公开参数：约 530 g；减速比 1:6.33；峰值扭矩 23.7 N·m；输出端扭矩常数 0.63895 N·m/A；最大电流 40 A；15 位绝对值编码器；RS-485 通信，速率 4 Mbps。试着算：

1. 输出 15 N·m 需要多大电流？（约 23.5 A）
2. 按最大电流算，扭矩上限是多少？和标称的峰值扭矩差在哪？（约 25.6 N·m，比标称值大一点，标称值留了余量）
3. 电机转子惯量折算到输出端要乘多少？（$6.33^2 \approx 40$ 倍）

**本章一句话**

**传统行业的工程师在硬件这一层最有优势：机器人公司最头疼的量产、可靠性和发热，正是你做了多年的事。**

下一章讲剩下的几类岗位：软件、数据、测试和交付。不写算法，也能进具身公司。

第 10 章

# 软件、数据、测试和交付岗

再看一次宇树官网的技术岗：软件 4 个，数据与 AI 基础设施 4 个，测试、安全、项目管理 3 个，一共 11 个，正好占一半。这还没算销售服务体系里的解决方案和技术支持岗位。

这些岗位不要求你会推动力学方程，也不要求你训练过强化学习策略。它们要的是：能把系统做得可靠，能把数据管好，能把问题找出来，能把机器人交到客户手里、让它在客户那里一直正常工作。

这一章把这几类岗位放在一起讲，面试题按岗位标在题目后面。动手环节是给一台机器狗写一页测试计划。

## 这几个岗位每天在干什么

- **系统软件**：维护机器人上的进程管理、通信中间件、日志和回放、对外 SDK；查“偶发的 30 毫秒卡顿”是从哪来的。
- **数据采集和运营**：排班组织操作员采数据，盯数据质量，维护采集设备。
- **数据管线和平台**：把机器人每天上传的几百小时数据变成可检索、可训练的数据集。
- **AI Infra**：维护训练集群，加速训练，把模型压缩以后部署到机器人上。
- **测试**：写测试计划，跑跌落、续航、环境、回归测试，追问题直到关闭。
- **技术支持和解决方案**：给客户做方案和演示，去现场部署，接到故障电话第一个响应。
- **产品和项目管理**：定义下一代产品，管研发进度、认证和量产节奏。

## 面试题和参考答案

### 10.1 （系统软件）机器人的日志和回放系统怎么设计？

**一句话**：在数据产生的地方打时间戳，把“输入”（传感器数据、收到的指令）完整录下来，这样可以离线确定性地回放，复现问题。

设计要点：

1. **录什么**：所有传感器原始数据、上层指令、关键的中间结果和状态切换；
2. **时间戳**：用采集时刻而不是写盘时刻，全系统统一时钟；
3. **格式**：ROS 2 生态里常用 rosbag2，底层格式多用 MCAP，自带索引，方便按时间和话题检索；
4. **存不下怎么办**：平时录在内存里的环形缓冲区，出事件（摔倒、急停、报错）时把前后几十秒落盘，像飞机的黑匣子；
5. **回放**：把录下的输入重新喂给算法，在开发机上复现现场问题。

**考官想听到**：录输入以便确定性回放；时间戳的来源；环形缓冲区加事件触发。

### 10.2 （系统软件）控制进程崩溃了，机器人会怎样？看门狗和安全状态怎么设计？

**一句话**：每一层都要有“收不到上层指令就进入安全状态”的机制，安全状态通常是阻尼模式，而不是直接断电。

- **关节驱动板**：超过几十毫秒没收到新指令，自动切到阻尼模式（只保留 kd，没有 kp），腿慢慢软下来，机器人缓缓趴下；
- **直接断电**的后果是机器人瞬间瘫倒，可能摔坏自己或砸到人；
- **监控进程**：检测控制进程的心跳，挂了就重启，或者通知上层进入降级模式；
- **硬件急停**：独立于所有软件，一按就切断电机输出（断电时腿会塌下来，所以急停之前最好先阻尼）。

**考官想听到**：分层的看门狗；阻尼模式比断电安全；硬件急停独立于软件。

### 10.3 （数据管线）每天几百小时的多模态机器人数据，怎么存、怎么检索、怎么喂给训练？

**一句话**：原始数据进对象存储，元数据进数据库，按任务、场景、质量建索引，训练时按配比从索引里抽样，转换成训练框架需要的格式。

- **入库**：机器人回充时上传，上传前在本地做完整性检查；

- **存储**：图像、视频、点云这类大文件放对象存储；每一段数据的元信息（哪台机器人、什么任务、哪个操作员、成功与否、质量分）放数据库；
- **检索**：能按“任务 = 叠毛巾、成功、相机正常、最近一个月”这样的条件查出数据；
- **格式转换**：转成训练框架的格式，比如 LeRobot 的数据集格式，或者 Open X-Embodiment 使用的 RLDS；
- **版本管理**：每次训练用的数据集要能精确复现；
- **可视化**：做一个能同时看视频、关节曲线、语言指令的浏览工具，数据和算法团队天天要用。

**考官想听到**：原始数据和元数据分开；按条件检索；版本可复现；可视化工具的重要性。

### 10.4 （数据采集）采集现场有哪些常见问题？怎么保证数据一致？

**一句话**：问题大多出在“人”和“流程”上，靠标准作业流程、开班检查和抽检来管。

常见问题：操作员的做法不一致；场景没有复位，物体的初始位置越来越集中；相机被碰歪了，外参变了却没人知道；操作员疲劳以后动作变慢、犹豫变多。

做法：

1. **标准作业流程**：每个任务写清楚怎么摆场景、怎么操作、怎么算成功；
2. **开班检查**：每天开始前做一次标定检查，比如拍一张固定的标定板，对比外参；
3. **实时监控**：采集时就能看到数据是否正常，而不是一周后才发现；
4. **抽检和反馈**：每天抽检一部分，问题当天反馈给操作员；
5. **轮换**：控制每个操作员连续采集的时间。

**考官想听到**：人和流程的问题；开班检查；当天抽检、当天反馈。

### 10.5 （AI Infra）数据并行和模型并行有什么区别？训练一个几十亿参数的 VLA 要注意什么？

**一句话**：数据并行是每张卡放一份完整的模型、各算一部分数据，再把梯度汇总；模型放不下一张卡时，才需要把模型本身切开（张量并行、流水线并行），或者把参数和优化器状态分片（ZeRO、FSDP）。

几十亿参数的 VLA，用 bf16 混合精度时，参数加优化器状态加梯度，一张卡往往放不下，常用 FSDP 把它们分片到多张卡上。另外几个要注意的地方：

- **数据加载**: 机器人数据有大量图像和视频, 解码和预处理很容易成为瓶颈, GPU 在等数据;
- **激活重计算**：用计算换显存;
- **检查点**：定期保存，训练中断能接着跑;
- **评测**：训练中途要有离线指标，不能每次都等真机。

**考官想听到**：两类并行的区别；FSDP 或 ZeRO 分片；数据加载瓶颈。

### 10.6 （测试）给一台四足机器人写测试计划，你会包括哪些测试?

**一句话**：功能、性能、续航、环境、机械、安全、回归七类，每一项都要有明确的通过标准。

表 10-1 四足机器人测试计划的七类测试

| 类别 | 例子 | 通过标准的写法 |
|---|---|---|
| 功能 | 各种步态、遥控指令、APP 功能 | 每项功能按用例逐条通过 |
| 性能 | 最高速度、爬坡角度、越障高度 | 达到规格书的数值 |
| 续航 | 满电连续行走、待机 | 时长不低于标称值 |
| 环境 | 高低温、淋水防尘、电磁兼容 | 按对应标准的等级 |
| 机械 | 跌落、侧翻、关节寿命 | 规定次数后功能正常 |
| 安全 | 急停、摔倒检测、故障注入 | 进入预期的安全状态 |
| 回归 | 每次软件发布前的自动化测试 | 全部用例通过 |

**考官想听到**：分类完整；每一项都有可量化的通过标准；知道哪些能自动化。

### 10.7 （测试）什么是故障注入？为什么重要?

**一句话**：故意制造故障（传感器断线、消息延迟、关节过温、电量低、网络断开），检查系统有没有按设计降级到安全状态。

机器人在现场遇到的问题，很多在实验室里从来没出现过。故障注入就是把“万一”提前演练:

- 在仿真和硬件在环测试里自动化地注入;
- 每种故障都要有预期行为，比如“IMU 断线 → 200 毫秒内进入阻尼模式并上报”;

- 真机上做破坏性较小的注入，比如断开一路网络、拔掉一个传感器。

**考官想听到**：故障的例子；每种故障有预期行为；仿真、硬件在环、真机三个层次。

### 10.8 （技术支持）客户说“机器人在我们仓库里老是定位丢失”，你怎么处理？

**一句话**：先收集信息、复现问题，给客户一个临时办法，再把根本原因反馈给研发。

1. **问清楚**：什么时候开始的？哪些区域？软件版本？仓库最近有没有变化（货架挪了、装了玻璃隔断、人多了）？
2. **拿数据**：导出日志和地图，看丢失前后的传感器数据；
3. **复现和定位**：常见原因是环境变化太大，旧地图对不上；反光和透明表面；长走廊这类特征少的地方；
4. **临时办法**：更新地图、在特征少的地方加标记物；
5. **长期改进**：把问题和数据整理给研发；
6. **沟通**：每一步都告诉客户在做什么、预计什么时候有结果，处理完写进知识库。

**考官想听到**：有条理的排查顺序；临时办法和长期改进分开；和客户的沟通。

### 10.9 （产品经理）如果让你定义一款面向高校科研的四足机器人，你会怎么定配置？

**一句话**：先想清楚科研用户最在意什么：开放的底层接口、仿真支持、可扩展性和耐用性，再按预算分出几个版本。

- **用户**：做运动控制、强化学习、感知的研究生和老师；
- **最在意的**：底层关节控制接口开放（能直接发力矩和 PD 指令）；有官方的仿真模型和 Sim2Real 示例；能加装激光雷达、相机、计算板；摔了不容易坏、坏了好修；文档和社区；
- **版本**：基础版满足教学，科研版开放底层接口、加算力；
- **验证**：先访谈二三十位目标用户，看他们现在用什么、最不满意什么。

《拆解宇树》第 2 章讲过，宇树的 Go2、G1 都有面向开发者的 EDU 版，开放二次开发，其中 Go2 的 EDU 版要单独询价，就是这种分版本的思路。

**考官想听到**：从用户需求出发；分版本；先验证需求再定配置。

### 10.10 （安全）机器人产品常见的安全风险有哪些？怎么防？

**一句话**：远程访问后门、硬编码密钥、默认密码、未签名的固件升级是最常见的几类；对应的防护是每台设备独立的密钥、签名升级、最小权限和漏洞披露流程。

公开的例子（《拆解宇树》第 17 章）：2025 年春天披露的 CVE-2025-2894，Go1 预装了一个远程访问隧道服务，持有密钥的人可以远程控制机器人、看摄像头；2025 年 9 月公开的 UniPwn，Go2、G1 等机型的蓝牙配网接口用硬编码的密钥加密，认证可以被伪造，注入的数据以最高权限执行。

防护：

1. **安全启动和签名升级**：只运行签过名的固件；
2. **每台设备独立的密钥**，不在所有设备上用同一个；
3. **最小权限**：对外服务不用最高权限运行；
4. **默认关闭**远程访问，需要时由用户显式打开；
5. **漏洞披露流程**：有渠道接收安全研究者的报告，并及时修复。

**考官想听到**：能举出真实案例；每类风险对应的防护；披露流程。

> **考官视角｜这些岗位比的是“靠谱”**
>
> 软件、数据、测试、交付岗位的面试，追问的往往是：“你上一份工作里，最难复现的一个问题是怎么查的?”“你怎么保证数据（或者测试结果）是可信的?”“客户发火的时候你怎么处理?”回答时，把你过去工作里最靠谱的一次经历讲具体，比背技术名词有用。

## 从哪些背景转过来最顺

- **互联网后端、数据开发、AI Infra**：直接对应系统软件、数据管线、AI Infra 岗位，要补的是机器人的实时性和数据的特点（时间同步、多模态）。
- **汽车、消费电子、工控的测试工程师**：测试方法是通用的，补机器人特有的测试项目。
- **工业自动化、AGV 的售前和现场工程师**：做解决方案和技术支持最顺，懂现场、懂客户。
- **硬件、汽车行业的产品和项目经理**：熟悉研发和量产节奏，补机器人的技术常识（这本书和《拆解宇树》就是为这个准备的）。
- **网络安全工程师**：产品安全是一个正在长大的岗位。

**转行者笔记｜别看不起“非算法岗”**

在具身公司里，测试、数据、交付岗位的人每天离机器人最近，最清楚它在哪里会出问题。很多公司的算法团队也在找“懂现场的人”。从这些岗位进门，再往算法或产品方向走，是一条很多人走过的路。

**动手试试｜给一台机器狗写一页测试计划**

照着表 10-1 的七类，给一台四足机器人写一页测试计划，每类至少写两项，每项写清楚“怎么测”和“什么算通过”。写完它就是你面试测试岗、产品岗时能拿出来的一份作品。

**本章一句话**

**不写算法也能进具身公司：一家机器人公司一半的技术岗位，在做“让机器人可靠、让数据可信、让客户满意”的事。**

第二部到这里结束。你已经知道目标岗位要什么、面试问什么。第三部讲怎么把这些变成 offer：先从作品集开始。

第三部

# 拿到 offer

第 11 章

# 作品集（上）：控制和学习

招聘启事里写“有真机调试经验者优先”，你没有真机；写“有足式机器人项目经验者优先”，你没做过。作品集就是用来补这一块的：用一个能演示、讲得清楚的项目，证明你能独立解决机器人问题。

这一章和下一章给出 6 个项目，全部只需要一台笔记本电脑，不用显卡。每个项目都在作者的机器上实际跑过，书里的每张图、每个数都来自这些运行结果。

先说一个原则：**项目的价值在对照实验，不在“跑通”**。跑通一个开源仓库的示例，面试官会觉得“谁都能做”；做一组对照（改一个变量、看结果怎么变、解释为什么），面试官会觉得“这个人会做研究、会查问题”。下面每个项目都按这个思路设计：一个问题，一组对照，一张图，一段“面试时这样讲”。

## 准备环境

所有代码在随书代码的 `代码/` 目录里，每个项目一个子目录。运行环境：

- Python 3.13，Linux（macOS 和 Windows 的 WSL 也可以）；
- 依赖见 `代码/requirements.txt`：NumPy、SciPy、Matplotlib、MuJoCo 3.17、Gymnasium 1.4、PyTorch 2.15（CPU 版）；
- 项目一要用到 DeepMind 维护的 MuJoCo Menagerie 里的 Go2 模型。

```
python3 -m venv venv && source venv/bin/activate
pip install torch==2.15.0 --index-url https://download.pytorch.org/whl/cpu   # 先装 CPU 版
pip install -r 代码/requirements.txt
git clone https://github.com/google-deepmind/mujoco_menagerie.git
```

书里的结果是在一台 4 核、无显卡的 Linux 机器上跑出来的。项目一、二、四、五都在一分钟以内跑完；项目三最久，一组完整对照约半小时；项目六十几分钟。

## 项目一：让 Go2 站起来，比较刚度和阻尼

**问题**：关节 PD 控制的 kp 和 kd 怎么影响机器狗站立？控制有延迟时会怎样？

**做法**：用 MuJoCo Menagerie 里的 Unitree Go2 模型，它的 12 个关节都是力矩电机，力矩上限和真机一样（髋关节 23.7 N·m，膝关节 45.43 N·m）。仿真和控制都是 1 kHz。前 0.5 秒保持趴姿，0.5–1.5 秒平滑地过渡到站姿，站稳以后在 3–4 秒给机身压一个向下 60 牛的力，模拟背上了负载。控制延迟用一个队列实现：用几毫秒之前测到的关节角和角速度算力矩。

核心的控制循环只有十几行：

```
for k in range(int(t_total / DT)):
    t = k * DT
    q_des = LIE + smoothstep((t - 0.5) / 1.0) * (STAND - LIE)
    d.xfrc_applied[base, 2] = -load_n if 3.0 <= t < 4.0 else 0.0
    history.append((d.qpos[7:].copy(), d.qvel[6:].copy()))
    q, dq = history.pop(0)                  # 取 delay_ms 毫秒之前的测量
    tau = kp * (q_des - q) - kd * dq
    d.ctrl[:] = np.clip(tau, lo, hi)        # 力矩不能超过电机上限
    mujoco.mj_step(m, d)
```

**结果**：

表 11-1　项目一的结果（目标站姿的机身高度约 0.27 m）

| kp / kd | 延迟 | 机身高度 | 膝关节误差 | 加载后下沉 | 最大力矩 |
|---|---|---|---|---|---|
| 10 / 0.5 | 0 | 0.151 m | 35.0° | 18.0 mm | 7.4 N·m |
| 40 / 1 | 0 | 0.241 m | 8.2° | 17.6 mm | 8.9 N·m |
| 100 / 2 | 0 | 0.264 m | 3.3° | 5.9 mm | 14.7 N·m |
| 300 / 3 | 0 | 0.272 m | 1.1° | 2.3 mm | 27.8 N·m |
| 300 / 3 | 5 ms | 0.272 m | 1.1° | 2.3 mm | 33.1 N·m |
| 300 / 8 | 5 ms | 0.269 m | 2.9° | 4.2 mm | 45.4 N·m（打满） |

kp=10 时机器狗根本站不起来；kp 越大，站得越准，负载下沉越小，代价是力矩需求变大。最有意思的是最后两行：有 5 ms 延迟时，kd=3 还是稳的，kd=8 却开始以约 45 Hz 的频率抖动，膝关节力矩在 ±45.43 N·m 之间来回打满，5 秒里有 4.8 秒处于饱和。

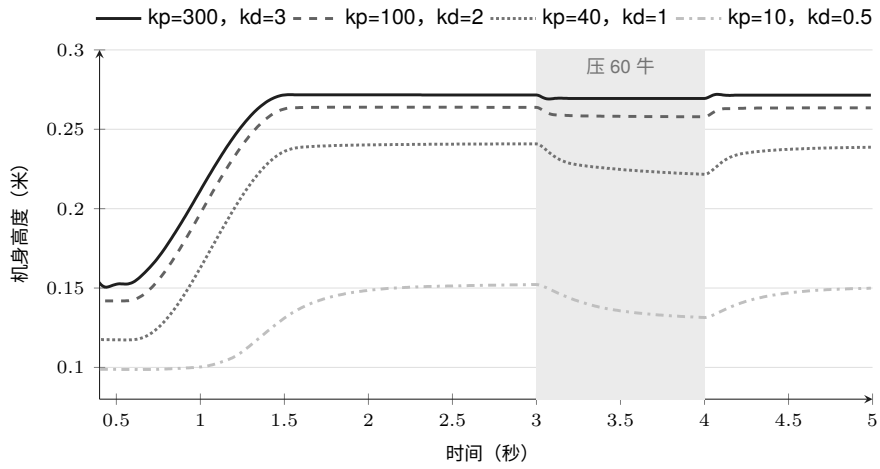



图 11-1　项目一：刚度越高，站得越高、负载下沉越小（3–4 秒压上 60 牛的力）

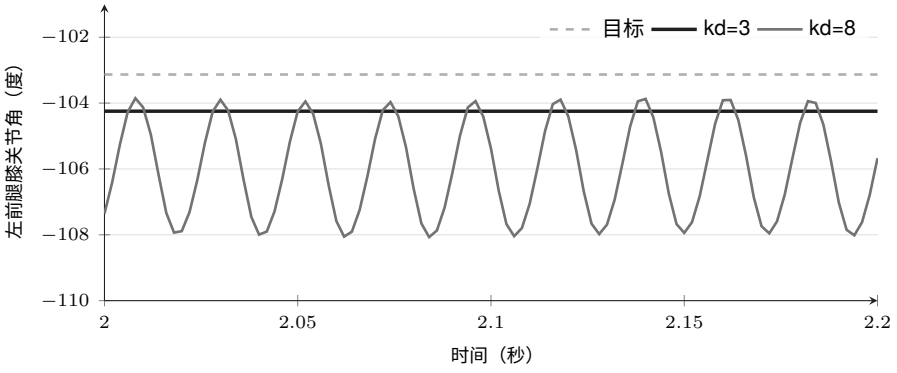



图 11-2　项目一：同样 kp=300、控制延迟 5 ms，kd=8 时膝关节以约 45 Hz 抖动，力矩在正负上限之间来回打满

**面试时这样讲**（一分钟版）：

我在 MuJoCo 里用 Go2 的模型做了一组 PD 增益的对照。kp 从 10 到 300，站立高度从 0.15 米到 0.27 米，加 60 牛负载时的下沉从 18 毫米降到 2 毫米。然后我加了 5 毫秒的控制延迟，发现 kd 从 3 加到 8，机器人反而开始 45 赫兹左右的抖动，力矩一直打满。原因是延迟带来的相位滞后，加上微分项对高频的放大。所以真机上腿抖的时候，第一反应不应该是加 kd，而是先查延迟和速度噪声。

**可以继续做**：加上重力补偿的前馈力矩，看能不能用更低的 kp 站得一样准；扫一组延迟（0、2、5、10 ms），画出“不抖的最大 kd”随延迟的变化；写一个对角小跑的步态。

## 项目二：倒立摆，PID 和 LQR

**问题**：为什么要“全状态反馈”？LQR 对模型误差和延迟有多宽容？

**做法**：经典的小车倒立摆（小车 1 kg，杆 0.1 kg，半杆长 0.5 m），用 Barto 等人 1983 年给出的非线性方程和四阶龙格库塔积分。仿真 1 kHz，控制 200 Hz，推力上限 10 牛。杆从偏离 0.1 弧度的位置起步，2 秒时侧推小车一把。

LQR 的增益在竖直位置线性化以后，解连续时间 Riccati 方程得到，SciPy 一行就能解：

```
Pm = solve_continuous_are(A, B, Q, np.array([[R]]))
K = (B.T @ Pm / R).ravel()          # u = -K x
```

用 $Q = \mathrm{diag}(1, 1, 10, 1)$、$R = 0.1$，得到 $K = [-3.16, -5.91, -49.54, -12.85]$。

**结果**：

表 11-2　项目二的结果

| 控制器 | 条件 | 是否倒下 | 最大摆角 | 8 秒末小车位置 |
|---|---|---|---|---|
| PID（只管摆角） | 标准 | 否 | 5.9° | −0.46 m |
| LQR | 标准 | 否 | 5.7° | 0.00 m |
| LQR | 实际的杆比模型重 5 倍 | 否 | 5.7° | 0.00 m |
| LQR | 控制延迟 80 ms | 否 | 8.2° | 0.00 m |
| LQR | 控制延迟 100 ms | 是 | — | — |

**面试时这样讲**：

我比较了只管摆角的 PID 和全状态的 LQR。PID 能让杆立住，但小车回不到原点，8 秒漂出去将近半米，因为它根本不知道小车在哪。LQR

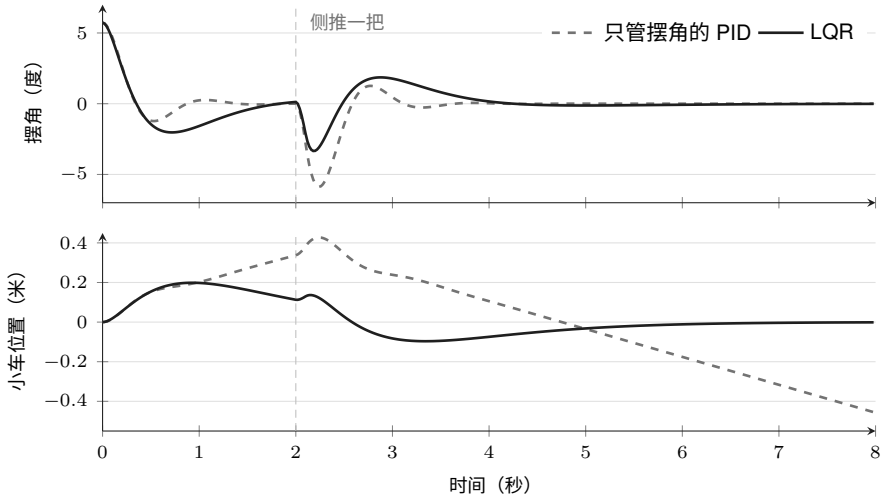



图 11-3　项目二：只管摆角的 PID 让杆立住了，小车却回不来；LQR 两样都管住了

同时用了四个状态，两样都管住了。然后我测了它的鲁棒性：杆实际比模型重 5 倍，它照样稳；但延迟到 100 毫秒就倒了。我的结论是，对这类系统，延迟比参数误差更危险。

**可以继续做**：只测摆角和小车位置、用卡尔曼滤波估计速度，再接 LQR（这就是 LQG）；把连续时间的 LQR 换成离散时间的，看 200 Hz 下有没有区别；做一个“摆起”控制器（能量法摆起 + LQR 稳定）。

## 项目三：用 PPO 训练 Hopper，对照域随机化

**问题**：域随机化到底有没有用？代价是什么？

**做法**：Gymnasium 1.4 里的 Hopper-v5，一个单腿跳跃的机器人。算法是单文件的 PPO，写法参考 CleanRL：每次采 2,048 步，每批数据用 10 遍、每遍 32 个小批，学习率 3e-4 线性衰减，$\gamma = 0.99$，$\lambda = 0.95$，截断系数 0.2；观测归一化、回报缩放、超时截断的价值补偿都自己实现了。

域随机化在每个回合开始时随机改三样东西：躯干质量乘以 0.7–1.3，地面摩擦乘以 0.6–1.4，电机力气乘以 0.85–1.15。核心是一个十几行的包装器：

```
class Dynamics(gym.Wrapper):
    def reset(self, **kw):
        m = self.env.unwrapped.model
        if self.randomize:
```

```
        f = {k: self.rng.uniform(*r) for k, r in DR_RANGES.items()}
    else:
        f = {"torso_mass": 1.0, "friction": 1.0, "motor": 1.0, **self.fixed}
    m.body_mass[:] = self.base_mass
    m.body_mass[self.torso] = self.base_mass[self.torso] * f["torso_mass"]
    m.geom_friction[:] = self.base_friction * np.array([f["friction"], 1, 1])
    m.actuator_gear[:] = self.base_gear * f["motor"]
    return self.env.reset(**kw)
```

对照设计：不随机化和随机化各训 3 个随机种子，每个 100 万步。训完以后，每个策略在四种环境里各测 20 个回合：原始环境，躯干重 1.5 倍，地面摩擦减半，电机力气只剩八成。后三种都超出了训练时随机化的范围。

在 4 核 CPU 上，每个训练进程每秒约采 1,230 步，100 万步约 13.5 分钟；同时跑 3 个进程，6 个训练加测试大约半小时。

**结果**（3 个种子的平均分，括号里是 3 个种子中的最低和最高）：

表 11-3　项目三的结果

| 测试环境 | 不随机化 | 随机化 |
|---|---|---|
| 原始环境 | 2,792（2,307–3,508） | 2,696（1,687–3,449） |
| 躯干重 1.5 倍 | 1,031（572–1,376） | 1,680（1,416–2,144） |
| 地面摩擦减半 | 364（76–569） | 691（575–769） |
| 电机力气八成 | 427（408–459） | 1,327（966–1,568） |

两个规律：在原始环境里，加不加随机化差不多；环境一变，不加随机化的策略分数掉到原来的一成到四成，加了随机化的掉到原来的两成半到六成。还有一个不能忽略的事实：**种子之间差别很大**，随机化组有一个种子在原始环境里只有 1,687 分。只跑一个种子就下结论，很可能被运气骗了。

**面试时这样讲**：

> 我用 PPO 训练 Hopper，对照了加不加域随机化，各跑了 3 个种子。在原始环境里两者分数差不多，但在训练没见过的环境里，比如电机力气只剩八成，不加随机化的平均 427 分，加了的有 1,327 分。我也注意到种子之间的方差很大，所以结论是基于 3 个种子的平均，不是挑出来的最好结果。实现上有一个坑：回合因为超时结束时，不能把下一个状态的价值当成零，否则价值估计会偏。

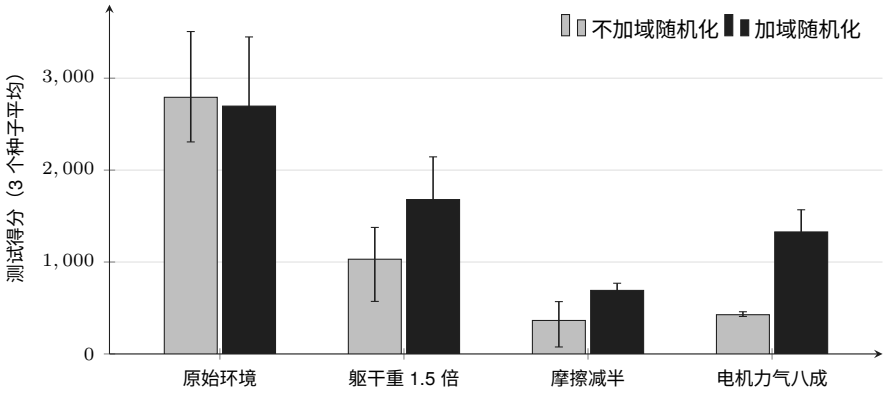



图 11-4　项目三：在原始环境里两者差不多，换了环境，加了域随机化的策略掉分少得多

**可以继续做**：把随机化范围加宽，看原始环境的分数会不会开始下降（太宽会变保守）；加观测噪声和动作延迟的随机化；换成 Walker2d 这类更难的环境；试一下教师-学生训练。

> **考官视角｜三个让项目加分的细节**
>
> 一是**对照**：至少有两组可以比较的结果。二是**数字**：讲结果时带上具体的数和单位。三是**局限**：主动说出这个实验哪里不够，比如“只有 3 个种子”“只在仿真里”。主动说局限的候选人，比被追问才承认的候选人可信得多。

**动手试试｜今天跑通一个**

三个项目里挑一个和你目标岗位最近的，今天跑通，改一个参数再跑一次，把两次的结果写成三句话。这三句话就是你简历上“项目经历”的初稿。

---

**本章一句话**

**项目的价值在对照实验，不在跑通。一个问题、一组对照、一张图、一段讲得清楚的话，就是一个合格的作品集项目。**

---

下一章是另外三个项目：感知、操作和模仿学习，以及怎么把项目整理成别人愿意看的样子。

第 12 章

# 作品集（下）：感知、操作和数据

这一章是另外三个项目：卡尔曼滤波（感知与定位岗）、机械臂逆运动学（操作岗）、模仿学习的多峰问题（具身大模型和数据岗）。最后一节讲怎么把项目整理成别人愿意点开、看得懂的样子。

和上一章一样，每个项目都实际跑过，书里的数字和图都来自运行结果。项目六还会讲一个我们**没做成**的实验，失败也是结果，而且是面试时很好用的材料。

## 项目四：卡尔曼滤波融合 IMU 和腿式里程计

**问题**：怎么把 IMU 和腿式里程计融合起来估计机身的位置？脚打滑时怎么办？

**做法**：为了看清原理，用一维的例子，但设置和真机一样。机器人在 30 秒里加速到约 1 m/s、巡航、再减速停下。

- IMU 200 Hz，带 0.08 m/s² 的零偏（滤波器事先不知道）和噪声；
- 腿式里程计 200 Hz 测速度，噪声 0.05 m/s；有三段时间脚在打滑（共 1.2 秒），这时测到的速度多出 0.8 m/s；
- 每 2 秒来一次绝对位置（比如激光雷达定位），噪声 5 cm。

滤波器的状态是位置、速度和 IMU 零偏三个量，用 IMU 做预测，用腿速和绝对位置做更新。打滑检测用卡方检验：

```
y = z - (H @ x)[0]                    # 新息：测量和预测之差
S = (H @ P @ H.T)[0, 0] + R           # 新息的方差
if gate is not None and H is H_v and y * y / S > gate:
    rejected += 1                     # 马氏距离太大：当成打滑，丢掉这次腿速
    continue
K = (P @ H.T / S).ravel()
x = x + K * y
P = (np.eye(3) - np.outer(K, H)) @ P
```

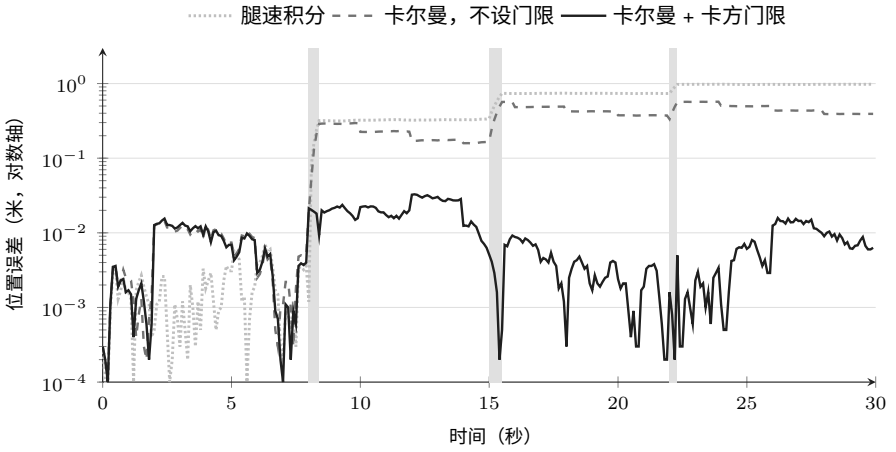



图 12-1　项目四：灰色区域是脚打滑的时段。不设门限的滤波器被打滑的腿速带偏，等下一次绝对位置来了才拉回来；加了门限以后几乎不受影响

**结果**：

表 12-1　项目四的结果

| 方法 | 位置均方根误差 | 30 秒末位置误差 |
|---|---|---|
| IMU 两次积分 | 15.9 m | 35.7 m |
| 腿速积分 | 0.63 m | 0.98 m |
| 卡尔曼滤波，不设门限 | 0.34 m | 0.39 m |
| 卡尔曼滤波，卡方门限 9 | 0.012 m | 0.007 m |

零偏估计在 30 秒后是 0.079，真值 0.08。门限一共挡掉 256 个腿速样本，其中 240 个确实是打滑时的，另外 16 个是正常测量里噪声偏大的，误伤了。

**面试时这样讲**：

> 我做了一个 IMU 加腿式里程计的卡尔曼滤波，状态里包括 IMU 的零偏。纯 IMU 积分 30 秒漂了 35 米，因为零偏被积分了两次；融合以后零偏估得很准。但脚打滑时腿速会错得很离谱，滤波器会被带偏，位置误差 0.34 米。加了卡方门限以后，打滑的测量基本都被挡在外面，误差降到 1 厘米左右。门限也有风险：如果滤波器本身已经发散，它会把好的测量也全部拒掉，所以实际系统里要监控被拒的比例。

**可以继续做**：扩展到三维，姿态用误差状态卡尔曼（第 7 章 7.5 题）；用足端力估计接触概率，代替单纯的门限；故意把初始协方差设得很小，复现“门限把好测量全挡掉”的问题。

## 项目五：机械臂逆运动学，伪逆和阻尼最小二乘

**问题**：接近奇异位形时，数值逆运动学会出什么事？阻尼最小二乘能解决多少？

**做法**：用 UR5e 公开的标准 DH 参数写正运动学和几何雅可比。先做两个检查：一是雅可比和有限差分的结果对比，最大差 1.75e-10；二是全零关节角时末端的位置 $(-0.8172, -0.2329, 0.0628)$ m，和手算一致：$x$ 是大臂和小臂长度之和（朝负方向），$y$ 是两个手腕偏移 $d_4$、$d_6$ 之和取负，$z$ 是底座高度 $d_1$ 减去 $d_5$。

然后让末端保持姿态不变，沿着远离肩膀的方向直线往外伸，伸到比手臂够得着的地方还远 1 厘米。每 10 毫秒一个点，每个点最多迭代 5 次逆运动学，分别用普通伪逆和阻尼最小二乘（$\lambda = 0.05$）：

```
def ik_step(q, T_des, damping):
    T = fk(q)
    e = np.concatenate([T_des[:3, 3] - T[:3, 3], rot_err(T_des[:3, :3], T[:3, :3])])
    J = jacobian(q)
    if damping == 0:
        dq = np.linalg.pinv(J) @ e                                  # 伪逆
    else:
        dq = J.T @ np.linalg.solve(J @ J.T + damping ** 2 * np.eye(6), e)  # 阻尼最小二乘
    return dq, np.linalg.norm(e[:3])
```

**结果**：

表 12-2　项目五的结果（π rad/s 约为 UR 系列关节的最高转速 180°/s）

| 方法 | 关节速度超过 π rad/s 的步数 | 最高关节速度 | 最大位置误差 |
|---|---|---|---|
| 伪逆 | 35 | 3,939 rad/s | 1.57 m |
| 阻尼最小二乘 | 0 | 2.46 rad/s | 23.8 mm |

**面试时这样讲**：

我用 UR5e 的 DH 参数实现了正运动学、雅可比和数值逆运动学，雅可比用有限差分验证过。然后让末端往外伸到够不着的地方，对比伪逆和

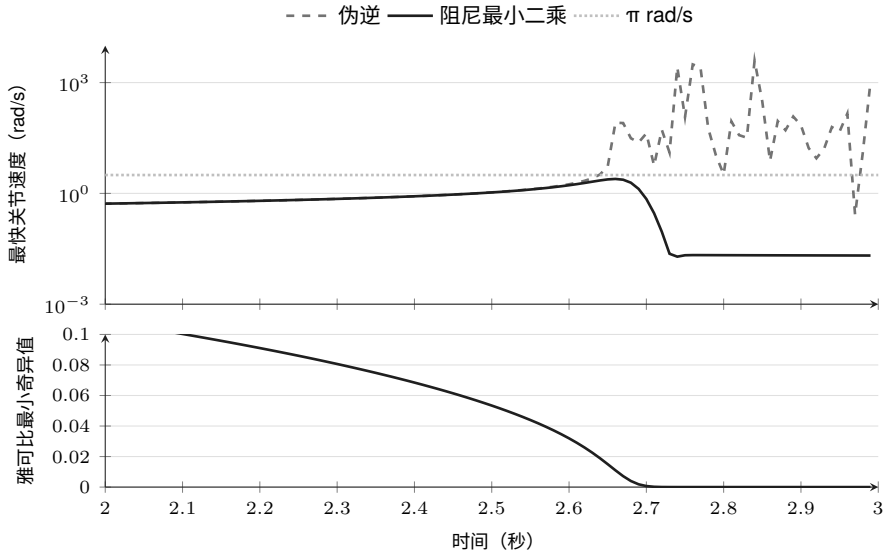



图 12-2　项目五：手臂接近伸直时，雅可比的最小奇异值趋近于零，伪逆给出的关节速度飙到几千 rad/s；阻尼最小二乘一直很平稳，代价是够不着时有几厘米的误差

> 阻尼最小二乘：接近伸直时最小奇异值趋近于零，伪逆的关节速度冲到几千 rad/s，位置也发散了；阻尼最小二乘的关节速度一直在 3 rad/s 以内，代价是目标够不着的时候，有两三厘米的误差。实际系统里还会再加关节速度和关节限位的约束。

**可以继续做**：用零空间投影加一个“远离关节限位”的次要任务；推导 UR 机械臂的解析逆解，和数值解对比；在 MuJoCo 的 UR5e 模型上验证你的正运动学（注意两边的基座坐标系可能不同）。

## 项目六：模仿学习里的“多峰”问题

**问题**：示范里有两种都对的做法时，行为克隆会怎样？怎么解决？

**做法**：一个点状机器人从左边出发，要到右边的目标，路中间有一个半径 1 米的圆形障碍。专家先直着走到障碍物前 1.5 米，再一半从上面绕、一半从下面绕，和起点在哪无关。比较三种学生：

- **A：均方误差回归**：神经网络直接输出“往哪个方向走”；
- **B：方向分类**：把方向离散成 16 格，当分类问题学，每一步选概率最大的格子；

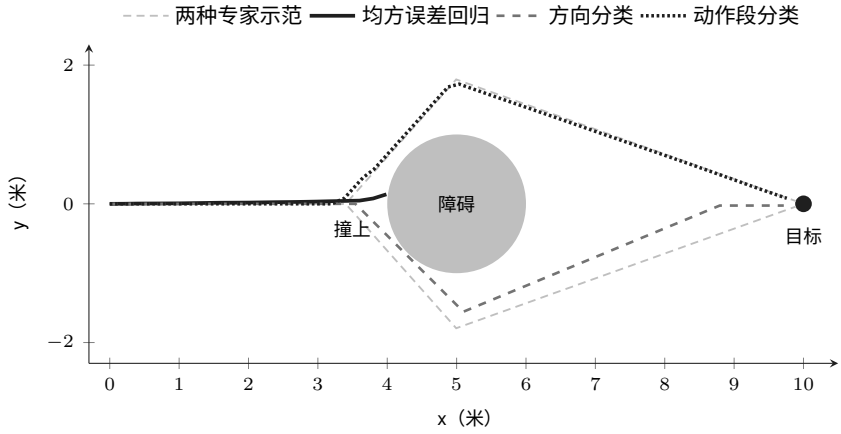



图 12-3　项目六：从正中间出发，均方误差回归把“上面绕”和“下面绕”平均成了直着走，撞上障碍物；动作段分类选定一种绕法就走到底

- **C：动作段分类**：把“接下来 8 步的方向”整段当成一个动作，先用 k 均值把示范里的动作段聚成 32 类，再学分类；选中一类，就把这一段走完再重新决定。

  C 的关键代码：

```
V = chunks_of(H, lengths)                # 每个状态往后 8 步的方向，编码成 (cos, sin)
protos, lab = kmeans(V, N_PROTO, seed)   # 聚成 32 个“动作段原型”
# …… 训练一个分类网络：状态 -> 原型编号 ……
if not plan:                             # 执行时：上一段走完了才重新选
    c = protos[out.argmax().item()]
    plan = list(np.arctan2(c[CHUNK:], c[:CHUNK]))
h = plan.pop(0)
```

**结果**（61 个起点，纵坐标在 −0.3 到 0.3 米之间均匀分布）：

表 12-3　项目六的结果

| 示范数 | A 均方误差回归 | B 方向分类 | C 动作段分类 |
|---|---|---|---|
| 40 条 | 撞了 8 次 | 0 次 | 0 次 |
| 200 条 | 撞了 8 次 | 5 次 | 0 次 |

A 的问题是平均：在分叉点上，一半示范往上、一半往下，平均下来就是直着走。B 能学出“上下各一半”的分布，我们在分叉点 (3.6, 0) 附近看了它的输出，两

个方向的概率是 0.57 和 0.43；可它每一步都重新挑，示范一多、两种走法接近五五开以后，就会在两个方向之间来回切换，200 条示范时撞了 5 次。C 选中一段就走完，前后一致，两种情况下都没有撞。这就是 Behavior Transformer、VQ-BeT 用“动作 token”、ACT 一次输出一段动作要解决的问题（第 8 章 8.2、8.4 题）。

**我们没做成的实验**：最开始想用这个项目演示 DAgger。先后试了小车倒立摆（专家是 LQR）和小车循迹（专家是 Stanley 路径跟踪控制器），结果普通的行为克隆都已经 100% 成功：示范里只要有几厘米的纠偏动作，网络就能学会纠偏；我们故意把“上一步的动作”加进输入，想引出“抄袭问题”，也没有出现。结论是：在低维、平滑的任务上，分布偏移不明显，它真正致命的是图像输入和长时序任务。这个失败本身就是一个好的面试素材。

**面试时这样讲**：

> 我做了一个关于示范多峰的小实验：一半示范从上面绕障碍、一半从下面绕。均方误差回归把两种做法平均成直着走，61 次里撞了 8 次；离散成方向分类以后，40 条示范时一次没撞，但 200 条示范时撞了 5 次，原因是它每一步都重新选，会在上下两个方向之间来回切换；把 8 步的动作段聚类成 token 再分类，两种情况都没有撞。另外我也试过用 DAgger，发现在低维任务上普通的行为克隆就已经够好，这让我理解了分布偏移在什么条件下才真正严重。

**可以继续做**：用一个小的扩散模型或混合高斯网络当策略，和 C 比较；加上 ACT 的“时间集成”；把输入换成一张俯视的图像，再试一次 DAgger。

## 把项目整理成别人愿意看的样子

面试官点开你的 GitHub，往往只看一两分钟。这一两分钟里，要让对方看到三样东西：你解决了什么问题，结果是什么，代码能不能跑。

**README 按这个顺序写**（附录 D 有模板）：

1. **一句话**：这个项目回答什么问题；
2. **一张结果图**：放在最上面，不要藏到最后；
3. **怎么跑**：三行命令以内，写清楚 Python 和库的版本；
4. **做了什么对照**：变量是什么，固定了什么；
5. **结论**：带数字；
6. **局限和下一步**：主动说出不足。

**演示视频**：30 到 60 秒，开头三秒就放结果（站起来的机器狗、绕开障碍的轨迹），再讲对照。MuJoCo 自带的查看器可以录屏；没有图形界面的话，把结果曲线做成动画也可以。

**代码的基本卫生**：依赖写版本号；固定随机种子；结果能复现；仓库里不放大文件和密钥；加一个开源许可证。

> **考官视角｜我们怎么看作品集**
>
> 我们最常问的不是“你做了什么”，而是“你为什么这么做”“结果和你预想的一样吗”“不一样的地方你怎么解释”。所以 README 里最好写一句“我原本以为……，结果发现……，原因是……”。这一句话比整个仓库的代码更能说明你会思考。

**动手试试｜给一个项目写 README**

挑你跑过的一个项目，照上面的六段写一份 README，把结果图放在最上面，发给一位不懂机器人的朋友看。对方看完能说出“这个项目发现了什么”，就合格了。

---

**本章一句话**

**一个讲得清楚的项目，胜过三个只跑通的项目。失败的实验也是结果，讲清楚为什么失败，同样有说服力。**

---

作品集准备好了，下一章把它和你过去的经历一起，写进一份机器人公司读得懂的简历。

第 13 章

# 简历：把过去的经验翻译成机器人公司的话

转行的人写简历，最常见的问题不是经历不够，而是**说的是上一个行业的话**。

一位做了五年电商后端的工程师，简历上写“负责订单系统的开发和维护”。机器人公司的面试官读完，脑子里只有一个问号：这和我们有什么关系？同样的经历换一种写法：“把一个高并发服务的 P99 延迟从 120 毫秒降到 40 毫秒，定位到的原因是锁竞争和垃圾回收停顿”。面试官立刻会想到：这个人能帮我们查控制进程偶发卡顿的问题。

这一章讲怎么把你的经历翻译成机器人公司听得懂的话：一页简历的结构，六种背景的改写对照，项目经历的写法，以及最容易犯的几个错。

## 一页简历的结构

社招三到十年经验，一页最好，最多两页；应届生一页。从上到下：

1. **头部**：姓名、联系方式、**目标岗位**（比如“机器人 SLAM 工程师”）。目标岗位写在最显眼的地方，让筛简历的人一眼知道你投的是什么。
2. **一句话总结**：你是谁、能解决什么问题。比如“4 年自动驾驶多传感器融合定位经验，熟悉激光雷达、相机、IMU 的标定和紧耦合定位，希望把它用在腿足机器人上”。
3. **技能**：按目标岗位的重要性排序，用招聘启事里的原词（第 3 章 `jd_check.py` 统计出来的那些）。很多公司用系统按关键词初筛，原词对不上就可能被漏掉。
4. **工作经历**：每段经历 3–5 条，每条写**做了什么、怎么做的、结果是什么**，尽量带数字。
5. **项目经历**：作品集项目放在这里，每个项目写问题、做法、结果和链接。
6. **教育、其他**（竞赛、论文、开源贡献、专利）。

## 六种背景的改写对照

下面的例子都是虚构的，用来演示“同样的经历怎么换一种说法”。改写的原则只有一条：**只换说法，不加内容**。没做过的事不能写，面试会被追问穿。

**互联网后端 → 机器人系统软件**

改写前：负责订单系统后端开发，使用 Go 和 Kafka。

改写后：设计并维护日均数千万请求的服务，用 Go 和 Kafka 搭消息管线；排查一次偶发超时，定位到锁竞争和垃圾回收停顿，P99 延迟从 120 ms 降到 40 ms。

翻译的关键：机器人公司关心实时性、稳定性和问题定位能力，就把这几样突出出来。

**推荐或 CV 算法 → 具身大模型、数据评估**

改写前：负责推荐模型的迭代优化。

改写后：负责多模态推荐模型的训练和评测，搭建从日志到训练样本的数据管线；发现离线指标和线上效果不一致，设计了分层抽样的评测集，离线指标和线上结果的相关性明显提高。

翻译的关键：具身大模型团队头疼的正是“数据”和“评测和真实效果对不上”。

**自动驾驶感知 → SLAM、定位导航**

改写前：负责感知模块开发。

改写后：负责激光雷达、相机、IMU 的外参标定和时间同步，把多传感器时间偏差从十几毫秒降到 2 毫秒以内；参与紧耦合激光惯性定位模块，在城市道路和隧道场景完成数千公里路测。

翻译的关键：标定、同步、退化场景（隧道）这些词，腿足机器人的感知团队一看就懂。

**新能源三电、电控 → 关节电控、嵌入式**

改写前：从事电机控制器软件开发。

改写后：负责永磁同步电机 FOC 软件，电流环 10 kHz；设计过温降额策略，把高温工况下的保护停机次数降到零；参与的控制器累计量产超过 10 万台。

翻译的关键：机器人关节和汽车电驱是同一套技术，量产规模和可靠性是你最大的卖点。

**手机、家电结构 → 机器人结构**

改写前：负责产品结构设计。

改写后：负责整机结构和关键结构件的设计，主导跌落和疲劳测试的改进，把一款产品的跌落不良率从 3% 降到 0.5%；熟悉注塑、压铸、钣金工艺和供应商导入。

翻译的关键：跌落、疲劳、量产不良率，正是机器人结构团队天天在处理的事。

**工业自动化 → 解决方案、技术支持**

改写前：从事自动化项目实施。

改写后：独立负责 12 个产线自动化项目的方案设计、现场调试和客户培训，涉及工业机器人、PLC 和视觉；平均交付周期缩短两周。

翻译的关键：方案、现场、客户、交付周期，就是机器人公司解决方案岗位要的东西。

**应届生**：把课程设计和作品集项目写成“项目经历”，按下一节的格式写，每个项目都带链接；竞赛写清楚名次和你负责的部分。

## 项目经历怎么写

一个作品集项目，按“问题、做法、结果、链接”四句话写。以第 11 章的项目一为例：

**四足机器人关节 PD 增益与控制延迟的仿真分析**（个人项目，2026 年）

- 问题：关节 PD 增益和控制延迟怎样影响四足机器人的站立和抗负载能力。
- 做法：在 MuJoCo 中用 Go2 模型实现 1 kHz 关节 PD 控制和可调延迟，对比 6 组增益和延迟组合。
- 结果：kp 从 10 增到 300，站立高度从 0.15 m 到 0.27 m，负载下沉从 18 mm 降到 2 mm；发现 5 ms 延迟下 kd 从 3 增到 8 会引起约 45 Hz 的抖动和力矩饱和。
- 代码和报告：GitHub 链接。

这四句话，面试官读完就知道你做了什么、做得多深。面试时他会顺着“结果”那一句往下问，你要准备好第 14 章说的追问链。

## 最容易犯的六个错

1. **堆技能名词**：写了二十个框架和算法，没有一个能在经历里找到证据。
2. **写职责不写结果**：“负责……”“参与……”，看不出你做到了什么程度。
3. **新学的东西写满一页，老本行写成一行**：第 9 章说过，机器人公司要的往往就是你的老本行。
4. **关键词对不上**：启事写“SLAM”，你写“建图定位算法”，系统可能匹配不上。两个都写。
5. **一份简历投所有岗位**：投 SLAM 岗和投系统软件岗，技能和经历的排序应该不一样。
6. **夸大**：把“参与”写成“主导”，把“跑过示例”写成“做过项目”。面试官追问三层就露馅，而且会怀疑简历上的其他内容。

> **考官视角｜面试官怎么读简历**
>
> 面试官拿到简历，通常先看目标岗位和最近一段经历，再找一两个最有意思的点准备追问。所以简历上的每一句话都要经得起追问；与其写十条普通的，不如写三条你能讲二十分钟的。

**转行者笔记｜一个方向一份简历**

你在第 2 章选了两个目标岗位，那就准备两份简历，技能排序、一句话总结、项目的侧重都不一样。用一份简历同时投 SLAM 和强化学习，两边看起来都不够专业。

**动手试试｜改写一版简历**

1. 选你最重要的一段工作经历，按“做了什么、怎么做的、结果是什么”改写三条，每条带一个数字；
2. 把第 3 章差距清单里你已经具备的技能，用启事里的原词写进技能栏；
3. 加一个作品集项目，按“问题、做法、结果、链接”四句话写。

附录 D 有一份完整的简历模板。

---

**本章一句话**

**简历要回答一个问题：“你能解决我们的哪个问题?”只换说法，不加内容。**

---

简历投出去以后，就是面试。下一章讲从笔试到 HR 面的每一轮。

第 14 章

# 面试：从笔试到 HR 面

面试不是考试，是双向的工作试用：面试官在判断“这个人进来以后能不能干活、好不好合作”，你也在判断“这个团队在做什么、我去了能学到什么”。

这一章按一场典型的技术面试流程走一遍：每一轮考什么、怎么准备、常见的坑，最后给一张复盘表。面试最快的进步方法不是多刷题，而是每面一次就认真复盘一次。

## 一般有几轮

机器人公司的社招，常见的是三到五轮：

1. **简历筛选**，有时加一个电话初筛；
2. **笔试或在线编程**（不是每家都有）：一两道算法题，偶尔有机器人相关的小题；
3. **技术一面**：基础知识加项目，常常要现场写代码；
4. **技术二面**：深挖项目、系统设计，看你的上限；
5. **主管面或交叉面**：方向匹配、潜力、团队合作；
6. **HR 面**：动机、稳定性、薪资期望。

校招流程类似，笔试更常见，项目深挖的重点是课程设计、实验室项目和竞赛。

## 手撕代码

两类题：

- **通用算法题**：数组、字符串、链表、树、图、动态规划，难度多在 LeetCode 中等左右。这类题靠练，每天一道，坚持两个月效果最明显。
- **机器人味的题**：四元数乘法和插值、旋转矩阵和欧拉角互转、齐次变换求逆、带抗积分饱和的 PID（第 4 章 4.12 题）、环形缓冲区、线性插值和样条、点到

线段的距离、简单的 A* 路径搜索。这类题网上的题库里少，但机器人公司很爱考，因为能同时看出编程和机器人常识。

写代码时的三个习惯：**先说思路再动手**；**边写边说**，让面试官知道你在想什么；**写完主动测**：说出几个边界情况（空输入、四元数接近反向、除以零），手动走一遍。

## 八股：背还是理解

第 4 到 10 章的面试题，就是机器人公司的“八股”。准备的方法：

1. 先自己答一遍，写下来；
2. 对照参考答案，补上缺的点，尤其是“考官想听到”的那几条；
3. 用自己的话再讲一遍，讲给朋友听，或者录下来听；
4. 能联系自己的经历就联系：“这个问题我在项目三里碰到过”。

背出来的答案，面试官一追问就散；理解了的答案，换个问法也能答上来。

## 项目深挖：准备好追问链

二面的大部分时间会花在一两个项目上。面试官会顺着你的回答一层层往下问，直到问到你答不上来为止。这不是刁难，是在找你的能力边界。

以第 11 章的项目三为例，一条典型的追问链：

1. “为什么用 PPO，不用 SAC?”（第 6 章 6.3 题）
2. “超参数是怎么定的？改过哪些?”
3. “为什么只跑 3 个种子？方差这么大，结论可靠吗?”
4. “随机化的范围是怎么定的？太宽会怎样?”
5. “如果要把这个策略用到真机上，还差什么?”（6.8 题）
6. “你会怎么设计 Sim2Real 的验证流程?”

准备的方法：给你简历上的每个项目，写一条五到六层的追问链，每一层都准备好答案。答不上来的地方，要么回去补，要么面试时坦白说“这一点我还没有验证，我的猜测是……，验证的办法是……”。后一种回答，往往比硬编一个答案得分高。

## 系统设计题

二面常有一道开放的设计题，比如：

- “设计一个四足机器人的状态估计模块。”
- “设计一套遥操作数据采集系统。”
- “机器人要在商场里自主巡逻，软件架构怎么设计?”

这类题没有标准答案，考的是思路。一个好用的框架：

1. **问清需求**：用在什么场景？精度要求多少？频率多高？有哪些传感器和算力？
2. **画出模块和接口**：输入输出是什么，谁调用谁；
3. **数据流和频率**：每个模块多快，延迟预算怎么分；
4. **失败模式**：传感器坏了、通信断了、算法发散了，系统怎么办；
5. **怎么测试和验证**：仿真、回放、真机测试分别测什么。

以“设计四足机器人的状态估计模块”为例：先问清楚要输出什么（姿态、速度、位置）、给谁用（运控要 500 Hz 以上、低延迟）；模块是“IMU 预测加腿式里程计更新的误差状态卡尔曼滤波，外加激光定位的低频校正”；失败模式包括打滑（卡方门限）、IMU 饱和（剧烈冲击时）、激光定位跳变；验证用回放数据和真机的动捕真值对比。第 7 章和第 12 章的项目四把其中的核心都讲过了。

## 行为面

常见问题和回答思路：

- **为什么想转到机器人行业?** 别只说“看好行业”。说清楚你过去的哪段经验让你对这个方向感兴趣，你已经为此做了什么（项目、学习），你的经验能为对方解决什么问题。
- **为什么选我们公司?** 提前看对方的产品、公开的技术分享和招聘启事，说出具体的点。
- **讲一次失败的经历**。用“情境、任务、行动、结果”四段讲，重点放在你学到了什么、后来怎么做的。
- **和同事意见不一致时怎么办?** 讲一个真实的例子：怎么用数据或实验说服对方，或者被对方说服。
- **你的职业规划?** 和对方的岗位能接上，有长期方向，不空洞。

## 反问：最后十分钟别浪费

面试最后通常会问“你有什么想问的”。这是你了解团队的最好机会，也会影响面试官对你的印象。值得问的问题：

1. 团队现在最想解决的问题是什么?

2. 新人进来前三个月一般做什么?
3. 团队有多少台真机?做一次真机实验的流程是什么样的?
4. 一次模型或软件的迭代周期多长?
5. 这个岗位和其他团队（比如运控和感知、算法和数据）怎么配合?

少问一上来就能在官网查到的问题，薪资和福利留到 HR 面再谈。

## 每面一次，复盘一次

面完当天就写复盘，别等到第二天。附录 D 有一张复盘表，核心是五栏:

表 14-1 面试复盘表的格式

| 题目 | 我当时怎么答的 | 卡在哪里 | 正确或更好的答案 | 下次怎么说 |
|---|---|---|---|---|
| 例：kd 调大会怎样 | 说越大越稳 | 没想到延迟 | 有延迟时 kd 太大会抖（5.1 题） | 先说理想情况，再说延迟和噪声的副作用 |

连续面五六家以后，你会发现同样的问题反复出现。复盘过的问题，第二次就能答好。

> **考官视角｜面试官最怕的两种人**
>
> 一种是什么都说“了解”，但每个都说不深；另一种是答不上来就硬编。面试官更愿意要一个能说清楚“这个我会，那个我不会，但我知道怎么去学”的人。承认不会，再说出你的思路，是加分项。

**转行者笔记｜先面几家“练手”**

投递时把公司分成三批：第一批是不太想去、但岗位对口的，用来练手；第二批是比较想去的；第三批是最想去的。前两批积累的复盘，能让你在最想去的公司发挥最好。

**动手试试｜做一次模拟面试**

找一位朋友，把第 4 章和你目标岗位那一章的题目交给对方随便挑，请对方面你 45 分钟：20 分钟基础题、15 分钟讲一个项目、10 分钟反问。面完按表 14-1 复盘。没有朋友帮忙，就自己录音，对着题目讲一遍再回听。

**本章一句话**

**面试是双向的。准备好追问链，答不上来就坦白说出思路，每面一次复盘一次。**

面试通过以后，你可能会同时拿到几个 offer，也可能只有一个。下一章讲怎么看一家公司，怎么比较和谈 offer。

第 15 章

# 选公司和谈 offer

拿到 offer 是好事，但对具身智能这样一个热闹的行业，选错公司的代价可能比没拿到 offer 还大：入职半年，公司融不到下一轮钱，项目停了，你的简历上多了一段很难解释的经历。

这一章讲两件事：**先看公司能活多久，再看给多少钱**。这里不评价任何一家具体的公司，只给你一套自己判断的方法。

## 先看公司能活多久

一家机器人公司能不能活下去，主要看四样东西。

**1. 钱：融资和现金流**

公开渠道能查到大部分公司的融资轮次、时间和金额（媒体报道、企业信息查询网站）。看两点：最近一轮是什么时候；按公司的人数粗算，这笔钱大概能撑多久。一家三百人的公司，人均每年的人力加研发开支按几十万元算，一年就要花掉一两个亿。融资两年没有新进展，就要多问一句。

**2. 产品：有没有在卖，卖给谁**

只有演示视频、没有交付的公司，和已经批量交付的公司，风险完全不同。问清楚：现在卖出去多少台？主要客户是谁（科研、教育、展示，还是真正在干活的工业客户）？有没有回头客？

**3. 生意：能不能赚钱**

上市公司可以直接读招股书和年报。《拆解宇树》第 16 章示范过怎么读：看收入和销量的增长、平均单价的变化、毛利率、成本结构。还没上市的公司拿不到这些数据，就在面试里问：产品的毛利大概在什么水平？今年的收入目标是多少？面试官不一定会答，但答不答、怎么答，本身就是信息。

**4. 人和方向：团队稳不稳，方向变不变**

核心团队的背景，最近一年有没有核心人员离开；公司的技术路线和产品方向，一年里是不是换了好几次。方向频繁变，对你意味着做的东西随时可能作废。

> **几个值得警惕的信号**
>
> 面试流程混乱、反复改时间；口头承诺很多期权，但不愿意写进 offer；岗位职责说不清楚，“进来再看”；只谈融资和估值，不谈产品和客户。单独一个信号说明不了什么，几个一起出现就要小心。

## 把 offer 拆开看

一份 offer 通常包括这些部分，逐项问清楚：

表 15-1　offer 要问清楚的几项

| 项 | 要问清楚的 |
| --- | --- |
| 月薪和薪数 | 月薪多少，一年发几个月（13 薪、15 薪、16 薪差别很大） |
| 年终奖 | 是写进合同的保底，还是看绩效；去年实际发了几个月 |
| 签字费 | 有没有；入职多久内离职要退回 |
| 期权或股票 | 数量、行权价、归属期（常见是 4 年、满 1 年才开始归属）、公司总股本或最近一轮估值、离职时怎么处理 |
| 社保公积金 | 按什么基数交；公积金比例 |
| 试用期 | 多长；试用期工资打不打折 |
| 竞业限制 | 有没有；范围、期限和补偿 |

期权最容易被高估。粗算一下：你的期权数量 ÷ 公司总股本 × 最近一轮估值，再减去行权成本，得到的是**纸面价值**，而且只有公司上市或被收购时才可能兑现。把它当成“可能有、可能没有”的奖励，别当成工资的一部分。

两条和法律有关的常识（以《劳动合同法》为准）：**试用期**有上限，三年以上的劳动合同，试用期不能超过六个月；**竞业限制**的期限不能超过两年，而且公司在

竞业限制期间要按月给你经济补偿。签之前看清楚竞业限制的范围，避免以后换工作时整个行业都不能去。

## 怎么谈

1. **先了解行情**：第 1 章的薪资例子、身边朋友的情况、几家公司给你的报价。
2. **先让对方报价**：HR 问你期望薪资时，可以说“我更看重岗位和发展，想先听听公司的方案”。
3. **谈总包，不只谈月薪**：月薪谈不上去，可以谈签字费、薪数、期权、职级。
4. **用真实的竞争 offer**：手上有别家的 offer，可以如实告诉对方；不要编造。
5. **书面确认**：所有谈好的条件写进 offer 或者邮件里。

转行的人在谈薪时常常心虚，觉得自己“是来学的”，开价不敢高。机器人公司愿意给你 offer，是因为你能解决他们的问题，按你的价值谈。也要现实一点：从一个高薪行业转过来，第一份工作的月薪持平甚至略降都很常见，用第一年换一个更好的方向是值得的，但要提前想清楚自己能接受的底线（第 1 章“我的起点”表的第 7 行）。

## 比较两个 offer

拿到两个以上 offer 时，用一张表把它们放在一起，给每项打 1–5 分，再按你自己的权重算总分：

表 15-2　offer 比较表（权重按你自己的情况调）

| 项 | 权重 | offer A | offer B |
|---|---|---|---|
| 公司能活多久（钱、产品、生意） | 25% | | |
| 岗位和你的目标方向的匹配度 | 25% | | |
| 能学到什么（团队水平、真机条件） | 20% | | |
| 现金收入（月薪 × 薪数 + 保底年终） | 20% | | |
| 其他（城市、通勤、加班强度、期权） | 10% | | |

权重没有标准答案。工作三年、想换方向的人，可以把“匹配度”和“能学到什么”加大；有房贷、家里需要稳定收入的人，可以把“公司能活多久”和“现金收入”加大。

**考官视角｜HR 面也在看你**

HR 面问“你为什么离开上一家”“你手上还有别的 offer 吗”“你最看重什么”，是在判断你的稳定性和动机。如实回答，说清楚你的考虑。一边说“最看重成长”、一边只追问薪资，会让人觉得你说的和想的不一样。

**转行者笔记｜第一份机器人工作，看“能学到什么”**

对转行的人来说，第一份机器人工作最重要的是让你真正接触到机器人：有没有真机、能不能参与从设计到测试的完整过程、身边有没有懂行的人带。这些比多几千块月薪更能决定你两年后的位置。

**动手试试｜用表 15-2 比一次**

哪怕你现在只有一个 offer，或者一个都没有，也可以拿两家你最想去的公司试着填一遍表 15-2。填不出来的格子，就是你下次面试要问的问题。

**本章一句话**

**先看公司能活多久，再看给多少钱。期权按“可能有”算，竞业限制签之前看清楚。**

最后一章，把前面的所有内容排成一张 90 天的行动计划。

第 16 章

# 90 天行动计划

前面十五章讲了这一行的岗位、每个岗位的能力和面试题、作品集、简历、面试和 offer。这一章把它们排进 90 天，变成每周要做的事。

90 天不长，不够你从零学会机器人学。但它足够做完这几件事：选定方向，补上最要命的差距，做出两个能讲清楚的项目，改好简历，投出去，面上几家。很多人卡在“一直在学、一直没投”，这一章就是为了打破这种状态。

## 两套节奏

先估算你每周能拿出多少时间（第 1 章“我的起点”表的第 8 行）：

- **在职转行**：每周 10–15 小时，工作日每天 1–1.5 小时，周末一天；
- **在读学生或全职准备**：每周 25–30 小时。

下面的计划按在职转行的节奏写。时间更多的人，可以把每个阶段压缩，或者多做一个项目。

## 第 1–2 周：定方向，搭环境

表 16-1　第 1–2 周

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 填“我的起点”表 | 一页纸 | 第 1 章 |
| 给候选岗位打分，选出两个 | 两个目标岗位 | 第 2 章 |
| 收集 20 条以上启事，跑差距清单 | 前十项差距，分进三个筐 | 第 3 章 |

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 装好环境，跑通一个最简单的项目 | 项目二的结果 | 第 11 章 |
| 做 30 分钟基础自测 | 自己的薄弱点清单 | 第 4 章 |

这两周最重要的产出是**两个目标岗位**和**一份差距清单**。后面的每一件事都围着它们转。

## 第 3–6 周：补核心，做第一个项目

表 16-2　第 3–6 周

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 学目标岗位那一章的知识地图里的“必会” | 每周读完一个主题，写一页笔记 | 第 5–10 章 |
| 每周自答 10 道面试题 | 自己写的答案，对照参考答案改 | 第 4–10 章 |
| 做第一个作品集项目，带一组对照实验 | 代码、结果图、三句话结论 | 第 11–12 章 |
| 写 README | 一份别人看得懂的 README | 第 12 章 |

第一个项目选和目标岗位最近的那一个。不要追求复杂，先保证有一组完整的对照、一张图、一个讲得清楚的结论。

## 第 7–9 周：第二个项目，简历，刷题

表 16-3　第 7–9 周

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 做第二个项目，或者把第一个做深一层 | 第二份 README | 第 11–12 章 |
| 每天一道算法题 | 三周约 20 道 | 第 14 章 |

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 写两份简历（每个目标岗位一份） | 简历第一版 | 第 13 章 |
| 给每个项目写追问链 | 每个项目五六层问答 | 第 14 章 |

第 9 周末，请一位懂行的朋友（或者前同事）看一遍简历，再做一次模拟面试。

## 第 10–11 周：投递

表 16-4　第 10–11 周

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 投第一批 10 家“练手”的公司 | 投递记录表 | 第 14 章 |
| 每次面试后当天复盘 | 复盘表 | 第 14 章、附录 D |
| 根据复盘改简历、补知识 | 简历第二版 | 第 13 章 |
| 投第二批 10–15 家 | 投递记录表 | |

投递渠道：公司官网和公众号、招聘平台、内推（前同事、校友、技术社区里认识的人）。内推的回复率通常最高。

## 第 12–13 周：面试，谈 offer

表 16-5　第 12–13 周

| 要做的事 | 产出 | 对应章节 |
| --- | --- | --- |
| 投最想去的几家 | | |
| 继续面试和复盘 | 复盘表 | 第 14 章 |
| 拿到 offer 后逐项问清楚，用比较表比较 | offer 比较表 | 第 15 章 |

## 卡住了怎么办

90 天里一定会卡住。最常见的四种情况：

1. **学不完**：知识地图里的“必会”永远学不完。规则是：一个主题学到能答出对应的面试题、能在项目里用上，就停，进下一个。
2. **项目跑不通**：环境问题先查版本；算法问题先缩小规模（更短的训练、更简单的场景）；一个问题卡住超过两天，换一个项目，回头再看。
3. **投了没回音**：先看简历（第 13 章的六个错），再看投的岗位是不是和你的经历差太远。两周投 20 家没有一个面试，多半是简历的问题。
4. **面了没过**：看复盘表，如果总是倒在同一类问题上（比如项目深挖），就集中补这一块。

还有一种情况：**90 天到了，还没拿到 offer**。这很正常，尤其是在行业招聘放缓的时候。这时你已经有了两个项目、一份改好的简历、一堆复盘，比 90 天前强得多。把计划再循环一轮，第二轮会快很多。

**转行者笔记｜每周有产出，比每天学习更重要**

很多人的问题不是不努力，是一直在“输入”：看课、看论文、看别人的面经，却没有任何输出。按上面的表，每周都要有一个看得见的产出：一页笔记、一张结果图、一版简历、一份复盘。产出才能证明你在前进，也才能拿去给别人看。

**动手试试｜把计划写进日历**

现在就打开日历，把接下来两周的事写进去，具体到哪天晚上做什么、周末做什么。再设一个第 14 天的提醒：“检查：两个目标岗位和差距清单出来了吗?”

---

**本章一句话**

**90 天，选定方向、补上差距、做出两个项目、投出去。每周有产出，卡住了就换个办法，第一轮不成就再来一轮。**

---

还剩一个绕不开的问题：如果这一波热度过去了，你学的东西还有没有用?

# 结语：如果这一波降温了

写这本书的时候，具身智能的招聘正热。热的东西都会降温，区别只在什么时候、降多少。

上一个例子离得不远。2017 年前后，自动驾驶是最热的方向，公司一家接一家地成立，算法工程师被高薪抢来抢去。几年以后，行业开始收缩：2022 年，福特和大众投资的自动驾驶公司 Argo AI 宣布关闭，很多公司裁员、合并、转向辅助驾驶。可那一波里认真打下基础的人，后来大多去了智能驾驶、机器人、无人机这些地方，有的就是今天具身智能公司里的骨干。

所以这一章不预测热度能持续多久，只回答一个问题：**万一降温了，你在这本书里学的东西还有没有用？**

## 哪些能力在别处也值钱

表　降温以后，这些能力还能用在哪

| 你在这本书里补的能力 | 降温以后还能去哪 |
| --- | --- |
| 运动控制、状态估计（第 5、7 章） | 工业机器人、协作机械臂、无人机、汽车的底盘和线控 |
| 强化学习、模仿学习（第 6、8 章） | 自动驾驶的规划、游戏 AI、推荐和决策系统 |
| 感知、标定、SLAM（第 7 章） | 智能驾驶、扫地机器人、AR/VR、测绘 |
| 关节、电控、嵌入式（第 9 章） | 新能源汽车电驱、伺服驱动、家电、工控 |
| 系统软件、数据、测试、交付（第 10 章） | 一切做硬件产品的公司 |

你会发现，这本书里八成以上的内容和“具身智能”这个词本身关系不大，它们是机器人工程的地基。人形机器人火不火，工厂里的机械臂、仓库里的搬运车、天上的无人机都要有人做。

## 怎么给自己留后路

1. **选岗位时想一想可迁移性**：同样是进具身公司，做关节电控、状态估计的人，换行业比只会调某一个框架的人容易得多。
2. **把作品留在外面**：作品集、技术博客、开源贡献，跟着你走，不跟着公司走。
3. **认识同行**：前同事、技术社区、开源项目里认识的人，是下一份工作最可靠的来源。
4. **财务上留余量**：转行的第一年，手里最好留出半年左右的生活费，这样面对一家不靠谱的公司，你才有底气说不。
5. **学原理，不只学工具**：仿真器和框架几年一换，动力学、控制、概率和优化不会过时。

## 回头看

这本书从一条看不懂的招聘启事开始：你在第 1 章看清了谁在招人，在第 2 章找到了自己在地图上的那一格，在第 3 章得到了自己的差距清单。第二部按岗位补了能力，第三部做了作品集、改了简历、准备了面试，最后排出了 90 天。

如果只记住一句话，那就是：**你学的是机器人工程，不是一个风口**。

## 最后

如果这本书帮到了你，欢迎在你买书的地方留下真实的评价，告诉后来的人它对谁有用、对谁没用。书里有错，或者某个数字过时了，也欢迎指出来，勘误和每年的数据更新会放在随书代码的仓库里，地址以出版时公布的为准。

想更深地了解机器人本身是怎么做出来的，可以读姊妹篇《拆解宇树》。

祝你拿到想要的 offer。更重要的是，希望几年以后你回头看，会庆幸自己当初选了这一行。

附录 A

# 面试题索引

第 4 到 10 章的全部面试题，按岗位排列。PDF 版每题后面是页码，电子书可以直接点题号跳过去。

## 第 4 章　所有岗位都会问的基础


- 4.1　旋转矩阵、欧拉角、四元数、轴角，各有什么优缺点？什么是万向锁？ 第 24 页
- 4.2　两个坐标系之间的变换怎么表示？$T_{AB}$ 的逆是什么？怎么把相机看到的点变到机器人基座坐标系？ 第 25 页
- 4.3　两个姿态之间怎么插值？为什么不能直接对四元数做线性插值？ 第 25 页
- 4.4　什么是雅可比矩阵？在机器人里有什么用？什么叫奇异？ 第 25 页
- 4.5　最小二乘怎么解？什么时候不该用正规方程？ 第 26 页
- 4.6　C++ 的 unique_ptr、shared_ptr、weak_ptr 有什么区别？什么情况下还会内存泄漏？ 第 26 页
- 4.7　std::move 做了什么？移动以后原来的对象还能用吗？ 第 27 页
- 4.8　互斥锁、条件变量、原子变量怎么选？实时控制线程里为什么尽量不加锁？ 第 27 页
- 4.9　一个 1 kHz 的控制循环，怎么保证它真的每 1 ms 跑一次？怎么测？ 第 28 页
- 4.10　ROS 2 的话题、服务、动作有什么区别？QoS 的 reliable 和 best effort 怎么选？ 第 28 页
- 4.11　DDS 是什么？为什么同一网段里两套 ROS 2 程序会“串台”？怎么隔离？ 第 29 页

- 4.12 手写一个带抗积分饱和的 PID 控制器 第 29 页


## 第 5 章 运动控制岗：从 PID 到 MPC 和 WBC


- 5.1 关节 PD 控制里的 kp 和 kd 分别像什么？调大调小各会怎样？ 第 34 页
- 5.2 四足机器人为什么常用“PD 加前馈力矩”，而不是纯位置控制或者纯力矩控制？ 第 35 页
- 5.3 线性倒立摆模型怎么推？时间常数是多少？捕获点是什么？ 第 35 页
- 5.4 ZMP 是什么？它在什么情况下不适用？ 第 35 页
- 5.5 四足机器人的 MPC 一般怎么建模？为什么用单刚体模型？ 第 36 页
- 5.6 摩擦锥约束怎么写？为什么常常线性化成“摩擦金字塔”？ 第 36 页
- 5.7 WBC 是什么？它和 MPC 是什么关系？任务优先级怎么实现？ 第 37 页
- 5.8 LQR 和 PID 有什么区别？LQR 的 Q 和 R 怎么调？ 第 37 页
- 5.9 控制频率怎么选？延迟对稳定性有什么影响？ 第 38 页
- 5.10 真机上一条腿在高频抖动，你怎么排查？ 第 38 页
- 5.11 摆动腿和支撑腿的控制有什么不同？落脚点怎么选？ 第 39 页
- 5.12 让你从零给一台四足写一个能站立、能慢走的控制器，你会怎么分步做？ 第 39 页


## 第 6 章 强化学习岗：奖励、PPO 和 Sim2Real


- 6.1 PPO 的核心思想是什么？“截断”在防什么？ 第 44 页
- 6.2 GAE 是什么？$\lambda$ 怎么影响偏差和方差？ 第 45 页
- 6.3 on-policy 和 off-policy 有什么区别？为什么足式机器人训练多用 PPO？ 第 45 页
- 6.4 训练四足走路，奖励函数一般有哪些项？ 第 45 页
- 6.5 什么是奖励作弊？举个例子，怎么发现？ 第 46 页
- 6.6 动作空间为什么常用“关节目标角度加底层 PD”，而不是直接输出力矩？ 第 46 页
- 6.7 什么是域随机化？随机化哪些参数？范围怎么定？ 第 47 页
- 6.8 Sim2Real 的差距通常来自哪里？怎么缩小？ 第 47 页
- 6.9 什么是教师-学生训练（特权学习）？为什么有效？ 第 48 页
- 6.10 为什么要几千个环境同时训练？并行环境数怎么影响训练？ 第 48 页
- 6.11 训练曲线不涨，或者涨了又崩了，你怎么排查？ 第 48 页

- 6.12 策略在 Isaac 里走得很好，放到 MuJoCo 里就站不住，可能是什么原因? 第 49 页


## 第 7 章 感知与定位岗：标定、SLAM 和状态估计


- 7.1 相机的内参、外参和畸变各是什么？怎么标定? 第 52 页
- 7.2 手眼标定是什么？$AX = XB$ 是怎么来的? 第 53 页
- 7.3 IMU 的误差模型是什么？零偏怎么估计? 第 53 页
- 7.4 卡尔曼滤波的预测和更新各做什么？Q 和 R 怎么调? 第 53 页
- 7.5 EKF 和误差状态卡尔曼有什么区别？姿态为什么常用误差状态? 第 54 页
- 7.6 腿式里程计怎么算？脚打滑了怎么办? 第 54 页
- 7.7 ICP 的原理是什么？什么时候会失败? 第 55 页
- 7.8 SLAM 里为什么需要回环检测？回环错了会怎样? 第 55 页
- 7.9 多传感器的时间同步怎么做？不同步会怎样? 第 55 页
- 7.10 腿足机器人的地形感知为什么常用 2.5D 高度图？它有什么局限? 第 56 页


## 第 8 章 操作与具身大模型岗：模仿学习、VLA 和数据


- 8.1 行为克隆为什么会“误差累积”？DAgger 怎么解决? 第 60 页
- 8.2 什么是示范的“多峰”问题？为什么均方误差回归会出事? 第 61 页
- 8.3 扩散策略（Diffusion Policy）是什么？为什么在机器人上效果好? 第 61 页
- 8.4 ACT 的“动作分块”解决什么问题? 第 62 页
- 8.5 VLA 模型一般由哪几部分组成？动作是怎么输出的? 第 62 页
- 8.6 遥操作采数据有哪些方式？各有什么优缺点? 第 63 页
- 8.7 怎么判断一批示范数据的质量？哪些该丢掉? 第 63 页
- 8.8 真机上测 50 次，新模型成功率 42%、旧模型 38%，能说明新模型更好吗? 第 64 页
- 8.9 什么是数据配比？为什么要混入互联网数据或者别的机器人的数据? 第 64 页
- 8.10 VLA 模型部署到机器人上，推理太慢怎么办? 第 64 页

## 第 9 章　硬件岗：结构、关节、电控和嵌入式


- 9.1　永磁同步电机的扭矩和电流是什么关系？发热主要来自哪里？ 第 68 页
- 9.2　FOC 的基本流程是什么？为什么要做 Clarke 和 Park 变换？ 第 69 页
- 9.3　电流环、速度环、位置环的带宽怎么安排？ 第 69 页
- 9.4　腿足机器人的关节为什么偏爱低减速比？减速比怎么选？ 第 70 页
- 9.5　编码器有哪些类型？为什么关节常用两个编码器？ 第 70 页
- 9.6　CAN、CAN FD、EtherCAT 怎么选？一条 CAN 总线能挂几个关节、跑多高的频率？ 第 70 页
- 9.7　RTOS 和 Linux 在机器人里分别干什么？中断服务程序里不能做什么？ 第 71 页
- 9.8　机器人结构怎么做轻量化？跌落和冲击怎么考虑？ 第 71 页
- 9.9　一个关节连续爬楼梯 20 分钟后过热保护，你怎么分析和解决？ 第 72 页
- 9.10　样机好好的，量产以后不良率很高，你怎么查？ 第 72 页


## 第 10 章　软件、数据、测试和交付岗


- 10.1　（系统软件）机器人的日志和回放系统怎么设计？ 第 76 页
- 10.2　（系统软件）控制进程崩溃了，机器人会怎样？看门狗和安全状态怎么设计？ 第 76 页
- 10.3　（数据管线）每天几百小时的多模态机器人数据，怎么存、怎么检索、怎么喂给训练？ 第 76 页
- 10.4　（数据采集）采集现场有哪些常见问题？怎么保证数据一致？ 第 77 页
- 10.5　（AI Infra）数据并行和模型并行有什么区别？训练一个几十亿参数的 VLA 要注意什么？ 第 77 页
- 10.6　（测试）给一台四足机器人写测试计划，你会包括哪些测试？ 第 78 页
- 10.7　（测试）什么是故障注入？为什么重要？ 第 78 页
- 10.8　（技术支持）客户说“机器人在我们仓库里老是定位丢失”，你怎么处理？ 第 79 页
- 10.9　（产品经理）如果让你定义一款面向高校科研的四足机器人，你会怎么定配置？ 第 79 页
- 10.10　（安全）机器人产品常见的安全风险有哪些？怎么防？ 第 80 页


共 76 道题。

附录 B

# 术语表

按书里第一次出现的顺序大致分组。括号里是常见的英文写法或缩写。

| 术语 | 英文 | 一句话解释 | 章 |
|---|---|---|---|
| 具身智能 | Embodied AI | 有身体、能在物理世界里感知和行动的智能 | 1 |
| 招聘指数 | — | 招聘平台按岗位发布量等算出的相对指标，不是岗位个数 | 1 |
| 齐次变换 | Homogeneous Transform | 用一个 4×4 矩阵同时表示旋转和平移 | 4 |
| 四元数 | Quaternion | 用 4 个数表示三维旋转，没有万向锁 | 4 |
| 万向锁 | Gimbal Lock | 欧拉角在某个角度下丢掉一个自由度 | 4 |
| 雅可比 | Jacobian | 把关节速度映射到末端速度的矩阵 | 4 |
| 奇异位形 | Singularity | 雅可比降秩、末端某个方向动不了的姿态 | 4 |
| 阻尼最小二乘 | Damped Least Squares (DLS) | 在伪逆里加一个阻尼项，奇异附近也不会给出极大的关节速度 | 4、12 |
| 服务质量 | QoS | ROS 2 里描述可靠性、历史深度等通信策略的参数 | 4 |
| 域号 | Domain ID | DDS 的分组编号，不同域之间互相看不见 | 4 |

| 术语 | 英文 | 一句话解释 | 章 |
|---|---|---|---|
| 优先级反转 | Priority Inversion | 高优先级任务被低优先级任务间接卡住 | 4 |
| PD 控制 | Proportional-Derivative | 比例加微分，像在关节上装弹簧和阻尼 | 5 |
| 前馈力矩 | Feedforward Torque | 按模型预先算好、直接加上的力矩 | 5 |
| 线性倒立摆 | LIPM | 假设质心高度不变的简化平衡模型 | 5 |
| 捕获点 | Capture Point | 踩到这里机器人就能停下来的点 | 5 |
| 零力矩点 | ZMP | 地面反力力矩为零的点，在支撑面内脚就不会翻 | 5 |
| 模型预测控制 | MPC | 每个周期优化未来一段时间的控制，只执行第一步 | 5 |
| 全身控制 | WBC | 在完整动力学上把上层目标变成每个关节的力矩 | 5 |
| 摩擦锥 | Friction Cone | 切向力不超过摩擦系数乘法向力的约束 | 5 |
| 线性二次调节器 | LQR | 基于线性模型、最小化状态和控制代价的全状态反馈 | 5、11 |
| 近端策略优化 | PPO | 限制每次更新幅度的策略梯度算法 | 6 |
| 广义优势估计 | GAE | 用多步时序差分误差加权估计优势 | 6 |
| 奖励作弊 | Reward Hacking | 策略钻奖励函数的漏洞，分高但行为不对 | 6 |
| 域随机化 | Domain Randomization | 训练时随机改仿真参数，让策略对变化不敏感 | 6、11 |
| 仿真到真机 | Sim2Real | 把仿真里训练的策略迁移到真机 | 6 |
| 教师-学生训练 | Teacher-Student | 先训练看特权信息的教师，再让只看传感器的学生模仿 | 6 |

| 术语 | 英文 | 一句话解释 | 章 |
|---|---|---|---|
| 手眼标定 | Hand-Eye Calibration | 求相机和机械臂法兰之间的固定变换，$AX = XB$ | 7 |
| 零偏 | Bias | 传感器测量里的固定偏差，会被积分放大 | 7、12 |
| 误差状态卡尔曼 | ESKF | 只对名义状态附近的小误差做滤波 | 7 |
| 腿式里程计 | Leg Odometry | 假设支撑脚不动，用关节运动学算机身速度 | 7、12 |
| 卡方门限 | Chi-square Gating | 新息的马氏距离过大就丢掉这次测量 | 7、12 |
| 迭代最近点 | ICP | 交替找对应点和求刚体变换的点云配准方法 | 7 |
| 回环检测 | Loop Closure | 认出回到了走过的地方，用来消除累积漂移 | 7 |
| 高度图 | Elevation Map | 每个格子存一个地面高度的 2.5D 地图 | 7 |
| 行为克隆 | Behavior Cloning (BC) | 把示范当监督学习的标签来模仿 | 8、12 |
| 分布偏移 | Distribution Shift | 测试时遇到的状态和训练数据不一样 | 8 |
| DAgger | Dataset Aggregation | 让学生自己执行、专家补标注、合进数据重训 | 8 |
| 多峰 | Multimodality | 同一状态下有几种都对的做法 | 8、12 |
| 扩散策略 | Diffusion Policy | 用去噪扩散生成一段动作 | 8 |
| 动作分块 | Action Chunking | 一次预测、执行一段动作 | 8、12 |
| 视觉-语言-动作模型 | VLA | 看图、听指令、输出机器人动作的大模型 | 8 |
| 遥操作 | Teleoperation | 人远程操纵机器人，用来采示范数据 | 8 |

| 术语 | 英文 | 一句话解释 | 章 |
| --- | --- | --- | --- |
| 磁场定向控制 | FOC | 在转子坐标系里分别控制两个电流分量的电机控制方法 | 9 |
| 反射惯量 | Reflected Inertia | 电机惯量折算到关节端，乘以减速比的平方 | 9 |
| 准直驱 | Quasi-Direct Drive | 低减速比、可反驱的关节方案 | 9 |
| 看门狗 | Watchdog | 一段时间收不到信号就触发保护的机制 | 10 |
| 故障注入 | Fault Injection | 故意制造故障，检验系统是否按设计降级 | 10 |
| 竞业限制 | Non-compete | 离职后一定时间内不能去竞争对手处工作的约定 | 15 |

附录 C

# 资源清单

只列经过时间检验、免费或容易买到的。课程和书名用原名，方便搜索。

## 书

- 《机器人学导论》（John J. Craig，*Introduction to Robotics*）：运动学、动力学入门经典。
- *Modern Robotics*（Kevin Lynch、Frank Park）：用旋量理论讲机器人学，网上有免费的预印本和配套视频课。
- *Probabilistic Robotics*（Sebastian Thrun 等）：滤波、定位、SLAM 的经典。
- 《视觉 SLAM 十四讲》（高翔等）：中文 SLAM 入门最常被推荐的一本。
- *Reinforcement Learning: An Introduction*（Richard Sutton、Andrew Barto）：强化学习教材，作者公开了免费电子版。
- 《拆解宇树：从一颗电机到会翻跟头的机器人》：本书的姊妹篇，讲一台腿足机器人从关节到大模型是怎么做出来的。

## 课程

- *Underactuated Robotics*（MIT，Russ Tedrake）：倒立摆、步行、轨迹优化，讲义和视频全部公开。
- *CS 285: Deep Reinforcement Learning*（UC Berkeley，Sergey Levine）：深度强化学习，课程视频公开。
- *Spinning Up in Deep RL*（OpenAI）：强化学习入门文档，配有简洁的参考实现。

## 开源软件和仓库

- **MuJoCo** 和 **MuJoCo Menagerie**（Google DeepMind）：物理仿真器和高质量的机器人模型库，本书项目一用的 Go2 模型就来自这里。
- **Gymnasium**：强化学习环境的标准接口，项目三用的 Hopper 就在里面。
- **CleanRL**：单文件的强化学习算法实现，适合读懂每一行，项目三的 PPO 参考了它的写法。
- **Isaac Lab**（NVIDIA）：GPU 并行仿真和机器人学习框架。
- **legged_gym**、**unitree_rl_gym**、**unitree_rl_lab**：腿足机器人强化学习训练的代表性代码。
- **LeRobot**（Hugging Face）：模仿学习、机器人数据集和低成本机械臂的工具链。
- **ROS 2** 官方文档和教程：话题、服务、动作、QoS、tf2 都有可以照着做的例子。

## 怎么用

不要试图把清单全部看完。第 3 章的差距清单里排在前面的是什么，就去找对应的那一本书或那一门课的那几章。

附录 D

# 模板

这些模板的电子版也放在随书代码的 `代码/模板/` 目录里，可以直接复制使用。

## D.1 一页简历

```
姓名 | 电话 | 邮箱 | GitHub
目标岗位：机器人 SLAM 工程师

一句话：4 年自动驾驶多传感器融合定位经验，熟悉激光雷达、相机、IMU 的标定和融合定位，
        希望把它用在腿足机器人上。

技能（按目标岗位排序，用招聘启事里的原词）
  - 定位与建图：SLAM、多传感器融合、卡尔曼滤波、因子图优化
  - 标定：相机内外参、激光雷达-IMU 外参、时间同步
  - 工程：C++、Python、ROS 2、Linux、PCL、OpenCV

工作经历
  某某公司 | 感知工程师 | 2022.07 –至今
  - 做了什么、怎么做的、结果是什么（带一个数字）
  - ……（3–5 条）

项目经历
  项目名称（个人项目，2026 年）
  - 问题：……
  - 做法：……
  - 结果：……（带数字）
  - 链接：……

教育
  学校 | 专业 | 学位 | 年份
其他
  竞赛、论文、专利、开源贡献
```

## D.2 项目 README

```
# 项目名称：一句话说清楚它回答什么问题

![结果图](results.png)      <- 放在最上面

## 怎么跑
pip install -r requirements.txt
python main.py               # Python 3.13，约 X 分钟

## 做了什么对照
- 变量：……
- 固定不变的：……

## 结论
- 结论一（带数字）
- 结论二（带数字）
- 我原本以为……，结果发现……，原因是……

## 局限和下一步
- ……
```

## D.3 面试复盘表

每次面试当天填一行。

| 日期 | 公司和岗位 | 轮次 | 题目 | 我当时怎么答的 | 卡在哪里 | 正确或更好的答案 | 下次怎么说 |
|---|---|---|---|---|---|---|---|
| | | | | | | | |

## D.4 投递记录表

| 公司 | 岗位 | 渠道（官网、平台、内推） | 投递日期 | 进度 | 下一步 | 备注 |
|---|---|---|---|---|---|---|
| | | | | | | |

## D.5 offer 比较表

见第 15 章表 15-2。权重按你自己的情况调，每项打 1–5 分。电子版 `代码/模板/打分和比较.xlsx` 里有这张表和第 2 章的岗位打分表，填完分数自动算总分。

## D.6 访谈提纲：向已经转进来的人请教

想了解一个岗位真实的样子，最好的办法是找一位已经在做这个岗位的人聊半小时。提前说明用途，聊完表示感谢；对方不愿意谈的话题不追问。

1. 你之前做什么？为什么转到机器人行业？
2. 你现在一天里大部分时间在做什么？
3. 转过来以后，哪些旧技能最有用？哪些新东西补得最辛苦？
4. 你们招人时最看重什么？最常见的让人不通过的原因是什么？
5. 如果重来一次，你会提前准备什么？
6. 对想转过来的人，你有什么建议？

附录 E

# 资料来源

访问日期没有特别写的，都是 2026 年 10 月 30 日。

## 招聘市场

- 脉脉高聘人才智库《2026 春招职场洞察报告》，2026 年 5 月 13 日发布；数据引自每日经济新闻 2026 年 5 月 14 日的报道。
- 脉脉高聘人才智库《Agent 重置职场—— AI 人才流动报告》，2026 年 9 月；数据引自紫牛新闻（扬子晚报）2026 年 9 月 16 日的报道。
- 猎聘大数据研究院《2026 机器人领域人才供需趋势洞察报告》、智联招聘 2026 年一季度机器人行业数据：均为转引，来自钛媒体 2026 年 5 月 16 日刊发的《2026 年具身赛道平均薪酬如何？哪些大牛最抢手？未来人才去哪找?》（作者镧瞳 AI）。
- 教育部《普通高等学校本科专业目录（2026 年）》新增“具身智能”专业的消息：转引自同一篇文章。
- 宇树科技官网“招贤纳士”页面（www.unitree.com/cn/position），2026 年 10 月 30 日的在招岗位清单。
- 猎聘、牛企招聘等平台上 2026 年公开发布的具身智能相关招聘启事：只用于归纳常见要求和薪资区间的例子，不对应具体公司的评价。
- 杭州网 2025 年 2 月 10 日关于宇树科技招聘的报道。
- 开卷《2025 年开卷图书零售市场趋势洞察报告》：数据引自新京报 2026 年 1 月 8 日的报道。

## 论文和技术资料

- Schulman 等，Proximal Policy Optimization Algorithms，2017（PPO）；High-Dimensional Continuous Control Using Generalized Advantage Estimation，ICLR 2016（GAE）。
- Rudin 等，Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning，CoRL 2021（legged_gym）。
- Hwangbo 等，Learning Agile and Dynamic Motor Skills for Legged Robots，Science Robotics 2019（执行器网络）。
- Lee 等，Learning Quadrupedal Locomotion over Challenging Terrain，Science Robotics 2020（教师-学生训练）。
- Kumar 等，RMA: Rapid Motor Adaptation for Legged Robots，RSS 2021。
- Di Carlo 等，Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control，IROS 2018。
- Ross、Gordon、Bagnell，A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning，AISTATS 2011（DAgger）。
- Chi 等，Diffusion Policy: Visuomotor Policy Learning via Action Diffusion，RSS 2023。
- Zhao 等，Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware，RSS 2023（ALOHA、ACT）。
- Shafiullah 等，Behavior Transformers，NeurIPS 2022；Lee 等，VQ-BeT，ICML 2024。
- Brohan 等，RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control，2023。
- Kim 等，OpenVLA: An Open-Source Vision-Language-Action Model，2024。
- Black 等，$\pi$0: A Vision-Language-Action Flow Model for General Robot Control，2024。
- Open X-Embodiment Collaboration，Open X-Embodiment: Robotic Learning Datasets and RT-X Models，2023。
- Zhang，A Flexible New Technique for Camera Calibration，IEEE TPAMI 2000（张正友标定法）。
- Barto、Sutton、Anderson，Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems，1983（小车倒立摆）。

- Universal Robots 公开的 UR5e DH 参数；宇树官网 GO-M8010-6 关节电机参数页。
- CVE-2025-2894 记录（美国国家漏洞库）；IEEE Spectrum 2025 年 9 月关于 UniPwn 漏洞的报道。

## 软件

- MuJoCo 3.17、MuJoCo Menagerie（提交 f3b6013，2026 年 10 月 23 日）、Gymnasium 1.4、PyTorch 2.15、NumPy 2.5、SciPy 1.18。
- 本书项目三的 PPO 写法参考了 CleanRL 的 ppo_continuous_action。